Technische Universität Berlin

Faculty of Electrical Engineering and Computer Science
Dept. of Computer Engineering and Microelectronics
Remote Sensing Image Analysis Group



On Detecting Hallucinations in Vision-Language Model Generated Captions for Remote Sensing Imagery

Master of Science in Computer Science

September, 2026

Filipe Alexandre Fabian Borges
Matriculation Number: 414221

Supervisor: Prof. Dr. Begüm Demir
Advisor: Marvin Beese

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln wörtlich oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.

Sofern KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die Art der Nutzung (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf habe ich zur Kenntnis genommen.

Berlin, den 10. September 2026



Filipe Alexandre Fabian Borges

Abstract

The rapid growth of Remote Sensing (RS) imagery has outpaced manual captioning. Thus, automatic captioning has become essential for data-analysis and retrieval necessary for decision-making. Vision-Language Models (VLMs) can generate rich and descriptive image captions at scale but often produce object hallucinations. This undermines trust and can mislead users in high-stakes applications like climate research and disaster relief. This thesis explores if object hallucinations in RS image captions can be detected and revised.

First, the problem of object hallucination detection is formulated as confidence estimation using object-probing questions answered by five different VLMs. All models detect hallucinations above random chance on the RS-POPE and LID datasets, the latter containing manually labeled hallucinations, with Qwen3.5 achieving the highest ROC-AUC (0.918). Performance drops on real hallucinations and no clear benefit was observed for RS-specific fine-tuning. Second, Visual Contrastive Decoding (VCD) is tested as a method to increase detection performance. The results suggest that VCD does not improve performance. Additionally, VCD seems to bias models towards higher confidence. This bias and greedier decoding may result in inflated scores in certain benchmarks. Third, Semantic Entropy was adapted for RS images. Semantic Entropy seems to be a viable approach for Visual Question Answering (VQA) hallucination detection in RS but is less effective than object-probing for object hallucination detection. Fourth, an object-probing hallucination detection pipeline was applied to Git-10M where the 1200 lowest-object hallucinations were manually evaluated showing a hallucination rate over 90%. Finally, a hallucination reviser based on LURE was fine-tuned using RS imagery, showing small reductions in hallucination rate while preserving text quality.

Zusammenfassung

Das rasante Wachstum der Fernerkundungsbilder (RS) hat die manuelle Bildbeschriftung überholt. Daher ist die automatische Bildbeschriftung für die Datenanalyse und den für die Entscheidungsfindung notwendigen Datenabruf unverzichtbar geworden. Vision-Language-Modelle (VLMs) können umfangreiche und beschreibende Bildbeschriftungen in großem Umfang generieren, produzieren jedoch häufig Objekt-Halluzinationen. Dies untergräbt das Vertrauen und kann Nutzer in Anwendungen mit hoher Risikobelastung wie der Klimaforschung und der Katastrophenhilfe in die Irre führen. Diese Arbeit untersucht, ob Objekt-Halluzinationen in RS-Bildbeschreibungen erkannt und korrigiert werden können.

Zunächst wird das Problem der Erkennung von Objekt-Halluzinationen als Konfidenzschätzung formuliert, wobei fünf verschiedene VLMs Fragen zur Objektüberprüfung beantworten. Alle Modelle erkennen Halluzinationen auf den Datensätzen RS-POPE und LID, die manuell markierte Halluzinationen enthalten, mit einer über dem Zufall liegenden Genauigkeit, wobei Qwen3.5 den höchsten ROC-AUC-Wert (0,918) erzielt. Bei echten Halluzinationen sinkt die Leistung, und es wurde kein klarer Vorteil durch RS-spezifisches Fine-Tuning beobachtet. Zweitens wird Visual Contrastive Decoding (VCD) als Methode zur Steigerung der Erkennungsleistung getestet. Die Ergebnisse deuten darauf hin, dass VCD die Leistung nicht verbessert. Zudem scheint VCD die Modelle in Richtung höherer Konfidenz zu verzerren. Diese Verzerrung und eine „gierigere“ Dekodierung können in bestimmten Benchmarks zu überhöhten Bewertungen führen. Drittens wurde die semantische Entropie für RS-Bilder angepasst. Die semantische Entropie scheint ein brauchbarer Ansatz für die Halluzinationserkennung bei VQA in RS zu sein, ist jedoch bei der Erkennung von Objekt-Halluzinationen weniger effektiv als das „Object-Probing“. Viertens wurde eine auf „Object-Probing“ basierende Pipeline zur Halluzinationserkennung auf Git-10M angewendet, wobei die 1.200 Halluzinationen mit der geringsten Objektdichte manuell ausgewertet wurden und eine Halluzinationsrate von über 90% aufwiesen. Schließlich wurde ein auf LURE basierender Halluzinationsrevisor anhand von RS-Bildmaterial “fine-tuned”, was zu einer geringfügigen Verringerung der Halluzinationsrate bei gleichbleibender Textqualität führte.

Table of Contents

List of Acronyms vii

List of Figures viii

List of Tables x

1. Introduction 1

2. Related Work 4
- 2.1. Image Captioning . . . 4
  - 2.1.1. Overview and State-of-the-Art . . . 4
  - 2.1.2. Vision-Language Models . . . 4
  - 2.1.3. Remote Sensing Image Captioning . . . 10
- 2.2. Hallucination . . . 12
  - 2.2.1. Hallucinations in Natural Language Generation . . . 12
  - 2.2.2. Hallucinations in Vision-Language applications . . . 13

3. Methods 25
- 3.1. Hallucination Detection . . . 25
  - 3.1.1. Object Hallucination Detection in Image Captions . . . 25
  - 3.1.2. Object probing questions . . . 27
  - 3.1.3. Vision-Language Model (VLM) Decoding Strategies for object detection . . . 28
  - 3.1.4. Visual Contrastive Decoding (VCD) Distortion Configurations . . . 31
  - 3.1.5. Semantic Entropy . . . 32
- 3.2. Hallucination Dataset Creation . . . 34
- 3.3. Hallucination reviser . . . 36
  - 3.3.1. Semantic CHAIR Evaluation . . . 37

4. Experimental Setup and Datasets 38
- 4.1. Datasets . . . 38
  - 4.1.1. Object-Probing Datasets . . . 38
  - 4.1.2. Remote Sensing Captioning Datasets . . . 39
- 4.2. Models . . . 40
  - 4.2.1. LLaVA 1.5 7B . . . 40

4.2.2. RS-LLaVA 7B 40
4.2.3. GeoChat 7B 40
4.2.4. Qwen3-VL 8B 40
4.2.5. Qwen3.5 9B 40
4.2.6. Caption Reviser 40
4.3. Tasks and Implementation Details 41
4.4. Object Probing Formats 41
4.4.1. Binary questions 41
4.4.2. Multiple Choice Questions 41
4.4.3. Decoding Strategies 42
4.4.4. Semantic Entropy Estimation 43

5. Results **45**
5.1. Baseline performance of hallucination detection using object probing questions 45
5.1.1. Binary questions vs multiple choice questions for hallucination detection 51
5.2. Visual contrastive decoding 54
5.2.1. Effect of distortion type and intensity on hallucination detection using visual contrastive decoding 55
5.2.2. Isolating the influence of contrastive decoding 59
5.2.3. Using uncertainty estimation to find hallucinations in VLM-generated captions in an existing Remote Sensing dataset 61
5.3. Semantic Entropy for uncertainty estimation in Remote Sensing (RS) 61
5.3.1. VQA 61
5.3.2. Captions 63
5.4. Using a reviser to remove hallucinations from existing Remote Sensing captions 65

6. Discussion **67**
6.1. Viability of VLMs as object-probing hallucination detectors 67
6.1.1. Performance effect of different object-probing formats 67
6.1.2. Effect of Visual Contrastive Decoding on hallucination detection 68
6.2. Viability of Semantic Entropy for Remote Sensing 69
6.3. Revising Remote Sensing image captions 69
6.4. Finding Hallucinations in Git-10M 70

7. Conclusion **71**
7.1. Limitations 71
7.2. Future Work 72

Bibliography **73**

Generative Tools Declaration 78

Appendix **79**

A.1. Multiple Choice Questions for hallucination detection: All Models 79

A.2. Isolating the influence of contrastive decoding: All Models 83

List of Acronyms

AP	Average Precision
APC	Adaptive Plausibility Constraint
AUC	Area Under the Curve
B2C	Box-to-Caption
CLIP	Contrastive Language-Image Pretraining
CNN	Convolutional Neural Network
GELU	Gaussian Error Linear Unit
LID	Labelled Illusion Dataset
LM	Language Model
LN	Layer Normalization
LoRA	Low-Rank Adaptation
M2B	Mask-to-Box
MAE	Masked Autoencoder
MCQ	Multiple-Choice Question
MIM	Masked Image Modeling
MLP	Multilayer Perceptron
MROPE	Multimodal Dimensional Positional Encoding
MSA	Multi-Head Self-Attention
MoE	Mixture-of-Experts
NLG	Natural Language Generation
NLP	Natural Language Processing
PCFG	Probabilistic Context-Free Grammar
POPE	Polling-based Object Probing Evaluation
Q-Former	Querying Transformer
ROC	Receiver Operating Characteristic
ROC-AUC	Area Under the ROC Curve
RS	Remote Sensing
RoPE	Rotational Position Encoding
TN	True Negative
TP	True Positive
UAV	Unmanned Aerial Vehicle
VCD	Visual Contrastive Decoding
VLM	Vision-Language Model
VQA	Visual Question Answering
ViT	Vision Transformer

List of Figures

2.1 Examples of different types of hallucinations and visual semantic errors. 14
2.2 An illustration of visual uncertainty amplifying language priors. 15
2.3 Comparison between hallucinatory and non-hallucinatory captions under different factors. 16
3.1 Visual Semantic Entropy pipeline. 33
3.2 RS-Image Caption Reviser Pipeline: Creating masked training captions. 35
3.3 Interface used to manually evaluate potentially hallucinated objects in Remote Sensing (RS) images. 35
3.4 RS-Image Caption Reviser Pipeline: Creating masked training captions. 36
3.5 RS-Image Caption Reviser Pipeline: Training the reviser. 37
3.6 RS-Image Caption Reviser Pipeline: Deployment. 37
5.1 Receiver Operating Characteristic (ROC) curves showing hallucination detection performance using binary questions on RS-POPE. 46
5.2 Confidence Distributions for Hallucination Detection Performance using Binary Questions on RS-POPE. 47
5.3 ROC curves showing hallucination detection performance using binary questions on Labelled Illusion Dataset (LID). 48
5.4 Confidence Distributions for Hallucination Detection Performance using Binary Questions on LID. 49
5.5 Hallucination detection performance using binary questions vs. multiple choice questions on RS-POPE with LLaVA 1.5. 52
5.6 Hallucination detection performance using binary questions vs. multiple choice questions on RS-POPE with Qwen3-VL. 53
5.7 Impact of distortion type and intensity on hallucination detection performance using Visual Contrastive Decoding (VCD) on binary questions on RS-POPE. 56
5.8 Impact of distortion type and intensity on hallucination detection performance using VCD on binary questions on LID. 57
5.9 Effect of distortion type and intensity on hallucination detection performance using multiple choice questions and VCD on RSPOPE. 59
5.10 Hallucination detection performance and Yes Ratios of different decoding strategies on Polling-based Object Probing Evaluation (POPE) and RS-POPE. 60
5.11 Confidence distributions for Hallucination Detection Performance using Semantic Entropy on VQA with the DDFAV dataset. 62

5.12 ROC-Curves for Hallucination Detection Performance using Semantic Entropy on VQA with the DDFAV dataset. 63
5.13 Confidence distributions for Hallucination Detection Performance using Semantic Entropy on image captions with the Git-10M dataset. 64
5.14 ROC-Curves for Hallucination Detection Performance using Semantic Entropy on images with the Git-10M dataset. 65

List of Tables

2.1 Side-by-side comparison of Git-10M, GAIA, and fMoW-mm datasets. 11

5.1 Baseline performance of hallucination detection using binary decoding with decision threshold fixed at 0.5 confidence score. 50

5.2 Performance of hallucination detection using multiple choice questions compared with binary questions baseline. 51

5.3 Performance of hallucination detection using Binary Decoding and Binary Visual Contrastive Decoding (VCD) across different confidence thresholds on RSPOPE and Labelled Illusion Dataset (LID) datasets. 55

5.4 VCD performance using a black image (masking ratio 1.0) and Gaussian noise (max intensity) as the distorted input, alongside the difference to the baseline targeting the hallucination class. The dataset used is RS-POPE. 58

5.5 Manual evaluation results for the lowest-confidence objects in Git-10M. 61

5.6 Effect of the reviser on Semantic-CHAIR scores on Remote Sensing (RS) captions. 66

5.7 Effect of the reviser on Remote Sensing captions on general caption quality metrics. 66

1. Introduction

The field of Remote Sensing (RS) faces a significant imbalance between the amount of available image data and the amount of labeled data [1]. In 2023, The European Space Agency's Copernicus satellite constellation generated 6.8 Petabytes of earth-observation data in just 10 months, more than in the 38 years before 2014 (6.8 PB vs 5 PB) [2]. While there is an abundance of image data from satellites and aerial platforms, the availability of high-quality captioned datasets is limited due to the time-consuming and costly nature of manual annotation. Image captions are important for data analysis, informed decision-making and efficient storage and retrieval of data [3]. Vision-Language Models (VLMs) are capable of generating rich textual descriptions of images, making them a promising solution for automated image captioning in RS [1]. Training on massive datasets gives these models remarkable versatility. In fact, domain-specific RS datasets like Git10M [4], GAIA [2], and RSTeller [5] used off-the-shelf, general-purpose VLMs to generate their captions.

Zavras et al. [2] state that VLMs have indeed been trained on RS images, but this is "incidental, unsystematic, and ultimately superficial". Additionally, VLMs struggle particularly with RS images due to complex backgrounds, imbalanced foreground and background pixel-ratios and different perspective from classical Computer Vision images [3, 6]. This exacerbates one of the main drawbacks of VLMs: Hallucinations.

Hallucinations are commonly defined as "content that is nonsensical or unfaithful to the provided source content" [7], in practice it is often used as a catch-all term for a wide variety of errors generated by language models. Hallucinatory captions are problematic as they can degrade model performance or directly mislead users who depend on accurate image interpretations [8]. This is especially problematic in RS where data problems can influence high-stakes applications, such as disaster relief and climate research.

Recent research has focused on object hallucinations in image captioning and Visual Question Answering (VQA). Object hallucinations happen when a model mentions objects that are actually absent in the input image. Researchers have explored different causes and types of object hallucinations, including co-occurrence, uncertainty, and multi-object hallucinations [8, 9]. Hallucination-specific metrics and detection methods have only recently been proposed, showing that the research field is still in its infancy. One of these metrics is the Polling-based Object Probing Evaluation (POPE) [10], which converts hallucination evaluation into a binary classification task. Different detection methods have also been proposed, including a hallucination reviser trained on artificially created hallucinations [8]. Training-free methods also exist, such as Semantic Entropy

[7], which estimates model uncertainty by measuring how often a model changes its answer for a single input. Another popular training-free method is Visual Contrastive Decoding (VCD), which purposefully induces hallucinations to extract internal model presumptions. While contrastive methods are popular, their efficacy and internal mechanism have recently been put into question [11]. These methods are fairly recent and have not been extensively tested in specific domains such as RS.

Most research has focused on general Computer Vision images, but some work on hallucinations in RS does exist. For example, the Labelled Illusion Dataset (LID) [3] is a possible benchmark for hallucination detection in drone imaging. Another example is RS-POPE [12], which applied the POPE framework to create an RS specific object hallucination benchmark. fMoW-mm [1] is a multimodal dataset that incorporates RS specific metadata to mitigate hallucinations. Still, significant work remains to explore how the nature of RS images, such as increased scale and differing perspective, affects hallucination production and what unique challenges are presented to existing hallucination-detection techniques. Exploring how existing methods need to be modified to embrace this field might shed light on hallucinatory behavior in RS and ultimately result in higher quality image captioning.

This thesis confronts two main issues in RS. The first problem is cost. Manually captioning the high volume of RS image data is prohibitively expensive [1]. The second problem is that the VLMs used for this automation produce captions containing hallucinations [1]. Currently, very little research exists on RS hallucination detection and revision techniques. Thus, the aim of this work is to explore how different hallucination detection and mitigation methods apply to RS.

Concretely, the first objective is to explore how to estimate model confidence. In other words: given an RS image and an object, how confident is a model that the object is actually present? A reliable confidence estimation method would allow for an analysis of how well different VLMs perform as object hallucination detectors. This work explores different object-probing strategies and whether they can be improved using visual contrastive decoding [13]. Additionally, this thesis tackles the challenges of adapting Semantic Entropy [7], a method developed in the context of unimodal language models, into the visual RS domain. The most effective detection strategy is subsequently applied to locate hallucinations within an existing, large-scale RS dataset.

The second objective is to train and deploy a hallucination reviser based on LURE [8]. This entails adapting the training pipeline introduced by Zhou et al. [8] for the RS domain and exploring if the trained reviser generalizes well enough across different RS datasets.

In summary, the goal of this thesis is to detect and revise object hallucinations in the RS domain and to explore if a selection of state-of-the-art methods is viable in RS.

The remainder of this thesis is structured as follows.

Chapter 2 reviews the foundational concepts and current state-of-the-art. It traces the evolution of VLMs and details the specific challenges of RS image captioning. It explores the mechanisms, causes, and evaluation metrics for hallucinations in text to image generation.

Chapter 3 details the methods used for hallucination detection and revision. It outlines object probing for confidence estimation, different decoding strategies, the adaptation of Semantic Entropy for the RS domain, and the pipeline for creating and training the hallucination reviser.

Chapter 4 presents the experimental setup. It describes the evaluation datasets, the specific vision-language models tested, and the decoding strategies and distortion configurations applied during testing.

Chapter 5 reports the experimental results. This includes the baseline performance of different detection strategies, the impact of VCD, Semantic Entropy, and the trained reviser performance.

Chapter 6 interprets these findings and discusses their broader implications.

The repository containing the entire codebase is available at https://git.tu-berlin.de/filipeborges/thesis-experiments.

2. Related Work

This chapter reviews the state-of-the-art research surrounding the most relevant topics for this thesis. It covers Image Captioning, Language Models (LMs) and Vision-Language Models (VLMs) leading to Remote Sensing (RS) Image Captioning and current topics on hallucination.

2.1. Image Captioning

This section reviews the mechanisms of modern image captioning. Starting with the Vision Transformer (ViT), it introduces the foundational concepts and early frameworks. It then follows the architectural shift in Vision-Language models from discriminative transformers to generative multimodal systems. Finally, it explores RS Image Captioning, highlighting the unique characteristics of geospatial data and the domain-specific methodologies required to adapt general-purpose models for Earth observation tasks.

2.1.1. Overview and State-of-the-Art

Image captioning is a task in Computer Vision and Natural Language Processing (NLP) that involves generating descriptive textual captions for images. The task is relevant across many distinct applications, such as aid for visually impaired people, robotics, medical imaging, search engine optimization and more recently for multimodal chatbot systems. In RS, image captions are important for data analysis, informed decision-making and efficient storage and retrieval of data [3]. Yet, automating this process remains a challenging problem. It requires analyzing visual content, applying field-specific knowledge, and translating that information into fluent language.

The encoder-decoder framework with attention established the foundation for automatic image captioning [14, 15]. Today, transformers dominate the field [15]. These models use self-attention to process sequential data and capture long-range context. This architecture also enabled VLMs like LLaVA [16]. By integrating visual data into LMs, VLMs leverage existing reasoning capabilities to generate coherent captions. However, VLMs still struggle with RS images due to complex backgrounds, imbalanced foreground and background pixel-ratios and different perspective from classical Computer Vision images [3, 6].

2.1.2. Vision-Language Models

Until recently, Convolutional Neural Networks (CNNs) represented the state-of-the-art in Computer Vision [17]. CNNs are designed to process grid-like data, for example images

in the form of a 2D-grid of pixel values. Four key concepts explain their success: local connections, shared weights, pooling, and deep architecture. They achieve this through convolutional layers and pooling layers. Each unit in a convolutional layer corresponds to a different feature map. Each unit connects to local patches in the feature maps of the previous layer through a set of weights called a filter bank. A single filter in a filter bank is a small matrix of weights and, as the network trains, these weights adjust to recognize specific visual patterns in the form of feature maps. This architecture leverages two natural properties of images. First, local groups of pixel values are highly correlated, forming distinct, easily detectable motifs. Second, these local image statistics are location-invariant, meaning a pattern appearing in one area could logically appear anywhere. Subsequently, pooling layers coarse-grain the position of these features, merging semantically similar motifs to create invariance to small shifts. Ultimately, these networks assemble compositional hierarchies. Lower-level features act as oriented edge detectors, which subsequent layers assemble into parts, and eventually into whole objects [17].

As the Computer Vision community pushed for deeper networks to learn more abstract representations, they encountered a critical bottleneck known as the degradation problem. Unexpectedly, as networks grew deeper, their training accuracy saturated and then rapidly degraded. This was not caused by overfitting, as the deeper "plain" models yielded higher training error than their shallower counterparts. To bypass this optimization barrier, He et al. [18] introduced Deep Residual Learning (ResNets). Instead of expecting a stack of nonlinear layers to directly approximate an unreferenced underlying mapping $H(x)$, the layers are explicitly reformulated to fit a residual mapping $F(x) := H(x) - x$. Intuitively, it is easier to optimize the required adjustments or perturbations to the original input than it is to learn a completely new function from scratch. If the optimal outcome is to leave the data unchanged, the network can simply push these corrections to zero, rather than struggling to exactly reconstruct the original input through a complex stack of non-linear layers. This is structurally realized using parameter-free identity shortcut connections that skip one or more layers. The residual building block with a projection shortcut is formulated as

$$\boldsymbol{y} = F(\boldsymbol{x}, \{W_i\}) + W_s\boldsymbol{x} \tag{2.1}$$

by He et al. [18]. CNNs and ResNets achieve good performance by building upon built-in inductive biases and, layer upon layer, learn to recognize abstract concepts. This paradigm has been challenged by the emergence of ViTs [19], which have become a core part of the current Computer Vision research landscape, especially regarding image-to-text models. ViTs stem directly from the transformer architecture that has been dominant in the field of NLP [14]. The standard Transformer avoids recurrence and convolutions, processing entire text sequences globally through attention mechanisms. ViT replicates this by splitting a 2D image into a grid of fixed-size patches (e.g., 16×16 pixels) and

treating each patch as a single token in a sequence. The ViT architecture strictly follows the original NLP Transformer model, intentionally avoiding image-specific structural biases. The first step is patch embedding, this step is the only one that was changed in order to accommodate visual input. The original image is flattened into a sequence of N patches, projected into a latent dimension D via a linear matrix $\boldsymbol{E}$, and appended with a learnable classification token $\boldsymbol{x}_{\text{class}}$ alongside 1D positional embeddings $\boldsymbol{E}_{\text{pos}}$:

$$\boldsymbol{z}_0 = \left[\boldsymbol{x}_{\text{class}}; \boldsymbol{x}_p^1\boldsymbol{E}; \boldsymbol{x}_p^2\boldsymbol{E}; ...; \boldsymbol{x}_p^N\boldsymbol{E}\right] + \boldsymbol{E}_{\text{pos}} \tag{2.2}$$

with $\boldsymbol{z}_0$ representing the initial sequence of latent representation vectors (embedded tokens) that serves as the input to the Transformer encoder. As in the standard transformer, the sequence is passed through L alternating layers of Multi-Head Self-Attention (MSA) and a Multilayer Perceptron (MLP). Layer Normalization (LN) is applied before every block, and residual connections are applied after. The MSA step can be formalized as:

$$\boldsymbol{z}'_l = \text{MSA}(\text{LN}(\boldsymbol{z}_{l-1})) + \boldsymbol{z}_{l-1} \tag{2.3}$$

Following the MSA block, the sequence is passed through the MLP block, which mirrors the same residual and normalization structure:

$$\boldsymbol{z}_l = \text{MLP}(\text{LN}(\boldsymbol{z}'_l)) + \boldsymbol{z}'_l \tag{2.4}$$

Internally, this MLP contains exactly two layers paired with a Gaussian Error Linear Unit (GELU) non-linearity. Given an input vector $\boldsymbol{x}$ (which corresponds to $\text{LN}(\boldsymbol{z}'_l)$ in the architecture), the transformation is defined as:

$$\text{MLP}(\boldsymbol{x}) = \boldsymbol{W}_2 \ \text{GELU}(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 \tag{2.5}$$

where $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ represent the learned wight matrices for the first and second linear layers, respectively, and $\boldsymbol{b}_1$ and $\boldsymbol{b}_2$ are their corresponding bias vectors.

The final representation used for image classification is derived exclusively from the state of the classification token at the final layer L.

$$\boldsymbol{y} = \text{LN}\big(\boldsymbol{z}_L^0\big) \tag{2.6}$$

ViT presents a simple architecture that, unlike CNNs, does not introduce any structural biases such as locality. A ViT must therefore learn all spatial relations directly from the

data from scratch. In practice, this means that ViTs are data-hungry and need large-scale training to overtake the performance of standard CNNs.

2.1.2.1. From vision to language-vision

In essence, standard vision transformers compress an image into a latent dimension to classify it into fixed categories. Contrastive Language-Image Pretraining (CLIP) [20] takes this one step forward and creates a shared latent dimension where both images and text can coexist together. Instead of just processing an image, CLIP uses two separate neural networks simultaneously: an image encoder f_θ (e.g. a ViT) and a text encoder g_φ (e.g. a transformer). The two encoders generate intermediate feature representations, given image input I and text input T:

$$\boldsymbol{h}_I = f_\theta(I), \boldsymbol{h}_T = g_\varphi(T) \tag{2.7}$$

Crucially, both encoders are forced to project their final inputs into the same shared latent dimension. Because $\boldsymbol{h}_I$ and $\boldsymbol{h}_T$ may have different dimensions, they are multiplied by learned linear projection matrices ($\boldsymbol{W}_I$ and $\boldsymbol{W}_T$) to force them into the same shape. They are then L2-normalized so that only their direction matters.

$$\boldsymbol{z}_I = \frac{\boldsymbol{h}_I \boldsymbol{W}_I}{\|\boldsymbol{h}_I W_I\|_2}, \boldsymbol{z}_T = \frac{\boldsymbol{h}_T \boldsymbol{W}_T}{\|\boldsymbol{h}_T W_T\|_2} \tag{2.8}$$

Once both modalities share the same space $\mathbb{R}^d$, the model can directly compare how closely the image matches the text by computing their cosine similarity. This simplifies to a dot product because the vectors are normalized:

$$\text{Similarity} = \boldsymbol{z}_I \cdot \boldsymbol{z}_T \tag{2.9}$$

Ultimately, CLIP was trained on generated image-text pairs to optimize a bi-directional InfoNCE loss:

$$\mathcal{L}_q = -\log \frac{\exp(\boldsymbol{q} \cdot \boldsymbol{k}_+ / \tau)}{\sum_{i=0}^{K} \exp(\boldsymbol{q} \cdot \boldsymbol{k}_i / \tau)} \tag{2.10}$$

where $\boldsymbol{q}$ is the representation vector of the query (e.g., an image embedding), $\boldsymbol{k}_+$ is the representation vector of the positive sample (e.g., the correct text embedding), $\boldsymbol{k}_i$ is the representation vectors of all $K+1$ samples in the batch (one positive key and K negative keys), and τ is the temperature. The InfoNCE loss effectively aligns the visual features with their corresponding text embeddings. In other words, in CLIP's shared latent

dimension, the visual concept of some object and the written word for the same object become mathematically identical.

One issue with CLIP is that it faces a bottleneck due to it using softmax contrastive loss [21]. Therefore, CLIP needs a global view of the pairwise similarities for normalization. In practice, this process takes up a lot of memory as all text vectors need to be held simultaneously in memory. Instead, SigLIP [21] processes every image-text pairing independently by treating the goal as a binary classification task using sigmoid loss with positive labels for the matching pairs (I_i ,T_i) and negative labels for all other pairs (I_i,$T_{j\neq i}$). While CLIP and SigLIP perform similarly well with larger batch sizes, SigLIP requires less memory and is computationally cheaper.

SigLIP 2 [22] builds upon the original by adding multiple independently developed techniques and achieves better performance than its predecessor across the board. SigLIP 2 incorporates supervised and decoder-based losses for better dense features (self-distillation and masking) and captioning-based pretraining.

While CLIP and SigLIP excel at embedding images and text into a shared latent representation space suitable for retrieval and classification, they are strictly discriminative models and thus not inherently generative. One influential step in the field of image captioning was the development of multimodal models that can perform open-ended, autoregressive text generation conditioned directly on visual inputs. This transition requires novel connection mechanisms to fuse frozen or trainable vision encoders with highly capable language models.

One early example of this is Flamingo [23], which introduces key architectural advancements that allow for training using interleaved data. While previous Vision-Language models were trained on simple image-text pairs, Flamingo is also trained on a vast dataset of web pages where images and text are naturally interwoven. Flamingo builds on a frozen language model and vision encoder and connects these with novel trainable layers: the perceiver resampler and gated cross-attention. High-resolution images or videos produce a massive and variable number of features, making it computationally prohibitive to feed them directly to an LM. The perceiver resampler ingests these variable features and distills them into a fixed number of visual tokens. Additionally, Flamingo inserts trainable cross-attention layers between the layers of the frozen LM. This allows the text-generation process to conditionally access the visual tokens produced by the resampler.

While Flamingo saves a lot of compute by utilizing a frozen vision encoder and LM, it still modified the language models architecture by inserting new layers that need training. BLIP-2 [24] goes one step further and does not change the language model at all. Only one new component, the Querying Transformer (Q-Former), is trained and used to connect the frozen vision encoder and language model. The Q-Former extracts visual features and outputs query embeddings that are linearly projected to match the dimension of the LM's text embeddings. Effectively, the Q-Former reduces the visual information and only feeds

the most important visual features into the language model. BLIP-2 achieves superior performance compared to Flamingo while using 54 times fewer trainable parameters.

In contrast with the previous two approaches, LLaVA [16] only freezes the vision encoder (CLIP) and connects it to the language model using a simple linear projection layer (W). It takes raw visual features (Z_v) from the vision encoder and multiplies them by a single trainable matrix to map them directly into the word embedding space (H_v) of the language model:

$$H_v = W \cdot Z_v \tag{2.11}$$

LLaVA essentially treats images as if they were text tokens and alongside human instructions. Crucially, LLaVA was trained on a new visual instruction based dataset. The authors fed rich image metadata to GPT-4 to create high quality multi-turn instruction-following samples including conversations, detailed captions and complex reasoning. Thanks to this dataset, the model behaves more like a chat-bot assistant instead of a pure caption generator. One issue that LLaVA has is that it rarely gives short few word answers which has a negative effect on benchmarks. LLaVA 1.5 improves upon this by fine-tuning using explicit rules that were appended to prompts such as "Answer the question using a single word or phrase". Additionally, LLaVA 1.5 [25] swaps the linear projection layer for a 2 layer Multi-Layer Perceptron and replaces the CLIP vision encoder for a higher resolution version.

Qwen3-VL [26] builds upon the basic architecture of LLaVA with few key architectural differences. While industry leading, production ready models such as the Qwen family of models incorporate many engineering optimizations and proprietary techniques, Qwen3-VL has two features that stand out for image captioning. First, Qwen3-VL utilizes Multimodal Dimensional Positional Encoding (MROPE). LLaVA maps image patches through a bottleneck projection layer and routes them into the LM as a flat string of tokens, relying on the LM's standard 1D positional embeddings. Qwen replaces 1D tracking with Interleaved MROPE, a 3D-coordinate system using width, height, and time. For example, the positional embedding for an image patch freezes the temporal axis:

$$\text{Patch Position} = (t_{\text{frozen}}, h_{\text{row}}, w_{\text{column}}) \tag{2.12}$$

By using MROPE the model does not need to learn how to reinterpret the 1D positional embedding for 2D-images (and video). Compared to LLaVA, Qwen3-VL also changes how the outputs of the Vision Transformer are fed into the language model. Instead of feeding the output of the Vision Transformer directly into language model, it extracts visual tokens from three distinct intermediate layers of the Vision Transformer. These multi-level features are injected directly into the corresponding first three layers of the language model via dedicated mergers. This enables the text decoder to access raw, low-level visual

semantics important for dense image captioning without increasing the overall context token length.

2.1.3. Remote Sensing Image Captioning

RS images differ fundamentally from general Computer Vision datasets. They feature vast scales, strict top-down perspectives, and imbalanced pixel ratios [3, 6]. Yet, Vision-Language models can already generate useful captions for them. For instance, the Git-10M [4] dataset authors used the GPT-4o API to caption 10 million RS images, mostly from Google Earth. To caption the Google Earth images (comprising 70% of the dataset) the model used only the image, visual resolution, and geographic coordinates as can be seen in Table 2.1. Off-the-shelf VLMs, without any fine-tuning, engineered prompting strategies or extensive metadata are already useful for automatic RS image captioning.

Even so, different approaches have emerged to tackle specific RS challenges. For example, RS VLM-generated captions are often generic and contain errors [1]. One approach is to incorporate rich metadata common in RS. Like Git-10M, both the fmow-mm [1] and GAIA [2] datasets use the GPT-4o API, but incorporate extensive metadata. The fmow-mm dataset integrates location, time and cloud-cover metadata as well as maps into the captioning prompt. The maps provide labels and segmentation information. The authors of GAIA used GPT-4o to clean, summarize and extract metadata from RS web-scraped articles. The resulting metadata contains information such as location, time, scene classification labels, satellite, sensor, and modality. In both cases, the resulting captions contain a significant proportion of text that stems directly from the metadata. Table 2.1 shows examples of all 3 GPT-4o based datasets.

Table 2.1: Side-by-side comparison of Git-10M, GAIA, and fMoW-mm datasets.

Dataset	Image	Metadata	Caption
Git-10M		**img_name**: Google-10-10_1005_271.jpg **img_quality_score**: 4.352512 **caption_length**: 56 **resolution**: 128m/pix **Google_location**: 10_1005_271	The satellite image predominantly shows a landscape with varying shades of green indicating vegetated areas. There are discernible brown patches likely representing areas with less vegetation or different land cover. A river with a meandering path is visible in the top left, and scattered cloud cover can be observed in the lower portion of the image.
GAIA		**id**: record-breaking-number-bird-nesting-pairs-eastern-spain **image_src**: Albufera.jpg **image_alt**: Record-breaking number of bird nesting pairs in Eastern Spain **location**: Albufera Natural Park, Valencia, Spain **lat**: 39.3488971 **lon**: −0.324044699999999 **credits**: European Union, Copernicus Sentinel-2 imagery **tag**: ['Biodiversity', 'Natural Events', 'Water Resources'] **modalities**: ['Optical'] **satellite**: ['Sentinel-2'] **sensor**: ['MultiSpectral Instrument (MSI)'] **resolution**: High	Albufera Natural Park in Eastern Spain, captured using optical RS by Sentinel-2. The high spatial resolution allows for detailed observation of the water bodies and surrounding landscapes, significant for monitoring the record number of bird nesting pairs in the region.
fmow-mm		**img_id**: railway_bridge_369_3 **label**: railway bridge **lat**: 51.647124 **lon**: −0.387169 **city**: Watford **state**: England **country**: United Kingdom **country_code**: GBR **country_from_code**: United Kingdom **gsd**: 0.534373 **cloud_cover**: 11 **year**: 2016 **month**: February **day**: 18 **sw_lon**: −0.389415 **sw_lat**: 51.645727 **ne_lon**: −0.384924 **ne_lat**: 51.648521 	The image depicts an area in Watford, England, near the coordinates 51.65 latitude and −0.39 longitude. The scene shows a railway bridge crossing over a river, with a nearby retail area, Arches Retail Park, to the north. The bridge is surrounded by dense greenery, likely part of Oxhey Dell to the south. Several large buildings are visible, including a B&Q and The Range to the north, and a Mercedes Benz dealership to the east. A parking area is evident, filled with numerous vehicles. The image was captured on February 18, 2016, with a spatial resolution of 0.53 meters.

A different strategy is to design and train a VLM to solve RS specific problems. For example, the authors of RemoteCLIP [27] state that Masked Image Modeling (MIM) pretraining, a succesful approach for VLMs, is not optimal for high-level RS semantic recognition tasks. MIM involves hiding portions of an image (patches or pixels) and forcing a model to learn by predicting the missing content. While MIM methods learn occlusion invariant features, occlusion invariance is less important for RS images. Instead of MIM, the authors trained a model based on CLIP. Another problem is the scarcity of RS pretraining data. To overcome this, the authors employed a data scaling technique via “annotation unification”. They converted heterogeneous annotations from ten object detection datasets and four semantic segmentation datasets into a unified image-caption format. This was achieved through novel Box-to-Caption (B2C) and Mask-to-Box (M2B) generation strategies. By combining these unified datasets with existing retrieval datasets

and incorporating Unmanned Aerial Vehicle (UAV) imagery, they generated a pretraining dataset 12 times larger than the combination of all previously available open datasets. Like CLIP, RemoteCLIP was trained generated image-text pairs to optimize a bi-directional InfoNCE loss.

LLaVA, a popular generalist VLM architecture, was also adapted for RS. RS-LLaVA [28] was fine-tuned on the RS-Instructions dataset, consisitng of 4 existing Visual Question Answering (VQA) and captioning RS datasets that were reformatted as conversations. The RS-LLaVA training process consists of 2 steps. The first step is pre-training the projection layer with a general image-text dataset akin to LLaVA. The second step freezes the projection layer and the image encoder to fine-tune the language model using the Low-Rank Adaptation (LoRA) method.

Instead of simply fine-tuning an existing VLM, RS-MoE [29] introduces a novel architecture designed for RS based on the Mixture-of-Experts (MoE) paradigm. Unlike traditional MoE models that rely on simple feed-forward networks, the RS-MoE block utilizes multiple lightweight LMs as its specialized expert models. Central to this design is an instruction router that generates tailored prompts based on input instructions and visual features. This router delegates the complex task of image captioning into three distinct subtasks: thematic interpretation, object identification, and spatial relationship analysis. It then assigns each prompt to a specific expert LM. The outputs of these experts are then aggregated into a final caption. To effectively adapt this architecture to the geospatial domain and prevent performance degradation caused by data sparsity, the authors employ a two-stage training strategy. The first stage initializes the Vision-Language encoder and base LM, while the second stage fine-tunes the MoE block, utilizing LoRA throughout both stages to maximize computational efficiency.

2.2. Hallucination

Generative foundation models must be reliable for them to become integrated into real-world applications. Their safe deployment stalls when they hallucinate and produce plausible but false outputs. This section explores the concept of hallucination across both text and image-text domains, examining its definitions, underlying mechanisms, and evaluation.

2.2.1. Hallucinations in Natural Language Generation

The term hallucination in the context of machine learning is commonly defined as "content that is nonsensical or unfaithful to the provided source content" [30]. In practice, "hallucination" is used to describe a vast array of different errors with different underlying mechanisms that result in similar symptoms. A few examples of errors commonly called hallucinations include: when LMs are consistently wrong because of erroneous training data; when LMs "lie" in pursuit of a reward or when an LM's answer is sensitive to irrelevant details such as a random seed (defined as "confabulation") [7]. Many more types of domain-specific hallucinations exist, such as object hallucinations and co-occurrence

hallucinations common in Vision-Language image captioning [8]. The term has expanded to include anything generated by a foundation model that is not based in "fact" or "faithfulness". Generated content is "faithful" if based in source knowledge and "factual" if based in world knowledge. However, many applications treat source input as "fact" making "faithfulness" and "factuality" interchangeable [31]. Thus, hallucinations can be categorized into "intrinsic hallucinations", unfaithful content that contradicts source knowledge, and "extrinsic hallucinations", unfaithful content that cannot be verified by source knowledge but might still be factual [31].

One of the main causes of hallucinations are data-related problems, such as when target data contains additional information not supported by the source information [31]. This source-reference divergence may be caused by heuristic data collection used to build large datasets. For instance, the authors of WIKIBIO [32] collected and utilized infoboxes from Wikipedia pages as source, and the first sentence of the same Wikipedia page as target. This resulted in source-reference divergence in 62% of cases, as most WIKIBIO first sentences contain additional information not supported by the corresponding infobox. Source-reference divergence may also be innate in some Natural Language Generation (NLG) tasks. For example, certain chat systems enhance their "engagingness" with relevant facts that are not present in the user input, history or provided knowledge source, leading to inevitable extrinsic hallucinations [31]. Another possible hallucination cause is the presence of duplicates in datasets, which may bias the model into repeating memorized sentences present in duplicated examples. Hallucinations may also be caused by limited exposure to niche domain-specific data or a general lack of source-target pairs, both problems are prevalent in RS image-captioning [1].

The second main cause for hallucinations are training-related issues that affect different components of neural models. Encoders, responsible for compressing input data into meaningful representations, can have defective comprehension abilities or learn wrong correlations in data, increasing the rate of hallucination. Decoders, responsible for transforming encoded representations into output text, can attend to the wrong part of the encoded input or introduce randomness through sampling strategies and increase hallucination probability. Large pre-trained models can also suffer from parametric knowledge bias, this can lead to the model prioritizing parametric knowledge over provided input [31].

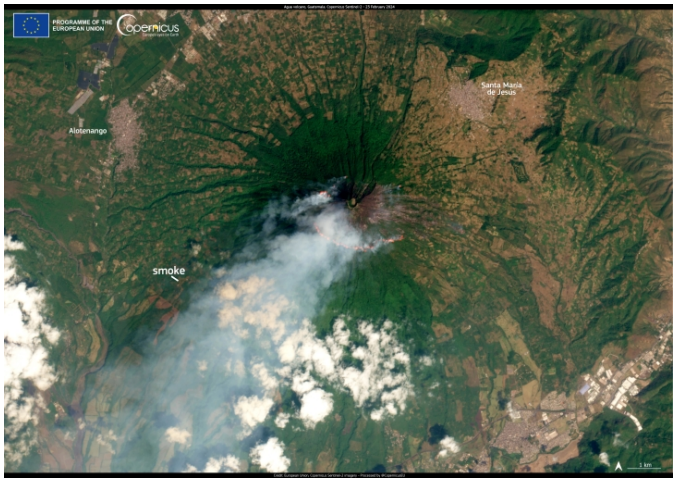
2.2.2. Hallucinations in Vision-Language applications

VLMs inherit many of the same hallucination causes as NLG models, but also have additional problems that are unique to the Vision-Language domain. This section explores the different types of hallucinations that occur in Vision-Language applications, their causes, evaluation metrics. It also explores current research on detecting, mitigating and revising hallucinations.

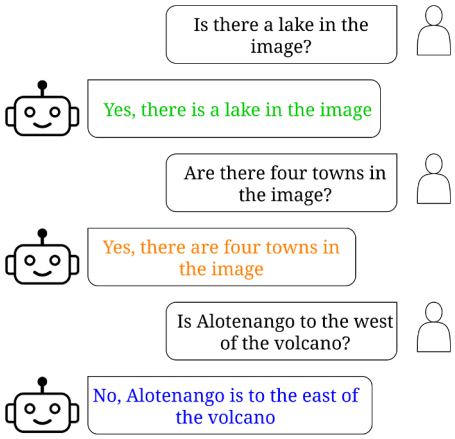
2.2.2.1. Hallucination Types

In Vision-Language, text is generated based on visual input. Images are a fundamentally different medium to text, and as such, the hallucinations that occur in image captioning are different from those that occur in other NLG tasks. VLM-Hallucinations primarily occur in two different settings: "judgment hallucinations", which occur when the response to a user's question contradicts the actual image, and "description hallucinations", which are failures to faithfully depict the contents of an image [19]. Both types of hallucination can be seen in Figure 2.1. Errors can be in different visual semantics, such as objects, attributes, relations, position and count. Examples of these errors are also depicted in Figure 2.1.

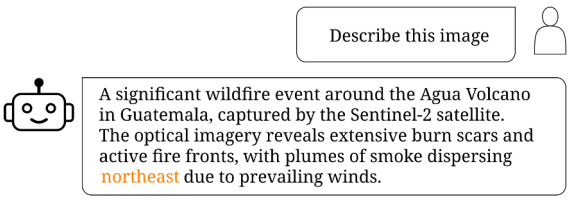
Object hallucinations happen when generated text includes objects that do not actually exist in the input image. Attribute hallucinations occur when generated text claims incorrectly assigns a quality, such as color, to an existing object in an image. A relation hallucination is when the relationship between two objects is incorrectly described, for example "the car is to the left of the bicycle" or "the cloud is casting a shadow on the beach". Hallucinations due to count error are when the count of a specific, existing object



(a) Image from GAIA



(b) Judgment Hallucinations



(c) Description Hallucination

Figure 2.1: Examples of different types of hallucinations and visual semantic errors. In green: object hallucination; in orange: attribute hallucination; in blue: relation hallucination. Adapted from Liu et al. [19]. Image taken from the GAIA dataset [2].

in an image is wrong, for example, if a generated description states that there are 4 cars in an image when in reality there are only 3 [3].

2.2.2.2. Hallucination Causes

Most of the literature on VLM-Hallucinations has focused on object hallucinations. The two primary causes of object hallucinations in VLMs are statistical bias and language priors [13].

Statistical bias refers to a model's tendency to rely on superficial object correlations and imbalanced distributions found in its Vision-Language training data. Due to statistical bias, models are more likely to hallucinate objects that are seen often during training. This also leads to co-occurrence hallucinations, models are more likely to hallucinate objects that frequently appear in the same images as other objects. For example, a model might incorrectly hallucinate a "fork" simply because it correctly identified a "dining table" in the image, or hallucinate a "parking lot" because it correctly identified a "shopping mall".

Language priors are inherited by VLMs from their language model component. Since LMs are pre-trained on vast amounts of text-only data, they are designed to predict the next word based on linguistic probability rather than visual evidence. For example, Alayrac et al. [23] show that Large VLMs, when prompted with a question and image pair, hallucinate by producing answers that seem plausible given the text only, but contradict the contents of the given image. Additionally, visual uncertainty, such as unclear or distorted images, can exacerbate the reliance on language priors. Leng et al. [13] illustrate this with an image of a "black banana", as seen in Figure Figure 2.2. While the VLM correctly describes the banana as black, it fails to do so when the image is distorted. As the level of visual uncertainty increases, the model increasingly relies on its language priors and favors more conventional banana colors such as "yellow" or "green".

A recent study performed a statistical analysis of key factors underlying object hallucinations: co-occurrence, uncertainty and text position. Uncertainty is the score used to measure a VLM's confidence in generating the next token during decoding. Text position

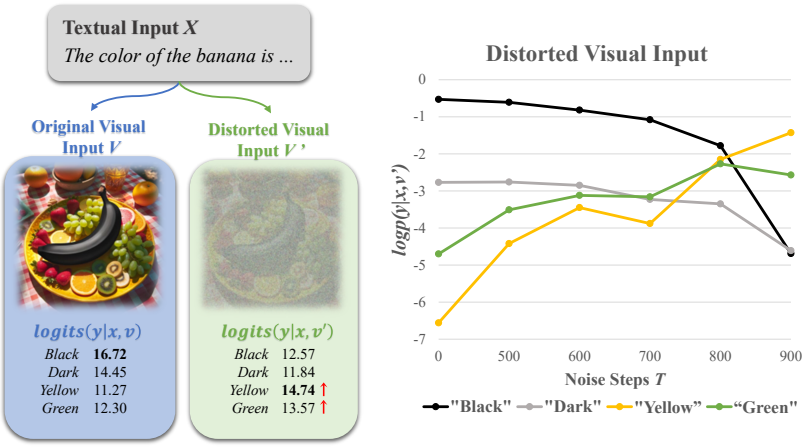
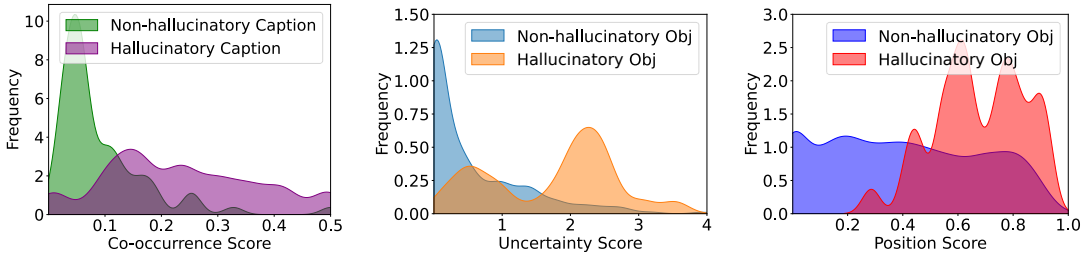


Figure 2.2: An illustration of visual uncertainty amplifying language priors [13].

measures how early or late an object appears in generated text. As seen in Figure 2.3, object hallucinations tend to occur among objects that frequently co-occur, objects with a high uncertainty score during decoding and among objects that are generated in the later part of generated text [8]. Indeed, high co-occurrence and high uncertainty lead VLMs to hallucination due to over-reliance on statistical bias and language priors. Additionally, models might hallucinate more at the later parts of generated text due to the autoregressive text generation process. While the model starts off by relying on the semantic information present in the image, the information of past hallucinations and increasing uncertainty cause the model to "snowball". Unlike initial halucinations, "snowball hallucinations" occur when the model maintains consistency with past mistakes [33].



(a) (b) (c)

Figure 2.3: Comparison between hallucinatory and non-hallucinatory captions under different factors [8].

2.2.2.3. Hallucination Metrics

General caption quality metrics do not measure hallucinations directly, but they are still an important measure alongside modern hallucination metrics. The standard way to measure the captioning performance of language models has been to measure the similarity of generated text to the corresponding reference caption. In practice, similarity based metrics are used to measure overall text quality, such as semantics and fluency. While these metrics do not directly probe hallucinations, an issue addressed by more recent hallucination-specific metrics, they still form the basis for evaluating text generation.

Stemming from the field of machine translation, BLEU [34] is an early attempt at automated text evaluation and still remains relevant today. BLEU is based on modified n-gram precision, which counts how many n-grams (phrases of 1 to 4 words) match a reference text. Matching single words measures the adequacy of generated text, while matching longer phrases measures fluency. BLEU combines these scores by taking the geometric mean of the modified n-gram precisions. As translations of the same text are expected to be similar in length, BLEU multiplies the n-gram precisions by a brevity penalty.

METEOR [35] was developed to address two issues in BLEU. First, BLEU's n-gram matching is very rigid and only counts exact matches. In contrast, METEOR is more flexible and also matches synonyms and words sharing the same stem (e.g., "speaking" and

"spoke"). Second, METEOR places more weight on unigram recall, the ratio of matched words to the total words in the reference text.

Adjacent to n-gram overlap metrics is ROUGE [36], a family of metrics focusing on different variations of n-gram recall. The recall based metric was developed for evaluating text summarization, consequentially it focuses on how much meaning is captured in comparison to reference summaries. ROUGE-N is the standard n-gram recall between a candidate text and a set of reference texts. Relying solely on n-grams can be rigid, one variant of ROUGE that addresses this issue is ROUGE-L, which measures the longest common subsequence between texts. For example the sequences "A B C D" and "A X B X C" do not share "A B C" as a strict 3-gram, but they do share "A B C" as their longest common subsequence. This has the advantage of capturing sentence structure and word order without needing a predefined n-gram length. ROUGE-L still has a blind spot, it weighs all subsequences equally, regardless of how spread out they are. For example, if the reference sequence is "A B C D E F", ROUGE-L would give the exact same score to the sequences "A B C X X X" and "A X X B X C". ROUGE-W addresses this by weighing common subsequences by how spread out they are. ROUGE-S evaluates texts based on skip-bigrams, that is any word pairs in their sentence order allowing for arbitrary gaps. ROUGE-SU is a slightly less rigid version that also takes unigrams into account.

Generally, the goal in machine translation is accuracy and this is reflected in both BLEU and METEOR. Conversely, the main goal in image captioning is to generate human-like descriptions. BLEU has been shown to correlate weakly with human judgment, while METEOR showed a stronger correlation. CIDER [37] addresses this issue directly by evaluating generated text on a consensus based on multiple human written captions. Each n-gram is weighted according to the frequency that it appeared on different human written descriptions (Term Frequency), in other words, it places higher weight on words that multiple humans agreed were important enough to mention. Additionally, CIDER assumes that sequences that appear on every description in the dataset are less meaningful (e.g., "The image depicts") and discounts these accordingly (Inverse Document Frequency). Based on these scores, CIDER finally compares the cosine similarity between the candidate description and the set of human reference sentences to calculate the final score. CIDER-D is a variation of CIDER designed to be more robust against the "gamification" of the metric. High CIDER scores can be achieved by repeating high confidence words over long sentences, CIDER-D remedies this by introducing a Gaussian penalty based on sentence length differences. Finally, CIDER-D also removes stemming to ensure the correct forms of words.

While the above metrics are based on n-gram overlap and recall, it's also important to remember that n-gram overlap is neither necessary nor sufficient to compare the meaning of two texts. Two texts may use the same words but have different meanings or use completely different words and be semantically equivalent. SPICE [38] is a metric designed for image captioning that directly analyzes the semantic content of a given text.

This is done by feeding the caption through a Probabilistic Context-Free Grammar (PCFG) dependency parser. The parser links words together based on their syntactic role in the sentence (e.g., Identifying "girl" as a central noun and drawing an adjectival modifier arrow from "young" to "girl"). This is used to build a semantic graph of the text. From this, tuples of word pairs (e.g., "(girl, young)") are formed. The presence (or absence) of these tuples is compared to a reference text to calculate the final score. SPICE correlates better with human judgment than all the discussed n-gram overlap based metrics.

Modern image captioning models are able to write fluent and coherent image descriptions that might still contain objects that are not present in the scene. A generated caption that is rich and fluent but contains a singular hallucinated object might still score highly with text similarity based metrics, which is an obvious issue for sensitive applications. The CHAIR [39] metric tackles this issue directly by measuring the ratio of hallucinated to real objects. The two variants of CHAIR are CHAIR_i which measures the ratio of hallucinated objects in a caption and CHAIR_s which measures the ratio of sentences containing hallucinations. One advantage of CHAIR for image captioning is that it directly measures the rate of hallucinations within captions. On the flip-side, CHAIR inherits the sensitivity of LMs to slight prompt variations. This means that 2 equivalent caption generation prompts might result in significantly different CHAIR scores (e.g., "Generate a short caption of the image." vs. "Provide a brief description of the image.").

The Polling-based Object Probing Evaluation (POPE) [10] addresses this stability issue by reframing the evaluation of object hallucinations as a binary classification task. The POPE framework starts by generating two lists of objects, a list of ground-truth objects extracted from a given image and a list of objects not present in the image. Both lists are then used to generate binary probing questions (e.g., "Is there a bus in the image?"). One further advantage of POPE is the ability to create multiple benchmarks of varying difficulty by adjusting the list of nonexistent objects. For example, by selecting absent objects that often occur in the dataset or by selecting objects that co-occur often with the objects present in the image.

As the problem of hallucination evaluation is turned into a binary detection task, the standard metrics of signal detection theory become relevant. The main metrics used in binary hallucination evaluation are precision, recall, F-score and Yes Ratio. Assuming the case where the positive class is when an object is visible in the image, precision is the ratio of correctly predicted visible objects to the total predicted visible objects. Given that the number of correctly predicted visible objects is the number of true positives TP, and the total predicted visible objects is the sum of true positives TP and false positives FP, then:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2.13}$$

This score is penalized when a model misses visible objects and incorrectly classifies them

as not visible. Recall would then be the ratio of correctly predicted visible objects to the total actual visible objects. Given that the number of correctly predicted visible objects is the number of true positives TP, and the total amount of objects is the sum of true positives TP and false negatives FN, then:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2.14}$$

This score is penalized when a model is overly confident and classifies not visible objects as visible. The F_1-score is the harmonic mean of precision and recall, this gives a good metric for model performance especially when the positive class is rare (if hallucinations are relatively rare, it might be sensible to use hallucination as the positive class). The F_1-score is defined as:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.15}$$

The F_{beta}-score is the more general formulation allowing for different trade-offs between precision and recall (e.g., $\beta = 2$ gives more weight to recall over precision):

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \tag{2.16}$$

One problem with the F_1-score is that it does not take into account the number of true negatives. Accuracy is the ratio of all correct predictions to the total number of predictions, measuring the overall model performance. Given that all correct prediction is the sum of the True Positives (TPs) and True Negatives (TNs), and that the total number of predictions is the sum of all postive predictions P and negative prediction N, then:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}} \tag{2.17}$$

Finally, the Yes Ratio is the ratio of all positive predictions P to the total number of predictions, measuring the model's overall confidence and possible bias:

$$\text{Yes Ratio} = \frac{\text{P}}{\text{P} + \text{N}} \tag{2.18}$$

2.2.2.4. Hallucination Mitigation

Current research into hallucination mitigation spans across the entire lifecycle of VLMs. Modern approaches include explicit visual grounding and contextual enrichment of data, inference and decoding strategies that leverage the current understanding of hallucinations, and training models specifically for hallucination detection and revision.

One of the main sources of object hallucinations is over-reliance on learned language priors [15]. For any domain, increasing the amount of data will not remove biases inherent in that domain. Thus, instead of increasing the amount of data, Biten et al. [15] suggest augmenting textual training data with context irrelevant objects. For example, changing "a *bus parked on a parking lot*" into "a *banana parked on a parking lot*". The introduced method consists broadly of three steps. The first step is a slight modification to the model architecture. Standard captioning models typically extract visual features from an image using a pretrained object detector. Specifically, an image I is processed by an object detector O to yield bounding box features V:

$$V = \{v_1, v_2, ..., v_n\} \tag{2.19}$$

These features are then combined with an attention mechanism and fed into an LM in order to generate a sentence. To enable data augmentation, the authors concatenate embedded object labels directly with the bounding box visual features. This extends the feature set from V to V'. Given a set of object labels $L = \{l_1, l_2, ..., l_n\}$, the new feature set is:

$$V' = \{v_1, v_2, ..., v_n, l_1, l_2, ..., l_n\} \tag{2.20}$$

The second step is to remove descriptive adjectives from sentences in order to prevent logical errors with inserted objects and to serve as a method of regularization. The final step is to replace the existing object labels with new labels. The authors introduce three sampling strategies. The simplest strategy is uniform sampling, where an object label is randomly selected from a uniform distribution. The second strategy is inverse multinomial sampling. Object labels are sampled from a distribution based on the co-occurrence matrix of all objects present in the dataset. Given an object o_i, an object o_j is more likely to be selected if their co-occurence is low. Inverse multinomial sampling directly addresses bias issues regarding objects that frequently co-occur, it in turn creates a new bias for low co-occurrence objects. The third sampling strategy addresses this issue by continuously updating the co-occurrence matrix as each label is sampled. All strategies improve performance of state-of-the-art, with inverse multinomial sampling achieving the best performance for standard image captioning metrics while continuously updating the co-occurrence matrix achieving the best hallucination specific CHAIR score.

The method by Biten et al. utilizes data scrambling to prevent hallucinations that emerge due to over-reliance on language priors. A different factor that can result in hallucinations are datasets or data domains that are inherently more ambiguous [1]. For example, the top-down perspective and large scale common in RS satellite images can create difficult ambiguous scenarios for VLMs. For example, there might not be enough visual information to correctly classify a beige rectangle in a city as a school, factory, or clinic. In this case hallucinations might arise due to uncertainty caused by lack of information instead of stemming from data bias. The dataset fMoW-mm [1] aims to improve RS image captioning by enriching the input with map metadata, which might contain information not present in the raw satellite images. The authors of fMoW-mm achieve this by combining location, time and weather metadata of the fMoW-rgb dataset with the corresponing Open Street Map tiles to ultimately generate rich image captions using GPT-4o. This type of data augmentation is useful to augmenting the input and thus resulting in richer, more accurate image captions but also useful to train more robust models. The fMoW-mm dataset was used to train the CLIP [20] model for object detection achieving better performance than the standard CLIP and even better performance than GeoRSCLIP[40] and RemoteCLIP [27] which were trained with significantly larger datasets.

While data centric hallucination mitigation methods are effective, they require data that might not be available or at least require model training which can be resource intensive [33]. One alternative are methods that alter inference and token selection directly. One such method is FarSight [33], a decoding strategy developed to prevent two issues resulting from the attention matrix in transformer neural networks. The attention matrix is an $N \times N$ matrix where each cell (i, j) contains a weight representing how heavily token i is influenced by token j. In standard causal text generation, a model is strictly forbidden from looking at future tokens by masking the upper-triangular portion of the attention matrix (setting its values to negative infinity) before applying the softmax function, which turns those probabilities to 0. The first issue tackled by FarSight arises here. The use of the softmax function forces all (unmasked) attention scores to be non-zero, which causes the model to allocate disproportionally large attention to low-information tokens, such as visual backgrounds. The authors call this issue attention collapse. FarSight addresses this issue by exploiting the masked corner of the attention matrix and turning it into an attention register. By strategically placing values into this upper triangle before the softmax operation, FarSight gives the model a safe buffer to dump its excess, unneeded attention. This prevents the excess attention from collapsing onto irrelevant outlier tokens in the actual text. After the softmax operation, FarSight re-applies the causal mask in order to still prevent access to future tokens. The second issue addressed by FarSight is positional information decay, which is the progressive loss of attention to early tokens and therefore to the vision tokens. This happens because the Rotational Position Encoding (RoPE) causes attention weight to decrease as the relative

distance between tokens increases. The causal mask turned attention register is also used to solve this. Inside the attention register matrix P, FarSight applies a linear penalty based on a decay factor σ:

$$P_{i,j} = -(j - i) \cdot \sigma \tag{2.21}$$

Because the total attention across a row (valid tokens + register space) must equal exactly 1, manipulating the register directly changes the actual attention. Because the register's absorption capacity decays for later rows (the progressively diminishing masking rate), the model is forced to allocate an increasingly larger share of its actual attention back to the valid tokens in the lower triangle. FarSight effectively decreases hallucinations produced by LLaVA-1.5, improving POPE and CHAIR scores.

2.2.2.5. Hallucination Detection

Closely related to hallucination mitigation is hallucination detection. Some hallucination detection techniques simply tag generated text that is likely to be hallucinated, others can be more fine-grained to detect hallucinatory tokens to revise them during inference.

One hallucination detection strategy is Semantic Entropy [7]. Semantic Entropy aims to detect when generated text is likely to be hallucinated. For example, this method could be used to refuse answering questions that a model is uncertain about. The method assumes that the entropy of LM generations increases with uncertainty. Thus, the method consists of generating n answers for the same prompt and measuring the entropy in the generations. To prevent different sentence structure and synonyms to inflate entropy scores, the method calculates entropy purely based on the meaning of each answer. This is achieved by measuring bidirectional entailment between two answers. The authors used both LMs and natural inference tools developed specifically for entailment estimation. When the entropy increases above a certain threshold, the model is considered uncertain and the answer is refused. In order to apply this method to longer form generations that contain multiple pieces of information, the authors applied factoid decomposition. For this, a text is divided into individual facts using an LM and for each fact m questions are generated, Semantic Entropy is then calculated for each question to calculate the entropy score for that specific fact.

Semantic Entropy relies on running computationally expensive language models many times, which adds significant cost. Additionally, Semantic Entropy classifies entire sentences or segments with a rigid binary outcome. In contrast, Visual Contrastive Decoding (VCD) aims at detecting hallucinations at the token-level to correct these during inference time. At it's core, VCD measures if a model's output is grounded on visual input or is instead relying on internal knowledge. The main assumption underlying VCD is that VLMs increasingly rely on their language priors with increasing visual uncertainty, thus making it possible to extract a model's language priors by artificially increasing model

uncertainty. To perform VCD each inference step is run twice, once with the original visual input and once with a distorted version of the same input. From the two resulting distributions a new contrastive probability distribution p_{vcd} is computed. Given a model parameterized by θ

$$p_{\text{vcd}}(y \mid v, v', x) = \text{softmax}\left[(1+\alpha)\text{logit}_\theta(y \mid v, x) - \alpha\text{logit}_\theta(y \mid v', x)\right] \quad (2.22)$$

where x denotes the textual query that is provided as input, y is the generated response, v and v' are the probability distributions resulting from the original and distorted visual inputs, and α defines how much to contrast both distributions. VCD thus corrects likely hallucinations from the original distribution by contrasting it with a distribution that, in theory, inherently favors hallucinations. The authors also note, that the aforementioned method might penalize fundamental linguistic standards and common reasoning which would not disappear given the distorted input. In order to account for this and preserve text quality, the authors combine VCD with an Adaptive Plausibility Constraint (APC) to only correct tokens below a certain threshold. Formally, APC is defined as:

$$\begin{gathered}\mathcal{V}_{\text{head}(y_{<t})} = \left\{y_t \in \mathcal{V} \mid p_\theta(y_t \mid v, x, y_{<t}) \geq \beta \max_w p_\theta(w|v, x, y_{<t})\right\}, \\ p_{\text{vcd}(y_t|v,x)} = 0 \text{ if } y_t \notin \mathcal{V}_{\text{head}(t_{<t}}\end{gathered} \quad (2.23)$$

where $\mathcal{V}$ is the output vocabulary of the VLM, β is a hyperparameter in $[0, 1]$ for controlling the truncation threshold of the next-token distribution. A higher β value results in more aggressive truncation, keeping only high probability tokens. VCD improved POPE scored for LLaVA 1.5, Qwen-VL and InstructBLIP across 3 different datasets, MSCOCO, A-OKVQA and GQA. Worthy of note is the recent work by Yin et al. [11], which puts the VCD mechanism into question. The authors hypothesize that the performance gains caused by VCD are mostly coincidental. They state that in practice APC effectively reduces decoding to greedy decoding which might artificially boost performance in certain benchmarks. Additionally, a highly distorted input image might cause a VLM to consistently say objects are not visible and as VCD punishes the distorted input responses this might boost "Yes" responses in binary benchmarks.

Another approach is the LURE hallucination reviser as introduced by Zhou et al. [8]. While not purely developed for hallucination detection, the LURE pipeline flags objects that are likely to be hallucinated based on a statistical analysis of hallucinations which looked at object co-occurrence, uncertainty and textual position. A model was then trained to revise the captions with flagged objects. Their results showed that tagging objects as likely hallucinations based on common patterns might be enough to correct hallucinations in image captions.

2.2.2.6. Remote Sensing Hallucinations

While not much work exists surrounding RS hallucinations, some research does exists. Basak et al. [3] explored hallucinations in drone imagery. Their main contribution regarding hallucinations is the Labelled Illusion Dataset (LID). Here, the authors provide

a set of manually evaluated VLM-generated captions. Additionally, the authors review how well LMs like GPT-4 can detect hallucinations in drone imagery, finding that even the best performing model had an error rate of 27.88%. Anderson et al. [1] also explore RS hallucinations. Their main contribution is the fMoW-mm dataset which aims at reducing hallucinations in RS image captioning by providing the model with extensive metadata, including rich map data.

3. Methods

This chapter explains the different techniques and approaches used throughout the thesis. This includes different hallucination detection techniques, Vision-Language Model (VLM) Decoding Strategies, the process used to detect isolate Remote Sensing (RS) Caption hallucinations in an existing dataset, and the pipeline associated with training a hallucination reviser.

3.1. Hallucination Detection

Throughout this thesis, the main form of hallucination detection explored is object hallucination detection. Object hallucination detection identifies when a VLM describes an object absent from an image. These errors surface in captions or answers to visual questions (VQA). Some detection methods flag the entire generated text. Other, more precise methods isolate and list each hallucinated object.

3.1.1. Object Hallucination Detection in Image Captions

The main hallucination detection methods explored in this thesis are based on visual probing questions. The question structure used was taken from the Polling-based Object Probing Evaluation (POPE) [10]. The intent of these questions as used in POPE is to evaluate a VLM's hallucination rate using a binary detection task. For a given image, a reference list of known present and absent objects is provided. A VLM is then prompted for the existence of each object and the error rate is measured. The question structure is as follows:

> "Is there a *{object}* in the image?"

In this thesis, the same question structure is adapted to be used, not only as a VLM hallucination rate metric, but to actually use VLMs to detect and list object hallucinations in generated captions. For this purpose, a binary "Yes" or "No" answer is not sufficient. Instead, a continuous confidence score is calculated. The simplest approach would be to calculate the sequence level probability of the generated answer. Given generated text $\boldsymbol{y} = y^1 y^2 ... y^n$ and a prompt $\boldsymbol{x}$, the sequence likelihood score is defined as:

$$\log p(\boldsymbol{y}|\boldsymbol{x}) = \sum_{t=1}^{l} \log p(y^t | y^1 ... y^{t-1}, \boldsymbol{x}) \tag{3.1}$$

by Ren et al. [41]. Because models tend to underestimate the likelihood of longer generations [42], the length normalized likelihood is also used:

$$\log \overline{p}(\boldsymbol{y}|\boldsymbol{x}) = \frac{1}{l} \sum_{t=1}^{l} \log p(y^t|y^1 ... y^{t-1}, \boldsymbol{x}) \tag{3.2}$$

While sequence likelihood scores do have some predictive power, it is weak. Additionally, confidence in free-form generation is hard to measure because different answers might have the same meaning [7]. To circumvent these issues, the confidence scores in this thesis are calculated using an adapted version of the "Sample and Eval" method introduced by Ren et al. [41]. Essentially, the object probing questions are reframed as binary multiple-choice questions like so:

"Is there a *{object}* in the image?
A) Yes
B) No
Letter:"

In this way, the entire meaning of an answer is forced into a single token. Additionally, the answer space is reduced to two possible answers (or tokens): "A" and "B". We can thus compute the model's confidence that an object o is visible in image i solely based on the token logits l_{A} for "A" and l_{B} for "B" and the softmax function:

$$p(\text{visible}|i, o) = \frac{e^{l_{\mathrm{A}}}}{e^{l_{\mathrm{A}}} + e^{l_{\mathrm{B}}}} \tag{3.3}$$

Besides "Sample and Eval", Ren et al. [41] also explored a different question format, namely "Sample and Select". Instead of a binary question, "Sample and Select" aggregates multiple options and builds a (non-binary) multiple-choice question. Ren et al. [41] used this method for self-evaluation of Language Models (LMs), thus the goal was to select a single most confident answer. The authors selected the highest confidence generated answer. Given a question $\boldsymbol{x}$, a set of possible answers $\boldsymbol{y}$ and their corresponding multiple choice letters $c = \{\mathrm{A}, \mathrm{B}, ...\}$:

$$\hat{\boldsymbol{y}} = \arg\max_i p(c_i|\ \boldsymbol{x}, \{c\boldsymbol{y}\}) \tag{3.4}$$

The negative of probability of the "None of the above" option is then used as the confidence score for the selected answer:

$$-p(c_{\text{nota}}|\boldsymbol{x}, \{c\boldsymbol{y}\}_{+\text{nota}}) \tag{3.5}$$

This thesis adapts this method for the vision domain using the following prompt:

"What object is visible in the provided image?
A) {object_1}
B) {object_2}
...
D) {object_n}
E) None of the above
Letter:"

For object detection, the goal is not to select a single best answer, but instead to assign a confidence score to every object. Thus, the confidence that an object o with corresponding letter c_o is visible in image i is:

$$p(\text{visible}|i, o) = \frac{e^{l_{c_o}}}{e^{l_{c_o}} + e^{l_{c_{\text{nota}}}}} \tag{3.6}$$

with $l_{c_{\text{nota}}}$ being the logit value of the corresponding letter of the "None of the above" option. As multiple objects probed by a single question might be visible, their position in the question becomes relevant. The approach used in this thesis to correct position bias is to run the same Multiple-Choice Question (MCQ) for every permutation of the objects and take the average.

3.1.2. Object probing questions

In this thesis, object probing questions are used to detect hallucinations using VLMs. Benchmarking different VLMs and different decoding strategies requires datasets that include two different types of questions per image, positive and negative questions. Positive questions probe for an object that is actually visible in a given image. Negative questions probe for objects that are not present in the same image. When testing hallucination detection performance, it is important for the negative questions to probe for objects that are likely to be hallucinated. One approach is to sample objects that are present the rest of the dataset but are not present in the given image. The POPE [10] benchmark introduces three different sampling strategies. The "random" sampling assigns all objects that appear in the dataset with the same probability. The "popular" sampling strategy ranks all objects by frequency in the dataset, and the most popular objects are sampled. The "adversarial" sampling strategy ranks all objects by co-occurrence frequency with the reference objects present in the image, the objects with highest co-occurrence with the reference objects are sampled. RS-POPE adapts this method for RS imagery and builds upon it by sorting images by creating data splits with varying difficulty based on object density for more fine-grained benchmarking.

Sampling negative objects to simulate hallucinations is a flexible approach that can be applied to any dataset with exhaustive object labels. Even so, hallucinations are still poorly understood and creating realistic object hallucination analogs is not a trivial task. This is especially the case for the RS domain where research on hallucinations is still scarce. For this reason, this thesis adapts the Labelled Illusion Dataset (LID) [3] for hallucination detection. The LID dataset contains VLM-generated captions of aerial imagery with extensive human labeling, including object hallucinations. Using the labeled hallucinations of LID to build the negative questions required for object probing bypasses the need to understand and model realistic hallucinations in the RS domain. Additionally, it allows for contrasting the hallucination detection performance of models on modeled hallucinations with real object hallucinations.

Reference based object probing questions are a good benchmarking tool. This thesis also explores how to use object-probing questions on caption datasets that do not have object references. To check for object hallucinations in an image caption, a VLM is used to extract all objects that are present. An object probing question is generated for each object in order to estimate the model's certainty that the object is present in the image.

3.1.3. VLM Decoding Strategies for object detection

LM Decoding is the iterative cycle of generating a token sequence. This process encompasses the forward pass, the probability transformation, the actual selection of the token, and the autoregressive update. The first step of the decoding loop is the logit computation. At this step, a model processes the current context to output a vector of raw scores (logits), one for each possible token. These logits are then converted into a probability distribution. The last step before the autoregressive update is token selection, where different strategies can be applied to pick the actual discrete token y_t.

A core part of this thesis is to explore how different decoding strategies can be leveraged to detect hallucinations in VLMs. This includes different ways to calculate the logits and different token selection strategies.

3.1.3.1. Regular Decoding

In this thesis, Regular Decoding is the standard decoding practice used in text generation by VLMs. Given visual input $\boldsymbol{v}$, textual input $\boldsymbol{x}$ and a VLM parameterized by $\boldsymbol{\theta}$, regular decoding can be formulated as:

$$\begin{aligned} y_t &\sim p_{\boldsymbol{\theta}}(y_t|\boldsymbol{v}, \boldsymbol{x}, \boldsymbol{y}_{<t}), \\ &\propto \exp \text{logit}_{\boldsymbol{\theta}}(y_t|\boldsymbol{v}, \boldsymbol{x}, \boldsymbol{y}_{<t}), \end{aligned} \tag{3.7}$$

where y_t is the token at time step t and $\boldsymbol{y}_{<t}$ is the token sequence generated until $(t-1)$, as defined by Leng et al. [13]. Additionally, the probability for each possible token is calculated using the softmax function. To get the true probability of token y_t, the model takes the corresponding logit and divides it by the sum of the exponentiated logits for all

possible tokens in the vocabulary. Given logits $\boldsymbol{l}$ generated by a model parameterized by $\boldsymbol{\theta}$ with $\boldsymbol{v}$ and $\boldsymbol{x}$ as inputs:

$$p(y_t) = \frac{e^{l_{y_t}}}{\sum_{y'} e^{l_{y'}}} \tag{3.8}$$

3.1.3.2. Binary Decoding

Binary Decoding adapts Regular Decoding for object confidence estimation as laid out in Section 3.1.1. The goal in Binary Decoding is not to generate free-form text, instead it is purely to compute the relative confidence between two specific tokens (e.g. "A" and "B"). Thus, the token vocabulary of the model is effectively reduced to two tokens, hence the name "Binary Decoding". Given logits l_{A} and l_{B} generated by a model parameterized by $\boldsymbol{\theta}$ with $\boldsymbol{v}$ and $\boldsymbol{x}$ as inputs, the token probabilities are defined as:

$$\begin{aligned} p(y_1 = \mathrm{A}) &= \frac{e^{l_{\mathrm{A}}}}{e^{l_{\mathrm{A}}} + e^{l_{\mathrm{B}}}}, \\ p(y_1 = \mathrm{B}) &= 1 - p(y_1 = \mathrm{A}). \end{aligned} \tag{3.9}$$

Unlike regular decoding, binary decoding is not autoregressive and the token is not selected stochastically. Stochastic sampling is usually used to improve quality and variety in free-form generated text. As only a single token is generated in this case, the token is selected based on a threshold τ:

$$y_1 = \begin{cases} \mathrm{A} \text{ if } p(y_1 = A) > \tau \\ \mathrm{B} \text{ else} \end{cases} \tag{3.10}$$

3.1.3.3. MCQ Decoding

Like Binary Decoding, MCQ Decoding aims to apply the confidence estimation logic as laid out in Section 3.1.1. The main difference to binary decoding is that MCQ decoding calculates the confidence of n tokens after a single forward pass. This process calculates the probability of all tokens in $\boldsymbol{y}_{\mathrm{MCQ}}$, each token corresponding to an MCQ answer, by contrasting it with an addtional token y_{nota} corresponding to the "None of the above" answer. Given logits l_i generated by a model parameterized by $\boldsymbol{\theta}$ with $\boldsymbol{v}$ and $\boldsymbol{x}$ as inputs and $y_i \in \boldsymbol{y}_{\mathrm{MCQ}}$, the token probabilities are defined as:

$$p(y_i) = \frac{e^{l_i}}{e^{l_i} + e^{l_{\mathrm{nota}}}} \tag{3.11}$$

MCQ Decoding outputs a set $\boldsymbol{y}_o$ of all tokens in $\boldsymbol{y}_{\text{MCQ}}$ with a probability $p(y_i)$ bigger than threshold τ:

$$\boldsymbol{y}_o = \left\{ y_i | y_i \in \boldsymbol{y}_{\text{MCQ}}, p(y_i) > \tau \right\} \tag{3.12}$$

3.1.3.4. Visual Contrastive Decoding

Visual Contrastive Decoding (VCD) [13] is a technique that aims at detecting and mitigating hallucinations in VLMs. More concretely, VCD targets hallucinations that occur when a VLM is not sufficiently grounded on the provided visual input $\boldsymbol{v}$ and therefore relies on its language priors. The core assumption is that it is possible to purposefully trigger hallucinations in VLMs by distorting the visual input $\boldsymbol{v}$. Thus, VCD runs the forward pass step twice generating two sets of logits, once with visual input $\boldsymbol{v}$ and once with a distorted version $\boldsymbol{v}_d$. Given a model parameterized by θ, textual input $\boldsymbol{x}$ and the set of generated tokens $\boldsymbol{y}_{<t}$ before time step t:

$$\begin{aligned} \boldsymbol{l_v} &= \text{logits}_{\boldsymbol{\theta}}(y_t |\ \boldsymbol{v}, \boldsymbol{x}, \boldsymbol{y}_{<t}), \\ \boldsymbol{l_d} &= \text{logits}_{\boldsymbol{\theta}}(y_t |\ \boldsymbol{v}_d, \boldsymbol{x}, \boldsymbol{y}_{<t}). \end{aligned} \tag{3.13}$$

The logic behind VCD is that if a high logit value $l_{\boldsymbol{v}_i}$ is truly based on visual input $\boldsymbol{v}$, then distorting the input should lower the models confidence, thus $l_{\boldsymbol{v}_i} > l_{\boldsymbol{d}_i}$. Conversely, if the logit $l_{\boldsymbol{d}_i}$ generated using the distorted input $\boldsymbol{v}_d$ has the same or bigger value than the $l_{\boldsymbol{v}_i}$, then $l_{\boldsymbol{v}_i}$ can not be based on visual input. Following this logic, VCD contrasts the logits $\boldsymbol{l_v}$ generated using visual input $\boldsymbol{v}$ with logits $\boldsymbol{l_d}$ generated using the distorted input $\boldsymbol{v}_d$:

$$\begin{aligned} p_{\text{VCD}}(y_t \mid \boldsymbol{v}, \boldsymbol{v}_d, \boldsymbol{x}, \boldsymbol{y}_{<t}) = \text{softmax}\ [(1+\alpha)\text{logit}_\theta(y_t | \boldsymbol{v}, \boldsymbol{x}, \boldsymbol{y}_{<t}) \\ -\alpha \text{logit}_\theta(y_t | \boldsymbol{v}_d, \boldsymbol{x}, \boldsymbol{y}_{<t})] \end{aligned} \tag{3.14}$$

with α amplifying the differences between both logit distributions. VCD punishes tokens whose logit values become higher with the distorted input $\boldsymbol{v}_d$.

The authors of the original paper mention that text output using the distorted visual input $\boldsymbol{v}_d$ might still be linguistically high quality text. Therefore, VCD could punish high plausible tokens and promote low confidence tokens. The proposed solution is to constrain the model's vocabulary to tokens above a dynamic threshold based on the models highest confidence token. This process is called the Adaptive Plausibility Constraint (APC) and is formally defined as:

$$\begin{gathered}\mathcal{V}_{\text{head}} = \left\{ y_t \in \mathcal{V} \mid p_\theta(y_t|\boldsymbol{v}, \boldsymbol{x}, \boldsymbol{y}_{<t}) \geq \beta \max_w p_\theta(w|\boldsymbol{v}, \boldsymbol{x}, \boldsymbol{y}_{<t}) \right\}, \\ p_{\text{VCD}}(y_t \mid v, v', x) = 0, \text{if } y_t \notin V_{\text{head}}(y_{<t})\end{gathered} \tag{3.15}$$

where $\mathcal{V}$ is the total vocabulary of the model and $\beta \in [0, 1]$ defines how aggressively to truncate the token distribution, with a large β only keeping high confidence tokens. Applying VCD with APC to regular decoding yields the following formulation:

$$\begin{gathered}y_t \sim \text{softmax}[(1+\alpha)\text{logit}_\theta(y_t|\boldsymbol{v}, \boldsymbol{x}, \boldsymbol{y}_{<t}) - \alpha \text{logit}_\theta(y_t|\ \boldsymbol{v}_d, \boldsymbol{x}, \boldsymbol{y}_{<t})], \\ \text{subject to } y_t \in V_{\text{head}}(\boldsymbol{y}_{<t})\end{gathered} \tag{3.16}$$

In this thesis, VCD was also applied to binary decoding in order to explore the effects of VCD on hallucination detection. For this, the process was adapted to operate strictly on the two-token vocabulary (e.g. $y \in \{\text{A}, \text{B}\}$). First, the original probabilities are calculated as described in Section 3.1.3.2 for token y. The probability is then used to calculate the APC threshold. Let $l_{\boldsymbol{v},y}$ be the logit for token y:

$$\begin{aligned}p(y|\boldsymbol{v}, \boldsymbol{x}) &= \frac{e^{l_{\boldsymbol{v},y}}}{e^{l_{\boldsymbol{v},\text{A}}} + e^{l_{\boldsymbol{v},\text{B}}}}, \\ \tau_{\text{APC}} &= \beta \max_{w \in \{\text{A},\ \text{B}\}} p(w|\ \boldsymbol{v}, \boldsymbol{x})\end{aligned} \tag{3.17}$$

If a token's probability falls below the dynamic threshold τ_{APC}, it is eliminated by setting its logit to $-\infty$. Otherwise, the standard VCD penalty is applied by contrasting the original logits $l_{\boldsymbol{v},y}$ with the logits $l_{\boldsymbol{d}_y}$ generated with distorted visual input $\boldsymbol{v}_d$, amplified by α:

$$\boldsymbol{l}_{\text{VCD}}(y) = \begin{cases} -\infty & \text{if } p(y|\boldsymbol{v}, \boldsymbol{x}) < \tau_{\text{APC}} \\ (1+\alpha) l_{\boldsymbol{v},y} - \alpha l_{\boldsymbol{d},y} & \text{otherwise} \end{cases} \tag{3.18}$$

The final confidence estimation or token selection proceeds like in section Section 3.1.3.2. VCD is applied to MCQ decoding analogously.

3.1.4. VCD Distortion Configurations

3.1.4.1. Diffusion Noise

Forward diffusion applies visual uncertainty following the implementation of the original VCD repository[1]. The algorithm generates a beta schedule over 1000 steps using a sigmoid

[1] https://github.com/DAMO-NLP-SG/VCD

function across a linear space from −6 to 6. A secondary function applies random normal noise corresponding to a specific timestep T. The system caps the maximum noise step at 999.

3.1.4.2. Speckle Noise

Multiplicative speckle noise is implemented as follows:

$$I' = I + I \times n, \text{where } n \sim N(0, \text{speckle_intensity}^2) \tag{3.19}$$

The variable n can be set from 0 to 1 to control the intensity of the noise.

3.1.4.3. Color MAE Masking

Color Masked Autoencoder (MAE) masking drops image patches based on frequency domain filtering. The algorithm divides the input image into discrete patches governed by a `patch_size` parameter, which is set to 64 pixels. It calculates the grid dimensions by performing integer division on the original image height and width. The system generates a random normal noise matrix matching the grid dimensions and shifts it into the frequency domain using a Fast Fourier Transform. To structure the spatial frequency filters, the algorithm maps coordinate matrices x and y linearly from −1 to 1. It then calculates the spatial frequency magnitude f.

$$f = \sqrt{x^2 + y^2} + 10^{-8} \tag{3.20}$$

The calculation adds 10^{-8} to prevent division by zero. The algorithm then applies a spatial frequency filter S based on the selected color parameter:

- Red: A low-pass filter using $S = 1/f^{1.5}$
- Blue: A high-pass filter using $S = f^{1.5}$
- Green: A band-pass filter $S = \exp(-((f - 0.4)^2/0.05))$
- Purple: A very high-pass filter using $S = f^{2.5}$

3.1.5. Semantic Entropy

Semantic Entropy is a hallucination detection approach introduced by Farquhar et al. [7]. The technique is based on the assumption that as model uncertainty increases so does the entropy of its generated answers. Thus, it is possible to calculate how likely a model is to hallucinate on input $\boldsymbol{x}$ by measuring the entropy of n generated answers $\boldsymbol{y}(\boldsymbol{x})$. The main technical issue that Semantic Entropy tackles is that model answers may contain different words but have identical meanings. This approach works by using language models to cluster n answers $\boldsymbol{y}(\boldsymbol{x})$ into m distinct semantic groups. Semantic Entropy is then calculated based on the empirical categorical distribution of these clusters. The discrete Semantic Entropy (SE) is defined as:

$$\mathrm{SE} = -\sum_{C} P(C) \ln(P(C)) \tag{3.21}$$

where the sum iterates over each semantic cluster C. The term $P(C)$ is represent the probability of a specific cluster, which is calculated as the number of answers assigned to that cluster divided by total amount of generated answers n:

$$P(C) = \frac{\text{number of answers in cluster } C}{n} \tag{3.22}$$

Importantly, the process as described thus far relies on simple questions with a single correct and verifiable answer. The authors take this one step further to detect hallucination in long-form generated text. To achieve this, the information dense text needs to be decomposed into individual factoids. For each fact, an LM is used to generate a question for which the fact could have been the answer. Factoid decomposition and question generation effectively enable Semantic Entropy to be used for longer form text.

Farquhar et al. [7] applied this method to textual question answering using LLMs. This thesis adapts this method for detecting object hallucinations in RS captions, the pipeline used is depicted in Figure 3.1.

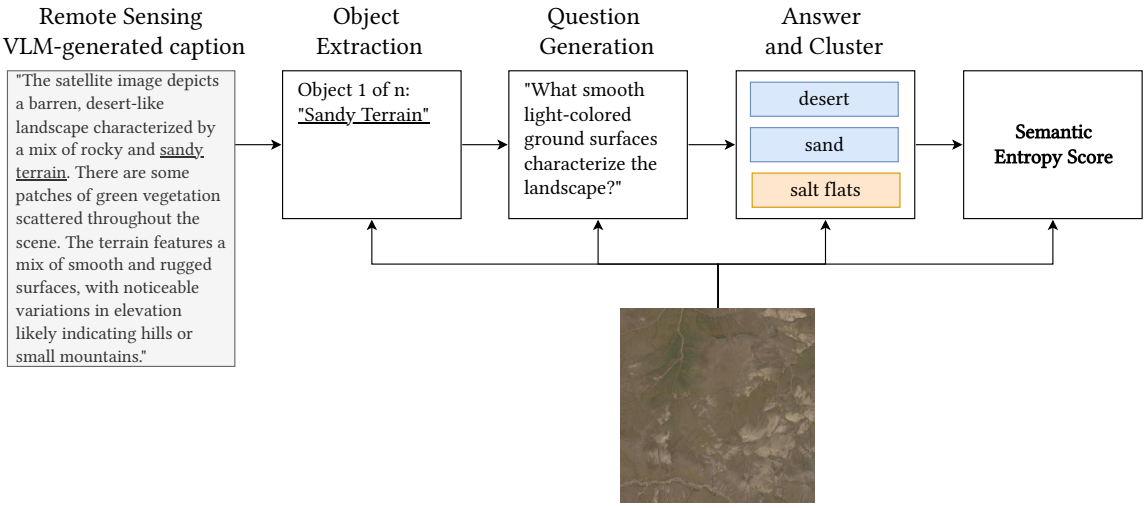


Figure 3.1: Visual Semantic Entropy pipeline. Each step of the pipeline is performed by a VLM grounded on the original RS image.

This domain shift presents unique challenges that require targeted modifications. First, RS image captions are often information dense and contain multiple objects. To probe individual object hallucinations, this thesis adapts the process of factoid decomposition into object extraction. The goal in object extraction is to use a language model to extract all

the objects from a given image caption. RS images span a wide range of scales, resolutions and scenes, thus it is not a trivial task to extract solely the objects that are relevant in a particular RS scene. A highly specific definition of what objects should be extracted or a predefined objects list might not translate well between datasets or even between images in a single varied dataset. The next step in the pipeline is generating visual object probing questions. This process inherits the same issues as before. The goal is to generate a visual question for which the probed object is the only correct answer. The question should be specific enough to probe for a single object, but not contain so much information that the VLM would be able to answer by lexical context alone. Additionally, the question should have information that stems mostly from the original image caption, as the caption is what is being evaluated for hallucinations. To address this issue, object extraction and question generation are performed in a single VLM conversation. The original image is provided alongside the caption. This provides the VLM with as much domain and image specific context as possible, while allowing the model to select which objects are good candidates for visual object probing. An example of this process is depicted in Figure 3.1.

The generated visual questions are fed alongside the RS image to a VLM and n answers are sampled. To improve semantic clustering, the original image and caption are given to a VLM for extra context. This alleviates the problem when two different words or concepts are only semantically equivalent for certain images. Finally, discrete Semantic Entropy is calculated based on the semantic clusters.

3.2. Hallucination Dataset Creation

The goal of the following method is detect and isolate hallucinations in an existing RS image captioning dataset, namely Git-10M. This is a four step-process. It starts with automatic extraction and confidence estimation for all objects using the process described in Section 3.1.1. Both object extraction was done with Qwen3.5 9b and confidence estimation with RS-LLaVA 7b. All objects are the sorted by confidence score. Starting from the object with the lowest confidence, object image pairs were manually evaluated using a Web-app. A screenshot of the developed web-app can be seen in Figure 3.3. As Git10-M is a vast dataset, only a small subset can be manually evaluated. The whole pipeline is depicted in Figure 3.2.

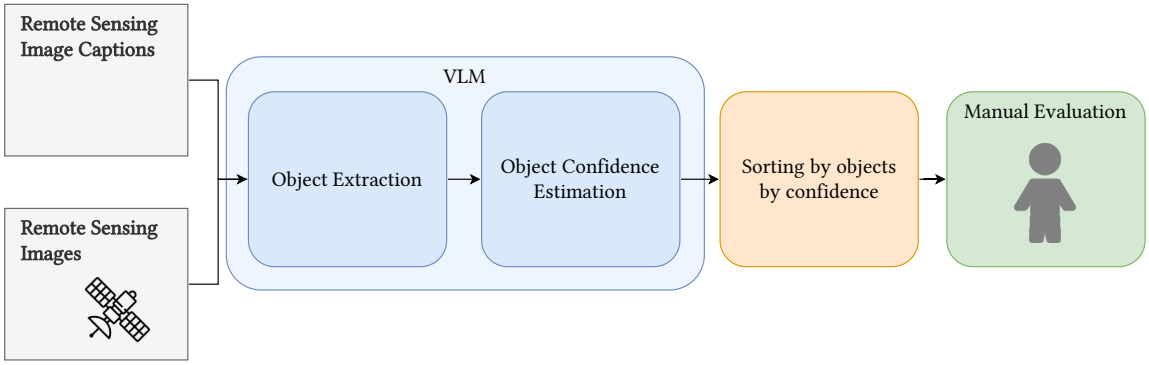


Figure 3.2: RS-Image Caption Reviser Pipeline: Creating masked training captions.

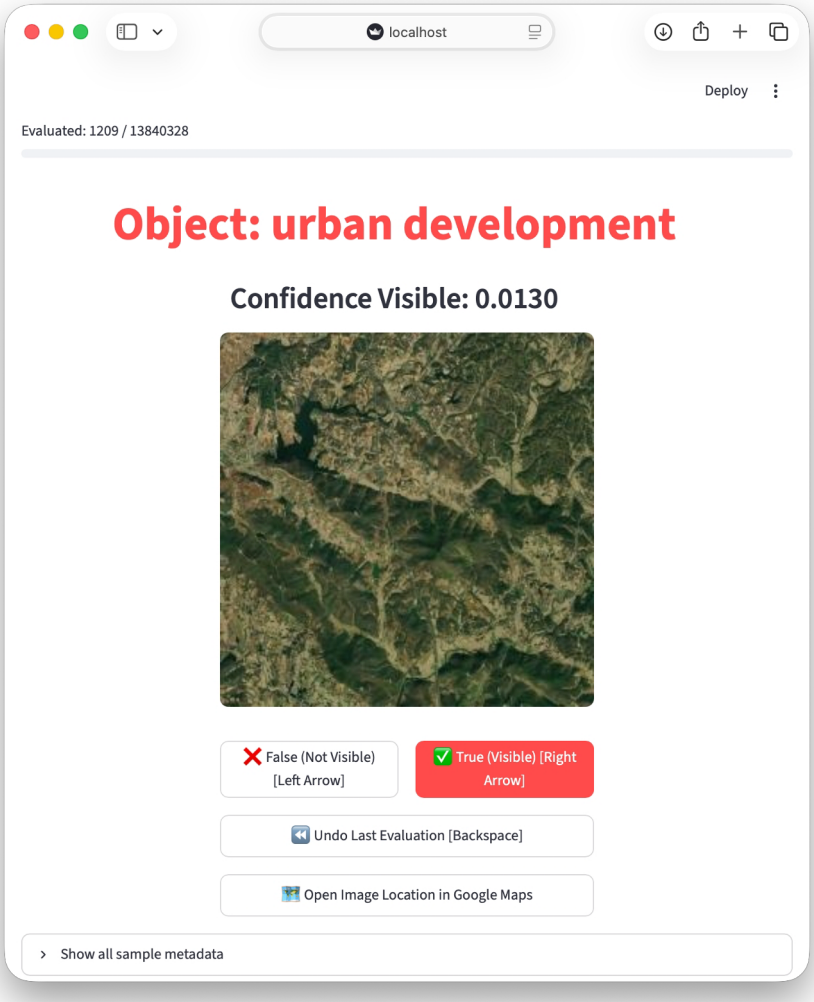


Figure 3.3: Interface used to manually evaluate potentially hallucinated objects in RS images.

3.3. Hallucination reviser

The hallucination reviser used in this thesis is based on LURE [8]. The LURE framework revolves around masking objects that are likely to be hallucinated in image captions and training a VLM to revise the masked captions into less hallucinatory versions. This thesis explores training a RS image caption reviser based on LURE, this is implemented in two separate stages. The training stage and the deployment stage.

The training phase includes building a training dataset. This entails inserting fake hallucinations into each caption and masking high uncertainty objects. For this, a subset $\boldsymbol{Y}$ of all image captions is isolated. For each caption $y \in \boldsymbol{Y}$ we get a list of objects $\boldsymbol{O}$. Based on all object lists $\boldsymbol{O}$, a co-occurrence matrix is built. For each caption y we get the object o_c with the highest co-occurrence among all objects present in caption y. The high-occurrence object o_c is inserted as a fake hallucination in caption y. The set of all captions with inserted objects is H_{old}.

Additionally, the binary object confidence estimation pipeline laid out in Section 3.1.3.2 is run on all objects in O. O_u contains all objects present in the caption that fall below the confidence threshold τ. For each caption in H_{old}, all objects in O_u are masked with "[IDK]", with "IDK" standing for "I don't know". The set of captions with an inserted object o_u and masked objects O_u is H_{new}.

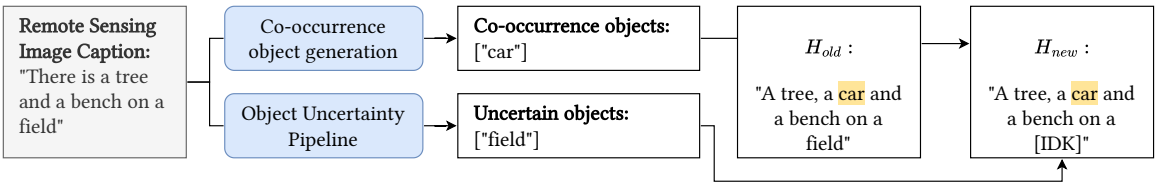


Figure 3.4: RS-Image Caption Reviser Pipeline: Creating masked training captions.

To finally train the reviser, the RS-LLaVA model is fine-tuned using the H_{new} dataset via Low-Rank Adaptation (LoRA). During this stage, the training data is structured into a conversational format where the user instruction provides the image alongside the captions in H_{new}, and the target are the original captions in Y.

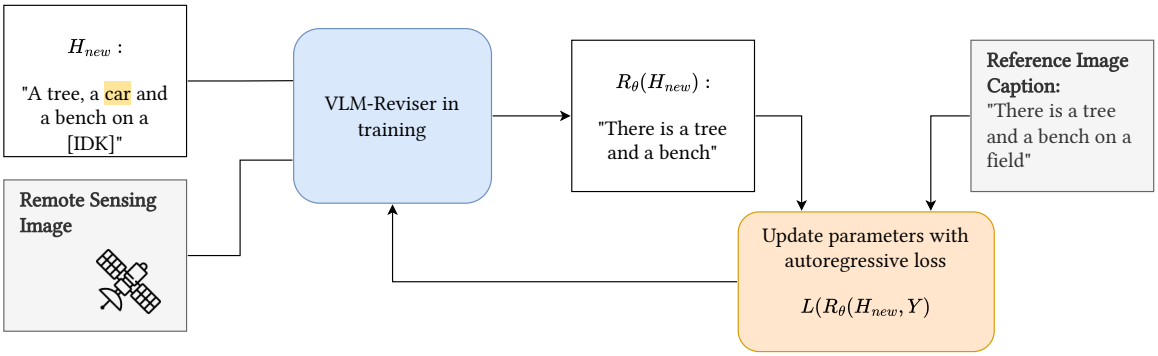


Figure 3.5: RS-Image Caption Reviser Pipeline: Training the reviser.

For deployment, the process is similar. To revise a caption, the objects are extracted and run through the binary object confidence estimation pipeline. All objects with confidence below threshold τ are masked with "[IDK]". The captions with masked low confidence objects is fed to the reviser, which outputs a revised version.

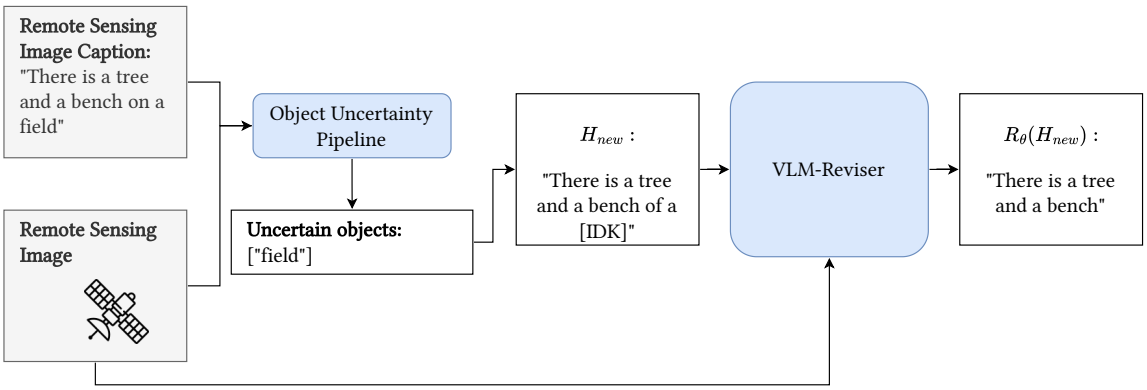


Figure 3.6: RS-Image Caption Reviser Pipeline: Deployment.

3.3.1. Semantic CHAIR Evaluation

The CHAIR [39] metric measured the ratio of hallucinated to real objects. The two variants of CHAIR are CHAIR_i which measures the ratio of hallucinated objects in a caption and CHAIR_s which measures the ratio of sentences containing hallucinations. Crucially, CHAIR relies on an exhaustive list of reference objects to determine what is hallucination. The datasets used with the hallucination reviser do not contain object reference lists. To alleviate this issue, the evaluation in this thesis uses an approximation of CHAIR, namely Semantic-CHAIR. Instead of string matching the extracted objects from an image caption to the reference list, Semantic-CHAIR uses a VLM to check if any of the extracted objects are semantically equivalent to any objects present in the reference caption, given the RS image as context. This is less precise than CHAIR but allows for an approximated evaluation when object references are not exhaustive.

4. Experimental Setup and Datasets

This chapter presents the datasets utilized for the experiments in this thesis. Additionally, parameters and technical details are stated to facilitate replication.

4.1. Datasets

Two types of datasets were used throughout this thesis. First, Object-Probing datasets contain images and object-probing questions based on object labels. Second, Remote Sensing (RS) Image Captioning datasets contain pairs of RS images and Vision-Language Model (VLM) generated captions.

4.1.1. Object-Probing Datasets

To test the hallucination detection performance of different VLMs, different binary object probing datasets were used.

4.1.1.1. POPE

The Polling-based Object Probing Evaluation (POPE) [10] dataset turns hallucination detection into a binary classification task. The POPE subset used in this thesis is the same as the one used by Leng et al. [13] and was taken from the corresponding GitHub repository[2]. The subset contains 1500 general computer vision images from the aokvqa [43], coco [44] and gqa [45] datasets. The dataset provides three different sets of object probing questions with varying difficulty: "random", "popular" and "adversarial". Each split contains a balanced set of 6 object probing questions, three positive and three negative. For a more detailed description on how the different splits differ consult Section 2.2.2.3. This POPE subset is used for direct comparison with the results in the original Visual Contrastive Decoding (VCD) paper by Leng et al. [13].

4.1.1.2. RS-POPE

RS-POPE [12] adapts the POPE framework for the RS domain. It contains images from the DIOR [46], DOTA [47], FAIR1M [48], AI-TOD [49], and VisDrone2019 [50] datasets covering a wide variety of RS images. Like POPE, this dataset also provides different splits with varying difficulty. RS-POPE contains a total of 9 splits divided by negative objects sampling strategy ("random", "popular", "adversarial") and by object density ("easy", "medium", "hard"). To test hallucination detection on the most realistic samples, the split

[2] https://github.com/DAMO-NLP-SG/VCD/tree/master/experiments/data/POPE

used in this thesis is the "adversarial hard" subset. This split contains a roughly balanced set of 1000 object probing questions, 408 positive and 592 negative.

4.1.1.3. Labelled Illusion Dataset (LID)

The Labelled Illusion Dataset (LID) [3] provides generated captions and fine-grained human annotations of hallucinations. It sources aerial imagery from the AeroCaps [3] and VisDrone [51] datasets. For this thesis, all 217 samples containing a manually labeled object hallucination were selected. The labeled hallucinations were then used to create a binary object probing dataset. As no samples contain more than one hallucination, the first non-hallucinated object from each sample was selected to balance the dataset. This resulted in balanced pair of positive and negative object probing questions for each image. The resulting dataset complements the POPE and RS-POPE datasets by providing real hallucinations instead of sampling-based negative objects.

4.1.2. Remote Sensing Captioning Datasets

To test how well different frameworks can detect or revise hallucinations in existing captions, three RS captioning datasets were used.

4.1.2.1. Git-10M

The Git-10M [4] dataset is a large-scale RS image captioning dataset containing 10 million image-caption pairs mostly taken from Google Earth. The image captions of Git-10M are particularly visually descriptive and thus are well suited for caption hallucination detection. The resolution varies from 0.5 – 128 m/pixel. A subset of 5000 images was isolated as a training set and a subset of 1500 images as the test set.

4.1.2.2. GAIA

The GAIA [2] dataset is contains 40,201 unique RS images. Each image is accompanied by five synthetic captions generated using GPT-4o. As GAIA's captioning process relied heavily on rich metadata, the captions are detailed and contain RS terminology but are not particularly visually descriptive. The spatial resolution is variable with 85.8% of the dataset in the 10-100 m/pixel range. To align with the testing scale of Git-10M, a subset of 1,500 samples from GAIA was isolated and utilized exclusively as a test set.

4.1.2.3. RSTeller

The RSTeller [5] dataset comprises over 1.3 million RS image patches sourced from the National Agriculture Imagery Program (NAIP). Each image features a high-resolution ground sample distance of 0.6 meters and is paired with two distinct VLM-generated captions and made use of OpenStreetMap tags as metadata. A subset of 1500 samples from RSTeller was isolated to serve as an additional test set.

4.2. Models

All models ran on GPUs in half-precision (`float16` or `bfloat16`). Experiments measuring next-token logits bypassed autoregressive decoding, leaving generation parameters like temperature, top-p, top-k, and max length unconfigured.

4.2.1. LLaVA 1.5 7B

The `liuhaotian/llava-v1.5-7b` [25] checkpoint pairs a Vicuna backbone with a Contrastive Language-Image Pretraining (CLIP) visual encoder through a projection layer. The official LLaVA library loaded the model via `load_pretrained_model` without a base-model override. The setup used the `llava_v1` conversation template and the default image processor.

4.2.2. RS-LLaVA 7B

RS-LLaVA (`BigData-KSU/RS-llava-v1.5-7b-LoRA`) [28] applies Low-Rank Adaptation (LoRA) fine-tuning to the LLaVA-1.5 architecture. It replaces the Vicuna backbone with `Intel/neural-chat-7b-v3-3`. The LLaVA library loaded the model by applying the LoRA weights directly onto this new base model. It retained the `llava_v1` template.

4.2.3. GeoChat 7B

Fine-tuned for RS imagery, `MBZUAI/geochat-7B` [52] shares the LLaVA-1.5 architecture. A custom `GeoChatLlamaForCausalLM` class loaded the model with mismatched sizes ignored. Fast tokenization was disabled for the `AutoTokenizer`, and conversations used the `vicuna_v1` template. The vision tower initializes with 504-pixel position embeddings, but the image processor defaults to 336 pixels. To bridge this gap, `clip_interpolate_embeddings` resized the embeddings back to 336×336 (patch size 14) in full precision before inference.

4.2.4. Qwen3-VL 8B

`Qwen/Qwen3-VL-8B-Instruct` [26] is an instruction-tuned model that processes text and images jointly through a unified transformer. The `Qwen3VLForConditionalGeneration` class loaded the model using Flash Attention 2, automatic device placement, and `bfloat16` precision. An `AutoProcessor` handled both tokenization and image preprocessing.

4.2.5. Qwen3.5 9B

The `Qwen/Qwen3.5-9B` [53] checkpoint was loaded through `AutoModelForImageTextToText`. It used the same `bfloat16` precision, Flash Attention 2, and automatic device placement as the previous model. The chat template disabled the model's internal reasoning mode (`enable_thinking=False`) during input construction.

4.2.6. Caption Reviser

To mitigate hallucinations in existing descriptions, a dedicated caption reviser was fine-tuned built upon a merged RS-LLaVA 7B base model following the LURE framework

[8]. LoRA replaced full-parameter fine-tuning to ensure parameter efficiency. The LoRA adapter used a rank of 128, a scaling α of 256, and a 0.05 dropout rate. These modules targeted all major linear projection layers in the attention and feed-forward networks to maximize adaptation capacity. Specific targets included the `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj` modules. Training ran for 3 epochs using a cosine learning rate scheduler. The peak learning rate was 2×10^{-4} with a 3% warm up ratio. Weight decay was set to 0.0. The setup used a per-device batch size of 4 and 4 gradient accumulation steps. Gradient checkpointing and half-precision (`fp16`) execution controlled memory overhead. Data collation applied prompt token masking. The system replaced input labels for user instructions and prompt context with an ignore index (-100). This forced the autoregressive loss to calculate exclusively on the generated caption.

For deployment, the trained LoRA weights were applied to the merged RS-LLaVA base model using the PEFT library. The tokenizer used left-side padding to process the data efficiently in batches of 16. Greedy decoding was employed with a the token limit set at 256.

4.3. Tasks and Implementation Details

This section outlines the technical details and parameters for the different tasks performed throughout the experiments.

4.4. Object Probing Formats

Different object probing prompts were tested for hallucination detection depending on the approach and decoding strategy.

4.4.1. Binary questions

The prompt used for binary object probing questions is as follows:

> "Is there a *{object}* in the image?
> A) Yes
> B) No
> Letter:"

When using binary object probing, but with a free-form generation decoding strategy, the prompt was extended with an instruction to answer in a single word:

> "Is there a *{object}* in the image? Please answer this question with one word."

4.4.2. Multiple Choice Questions

The multiple-choice object probing prompt is as follows:

"What object is visible in the provided image?
A) {object_1}
B) {object_2}
…
D) {object_n}
E) None of the above
Letter:"

To correct for positional bias, the question is run for each permutation of the answer choices. The final confidence score is calculated as the average of all permutations, capped at 1000 permutations per questions. In case the number of permutations exceeds this limit, a random subset is selected.

4.4.3. Decoding Strategies

4.4.3.1. Regular Decoding

Regular decoding samples tokens autoregressively. The generation function limits the output to a maximum of 16 new tokens and executes with `use_cache` enabled. The configuration activates sampling by setting `do_sample=True` and locks the temperature at 1.0. The `top_p` parameter is left at the default value of 1.0.

4.4.3.2. Greedy Decoding

Greedy decoding eliminates probability sampling by explicitly setting `do_sample=False`, equivalent to setting the temperature to 0. The system selects the most probable token at each step. Generation is capped at 16 new tokens and runs with `use_cache` enabled.

4.4.3.3. VCD

The implementation configures the penalty scalar α to 1.0. It also enforces adaptive constraints using a β parameter of 0.1. The model samples tokens autoregressively (`do_sample=True`) with both the temperature and `top_p` fixed at 1.0. Output length is restricted to 16 new tokens with `use_cache` active.

4.4.3.4. APC

The Adaptive Plausibility Constraint (APC) restricts the generation vocabulary by masking improbable tokens. The processor locates the highest token probability and multiplies it by a scaling factor β, configured to 0.1. This mathematical operation produces a dynamic threshold. The system evaluates the distribution and assigns a score of negative infinity to any token falling below this threshold. The model generates up to 16 new tokens with `use_cache` enabled. The configuration explicitly activates sampling (`do_sample=True`) with both temperature and `top_p` forced to 1.0.

4.4.3.5. Binary Decoding

Binary decoding bypasses standard text generation to evaluate specific next-token logits. The input prompt outlines a binary choice and terminates by requesting a single letter. The system queries the final input position and extracts the logits corresponding exclusively to the "A" and "B" tokens. A sigmoid function converts the difference between the "A" logit and the "B" logit into a normalized confidence score bounded between 0.0 and 1.0. Equivalent tokens for "A" and "B" are also considered as some models tend to add a prefix such as "_A".

4.4.3.6. Binary VCD

Binary VCD subjects the extracted binary logits to the contrastive penalty. The system first tests the original token probabilities against the APC threshold utilizing a β parameter of 0.1. Any token failing to meet this threshold is eliminated and scored as negative infinity. For the surviving tokens, the system calculates the penalized logit. The calculations run with the α parameter set to 1.0.

4.4.4. Semantic Entropy Estimation

Estimating uncertainty using Semantic Entropy requires sampling multiple candidate responses for each question and grouping the answers into semantic clusters.

4.4.4.1. Answer Sampling

The model generates 11 candidate answers for each question. The generation process relies on two distinct sampling strategies. One answer is generated with sampling enabled, a temperature of 0.1, and a limit of 50 new tokens. The remaining 10 answers are generated with sampling enabled, a temperature of 1.0, a top-p of 0.9, a top-k of 50, and a limit of 50 new tokens.

4.4.4.2. Correctness Check

A VLM is used to check if the low-temperature answer is equivalent to the reference answer. A vision-language model receives the image and a query. This query runs with `do_sample=False`, a temperature of 0.1, `use_cache=True`, and `max_new_tokens=5`. The prompt template is:

> We are assessing the quality of answers to the following visual question: '*{question}*' based on the provided image.
> The expected answer is: *{reference}*
> The proposed answer is: *{predicted}*
> Based on the visual evidence in the image and the context of the question, does the proposed answer mean the same as the expected answer?
> Respond ONLY with 'yes' or 'no'.

4.4.4.3. Visually Grounded Bidirectional Entailment

A VLM is used to check for semantic bidirectional entailment of all 11 generated answer pairs. To save compute, an exact string match runs between candidate strings before invoking the model. If the exact match fails, a VLM evaluates entailment in both the forward and backward directions. This evaluation uses `do_sample=False`, a fixed temperature of 0.1, `use_cache=True` and `max_new_tokens=10`. The prompt template is:

> We are evaluating answers to the visual question: '{question}' based on the provided image.
> Here are two possible answers:
> Possible Answer 1: *{text1}*
> Possible Answer 2: *{text2}*
> Based on the visual evidence in the image and the context of the question, does Possible Answer 1 semantically entail Possible Answer 2?
> Respond ONLY with 'entailment', 'contradiction', or 'neutral'.

Two answers enter the same cluster if and only if bidirectional entailment holds. Both forward and backward evaluations must return "entailment". The system then computes the discrete Semantic Entropy from the resulting empirical cluster probability distribution. The formula is:

$$\text{Semantic Entropy} = -\sum P(C)\ln(P(C)) \tag{4.1}$$

with $P(C)$ being the empirical probability of a specific cluster. The computation determines this probability by dividing the number of answers inside cluster C by the total number of sampled generations.

5. Results

This chapter presents the results of all performed experiments regarding Remote Sensing (RS) object hallucination detection and revision.

5.1. Baseline performance of hallucination detection using object probing questions

All five Vision-Language Models (VLMs) detect hallucinations in the RS-POPE dataset well above random chance as seen in Table 5.1. Qwen3.5 9B achieves the highest Area Under the Curve (AUC) at 0.918, and GeoChat 7B yields the lowest at 0.8 as shown in Figure 5.1. The optimal decision threshold varies substantially across models. This variation reflects differing overall confidence levels and aligns with the diverse "Yes Ratios" recorded during baseline testing.

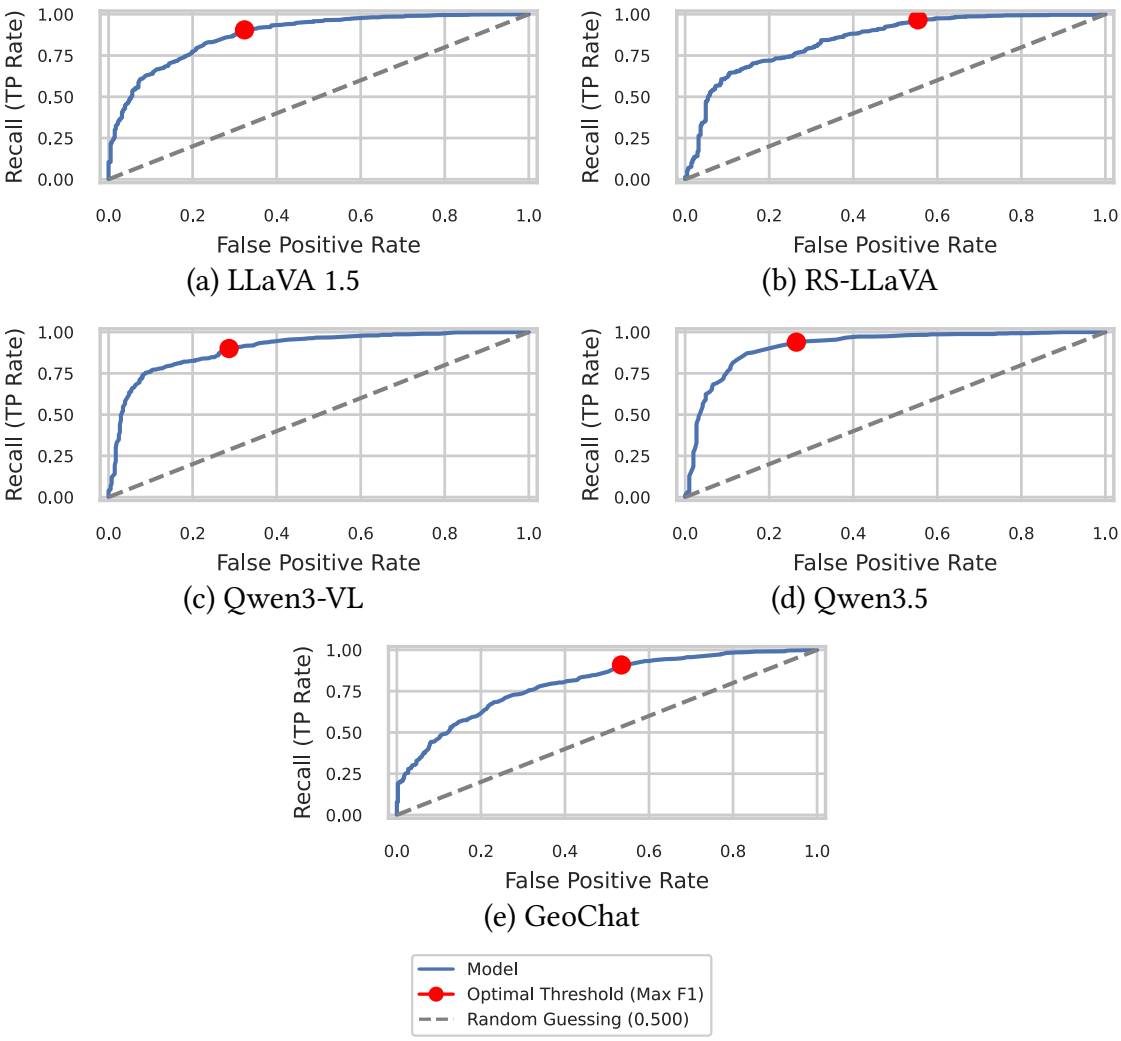


Figure 5.1: Receiver Operating Characteristic (ROC) curves showing hallucination detection performance using binary questions on RS-POPE.

As seen in Figure 5.2, the models exhibit distinct confidence distributions. All models achieve class separation. LLaVA 1.5 and GeoChat spread confidence more evenly. RS-LLaVA, Qwen3-VL, and Qwen3.5 behave more decisively, clustering predictions near the extremes (0.0 and 1.0). Across all models, peak confidence for visible objects exceeds the 0.5 threshold, while peak confidence for non-visible objects consistently falls below it.

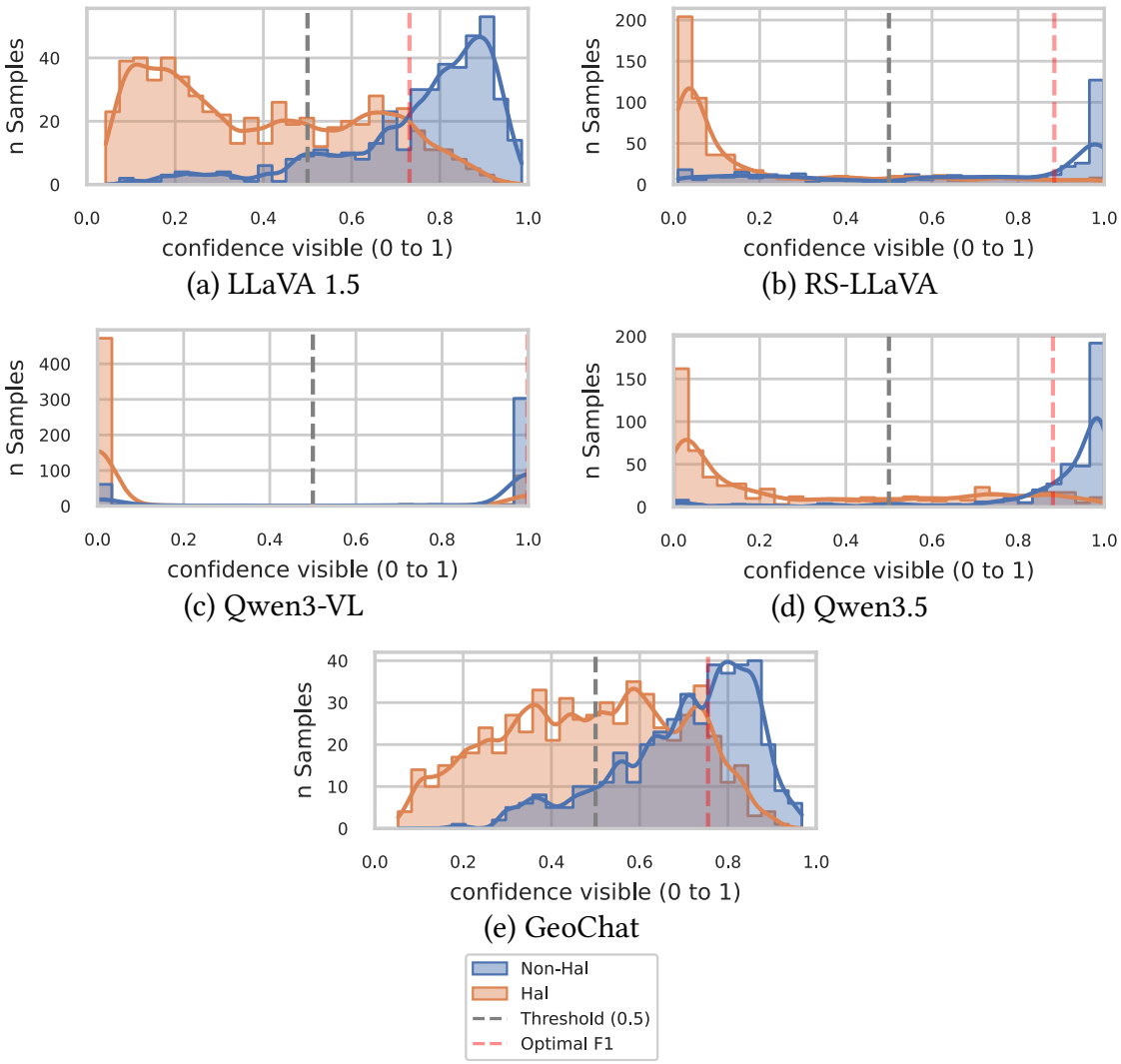


Figure 5.2: Confidence Distributions for Hallucination Detection Performance using Binary Questions on RS-POPE.

Performance decreases universally on the Labelled Illusion Dataset (LID) dataset compared to RS-POPE. All models still perform above a random classifier as seen in Figure 5.3. Qwen3-VL achieves the highest Area Under the ROC Curve (ROC-AUC) on LID at 0.868, and GeoChat records the lowest at 0.644. The optimal decision threshold again shifts significantly depending on the model.

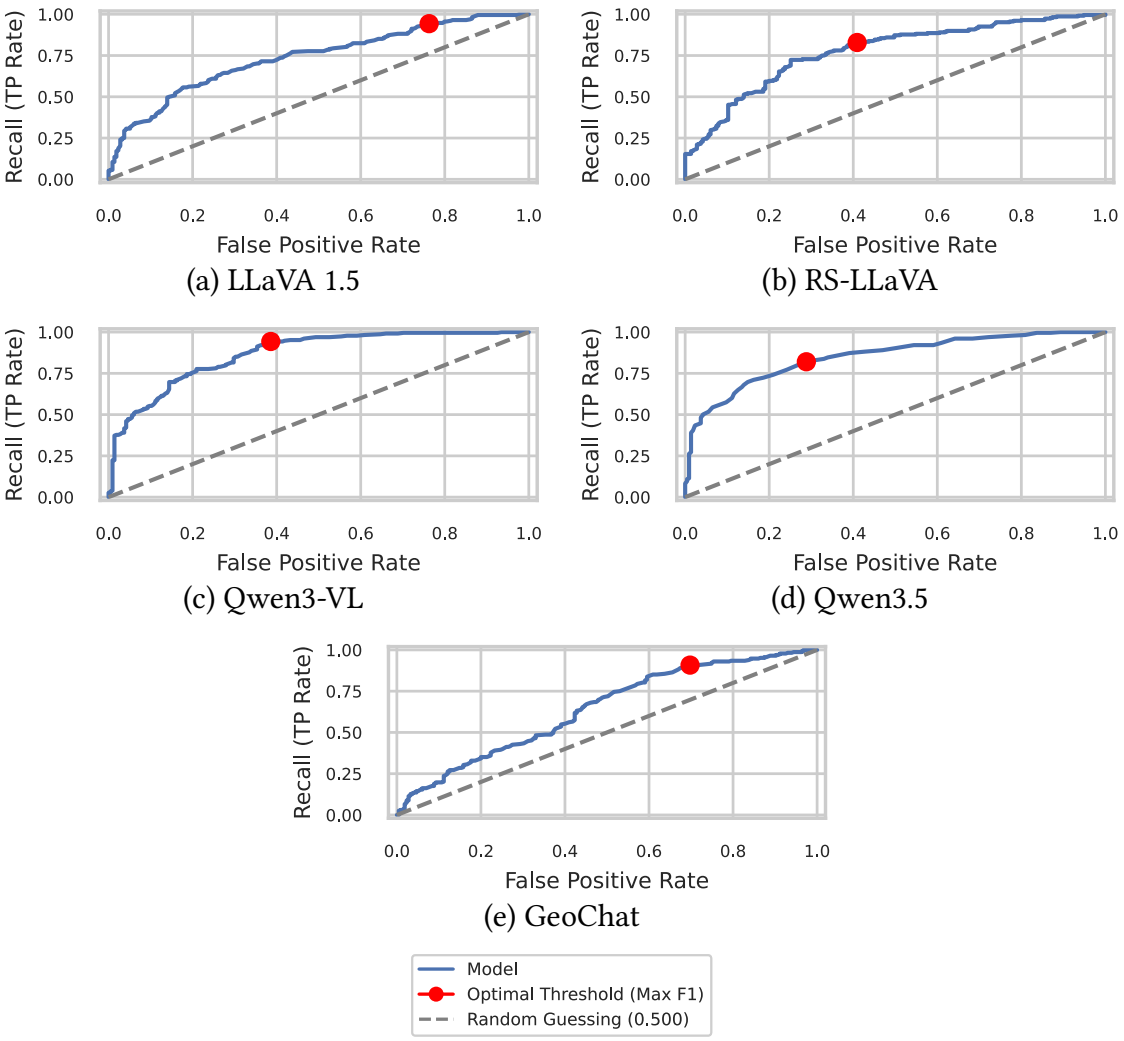


Figure 5.3: ROC curves showing hallucination detection performance using binary questions on LID.

Figure 5.4 indicates that confidence distributions for LID show less class separation than those in RS-POPE. LLaVA 1.5 and GeoChat maintain a distributed spread, but all models exhibit substantial overlap between visible and non-visible object confidences. The peak confidence for non-visible objects in LID consistently sits above 0.5, closely mirroring the peak for visible objects. This overlap reduces hallucination recall rates across all models (Table 5.1). The models detect the manually labeled hallucinations in LID less effectively than the synthetically sampled hallucinations in RS-POPE.

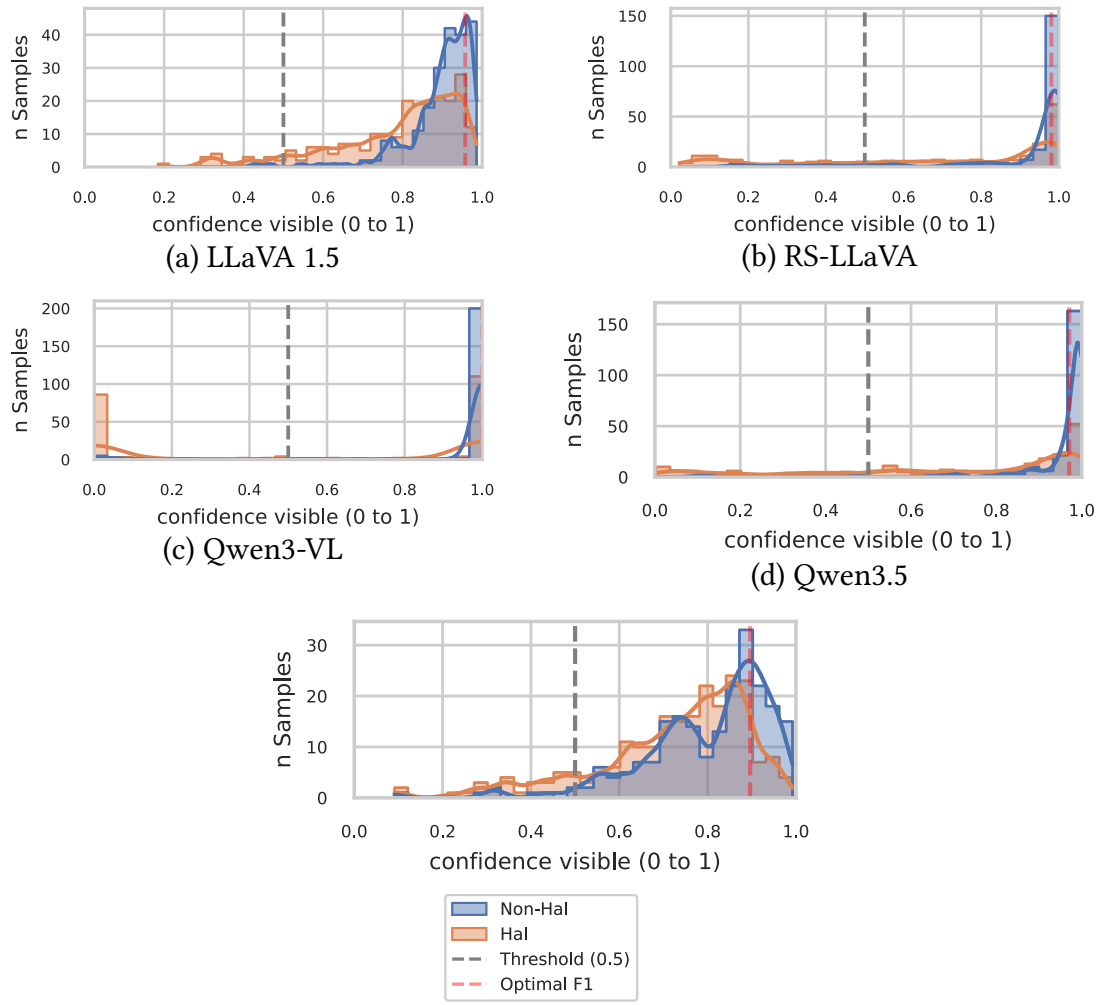


Figure 5.4: Confidence Distributions for Hallucination Detection Performance using Binary Questions on LID.

Table 5.1: Baseline performance of hallucination detection using binary decoding with decision threshold fixed at 0.5 confidence score. Best performance values are in bold.

Dataset	Model	Size	Precision	Recall	F_1-Score	Accuracy	Yes Ratio	ROC-AUC	AP
RS-POPE	LLaVA 1.5	7B	0.898	0.654	0.757	0.751	0.569	0.8794	0.9081
	RS-LLaVA	7B	0.789	0.816	0.802	0.762	0.388	0.8482	0.8690
	Qwen3-VL	8B	0.858	**0.826**	**0.842**	**0.816**	0.430	0.9017	0.9193
	Qwen3.5	9B	**0.926**	0.699	0.797	0.789	0.553	**0.9177**	**0.9201**
	GeoChat	7B	0.856	0.503	0.634	0.656	0.652	0.7998	0.8532
LID_{bal}	LLaVA 1.5	7B	0.900	0.079	0.145	0.521	0.955	0.7374	0.7597
	RS-LLaVA	7B	0.824	0.307	0.447	0.609	0.808	0.7747	0.7872
	Qwen3-VL	8B	0.921	**0.461**	**0.614**	**0.702**	0.743	**0.8680**	0.8594
	Qwen3.5	9B	**0.955**	0.281	0.434	0.623	0.849	0.8497	**0.8621**
	GeoChat	7B	0.806	0.127	0.220	0.535	0.919	0.6437	0.6371

5.1.1. Binary questions vs multiple choice questions for hallucination detection

Table 5.2 summarizes the efficacy of two object probing strategies: binary questions and Multiple-Choice Questions (MCQs). Binary questions generally yield superior hallucination detection. The impact, however, varies by model. LLaVA 1.5 saw marginal gains in F1-Score (+0.005) and Accuracy (+0.021) with MCQs at the default decision threshold of 0.5. Yet ROC-AUC and Average Precision (AP) metrics declined for all models using MCQs, with some exceptions: RS-LLaVA and GeoChat's performance remained stable for RS-POPE. Qwen3-VL and Qwen3.5 suffered severe drops across every recorded metric.

Table 5.2: Performance of hallucination detection using multiple choice questions compared with binary questions baseline. Performance improvements over baseline in green and declines in red.

Dataset	Model	Size	Precision	Recall	F_1-Score	Accuracy	Yes Ratio	ROC-AUC	AP
RS-POPE	LLaVA 1.5	7B	0.856	0.686	0.762 (+0.005)	0.772 (+0.021)	0.574	0.8570 (−0.0224)	0.8477 (−0.0604)
	Qwen3-VL	8B	0.865	0.531	0.658 (−0.184)	0.705 (−0.111)	0.672	0.8723 (−0.0294)	0.8321 (−0.0872)
	Qwen3.5	9B	0.560	0.036	0.068 (−0.729)	0.468 (−0.321)	0.966	0.6187 (−0.2990)	0.6211 (−0.2990)
	RS-LLaVA	7B	0.782	0.843	0.811 (+0.009)	0.790 (+0.028)	0.423	0.8565 (+0.0083)	0.8392 (−0.0298)
	GeoChat	7B	0.668	0.975	0.793 (+0.159)	0.731 (+0.075)	0.229	0.8752 (+0.0754)	0.8523 (−0.0009)
LID	LLaVA 1.5	7B	0.514	0.158	0.242 (−0.003)	0.651 (+0.130)	0.892	0.5482 (−0.1892)	0.4283 (−0.3314)
	Qwen3-VL	8B	0.765	0.057	0.106 (−0.508)	0.663 (−0.039)	0.974	0.7960 (−0.0720)	0.6775 (−0.1819)
	Qwen3.5	9B	0.343	0.513	0.411 (−0.023)	0.481 (−0.142)	0.471	0.4877 (−0.3620)	0.3777 (−0.4844)
	RS-LLaVA	7B	0.750	0.118	0.205 (−0.242)	0.674 (+0.065)	0.944	0.6072 (−0.1675)	0.4964 (−0.2908)
	GeoChat	7B	0.415	0.298	0.347 (+0.127)	0.603 (+0.068)	0.746	0.5269 (−0.1168)	0.4082 (−0.2289)

Figure 5.5 shows the confidence score distributions and ROC curves for LLaVA 1.5 under both strategies. The overall shape of the distributions remains similar. The MCQ strategy produced more highly confident predictions visible objects. This shift flattens the ROC

curve for the MCQ format compared to the binary baseline. This matches the recorded drop in ROC-AUC.

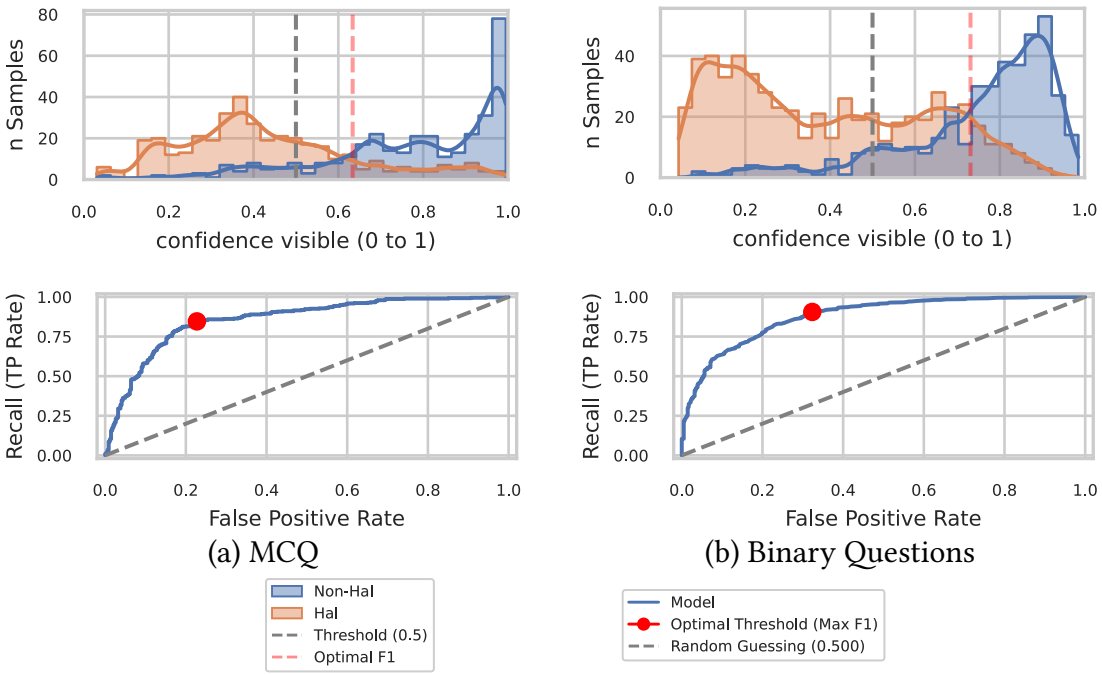


Figure 5.5: Hallucination detection performance using binary questions vs. multiple choice questions on RS-POPE with LLaVA 1.5.

MCQs disrupted the confidence distribution of Qwen3-VL much more, as Figure 5.6 details. Under the binary baseline, Qwen3-VL maintained a sharp peak near a confidence score of 0.0 for non-visible objects. With MCQs, the confidence distribution for non-visible objects flattened dramatically across the 0.0 to 1.0 range. This flattening eliminates the earlier class separation. It directly reflects the model's severe drop in precision and recall.

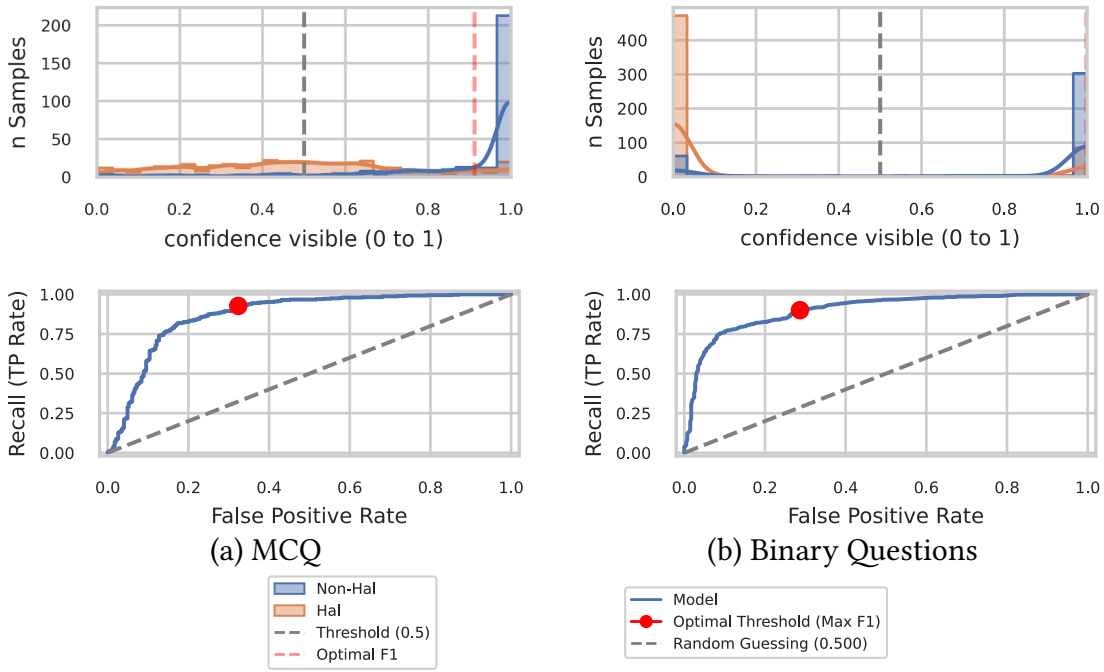


Figure 5.6: Hallucination detection performance using binary questions vs. multiple choice questions on RS-POPE with Qwen3-VL.

5.2. Visual contrastive decoding

Table 5.3 shows the effect of Visual Contrastive Decoding (VCD) on hallucination detection using the Binary Decoding strategy. Applying VCD with the original parameters from Leng et al. degrades overall detection performance on the RS-POPE dataset. The impact varies across metrics. Binary VCD improves Precision for all models except GeoChat. These gains range from +0.0208 for Qwen3-VL to +0.0586 for RS-LLaVA. Severe drops in Recall offset these improvements. Qwen3.5 sees a Recall drop of −0.2787. This drives a decrease in its F1-Score (-0.2124) and Accuracy (-0.1430). LLaVA 1.5 suffers a steep F1-Score reduction (-0.1589). The ROC-AUC metric declines for most models. RS-LLaVA's ROC-AUC remains essentially unchanged (-0.0001). VCD inflates the "Yes Ratio" for all models except GeoChat. This skew is most pronounced in Qwen3.5 and LLaVA 1.5. Their Yes Ratios surge by +18.7 and +15.3. VCD strongly biases model outputs toward positive responses. On the LID dataset, the pattern are less clear. Notably, the Yes Ratio either stayed or got even higher with Qwen3-VL showing the lowest Yes-Ratio at 78% and LLaVA 1.5 at 98%.

Table 5.3: Performance of hallucination detection using Binary Decoding and Binary VCD across different confidence thresholds on RSPOPE and LID datasets. Increases over baseline in green and declines in red.

Model	Precision	Recall	F_1-Score	Accuracy	Yes Ratio	ROC-AUC
RSPOPE						
LLaVA 1.5	0.9353 (+0.0373)	0.4392 (−0.2145)	0.5977 (−0.1589)	0.6500 (−0.1010)	72.2000 (+15.3000)	0.8722 (−0.0071)
rs-LLaVA	0.8478 (+0.0586)	0.6774 (−0.1385)	0.7531 (−0.0493)	0.7370 (−0.0250)	52.7000 (+13.9000)	0.8481 (−0.0001)
Qwen3-VL	0.8787 (+0.0208)	0.8074 (−0.0186)	0.8415 (−0.0001)	0.8200 (+0.0040)	45.6000 (+2.6000)	0.8325 (−0.0692)
Qwen3.5	0.9577 (+0.0315)	0.4206 (−0.2787)	0.5845 (−0.2124)	0.6460 (−0.1430)	74.0000 (+18.7000)	0.9159 (−0.0018)
GeoChat	0.8078 (−0.0485)	0.5963 (+0.0929)	0.6861 (+0.0521)	0.6770 (+0.0210)	56.3000 (−8.9000)	0.7793 (−0.0205)
LID						
LLaVA 1.5	1.0000 (+0.1000)	0.0351 (−0.0439)	0.0678 (−0.0774)	0.5034 (−0.0181)	98.1941 (+2.7088)	0.7225 (−0.0149)
rs-LLaVA	0.9348 (+0.1113)	0.1886 (−0.1184)	0.3139 (−0.1334)	0.5756 (−0.0339)	89.6163 (+8.8036)	0.7184 (−0.0563)
Qwen3-VL	0.9175 (−0.0035)	0.3904 (−0.0702)	0.5477 (−0.0663)	0.6682 (−0.0339)	78.1038 (+3.8375)	0.7220 (−0.1460)
Qwen3.5	0.9524 (−0.0028)	0.0877 (−0.1930)	0.1606 (−0.2733)	0.5282 (−0.0948)	95.2596 (+10.3837)	0.7745 (−0.0752)
GeoChat	0.7727 (−0.0328)	0.1491 (+0.0219)	0.2500 (+0.0303)	0.5395 (+0.0045)	90.0677 (−1.8059)	0.6824 (+0.0387)

5.2.1. Effect of distortion type and intensity on hallucination detection using visual contrastive decoding

Figure 5.7 shows the impact of distortion types and intensities on the F1-Score for hallucination detection using VCD. For LLaVA 1.5, RS-LLaVA, and Qwen3.5, the F1-Score stays below the binary decoding baseline. It drops as distortion intensity increases. Qwen3-VL shows the opposite. It starts below the baseline at low intensities. Its F1-Score then rises with distortion, recovering to nearly match the baseline at maximum intensity. At a relative intensity of 1.0, the image is very distorted and in the case of all masking strategies the image is completely black. The performance of all masking strategies converges to a

single point for each model, except for GeoChat for which the masking strategies show a slight dip in comparison with the rest.

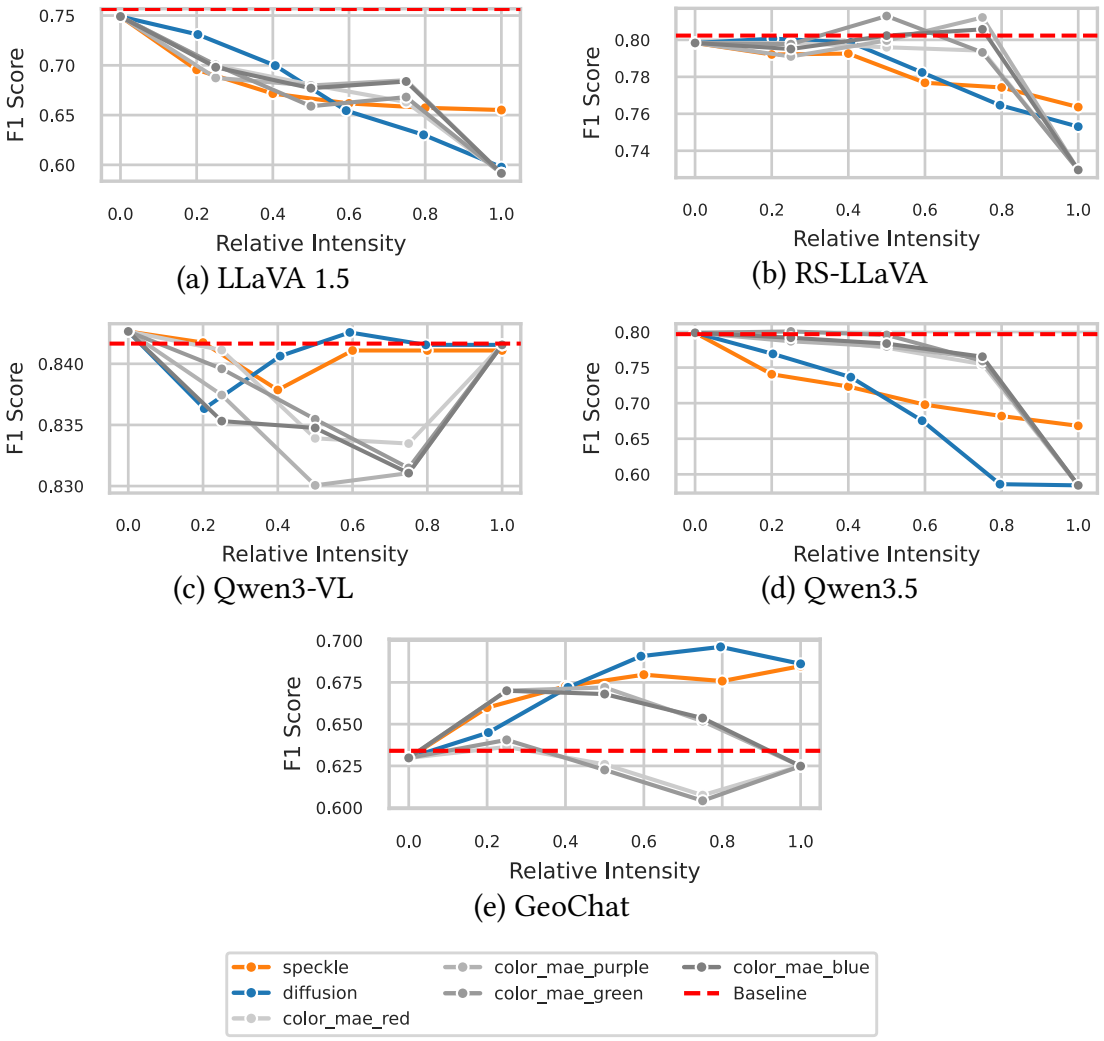


Figure 5.7: Impact of distortion type and intensity on hallucination detection performance using VCD on binary questions on RS-POPE.

Similarly, perfromance does not seem to be significantly improve on LID as seen in Figure 5.8. For LLaVA 1.5, RS-LLaVA, and Qwen3.5, performance initially rises above the baseline for certain distortion types at lower intensities before worsening as distortion increases. GeoChat, however, sees a significant increase across all intensities but overall performance is still poor.

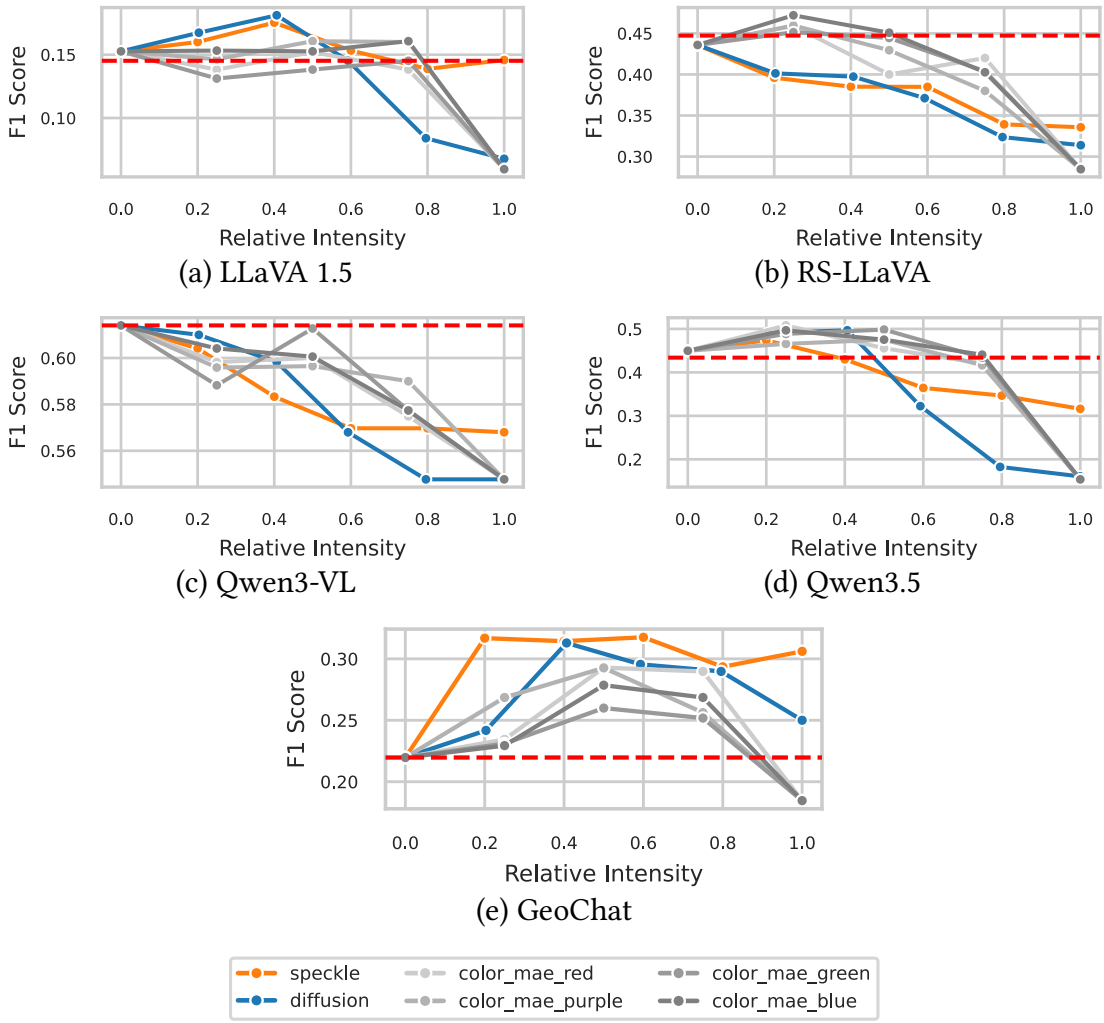


Figure 5.8: Impact of distortion type and intensity on hallucination detection performance using VCD on binary questions on LID.

Table 5.4 details performance shifts when applying VCD with a completely black image. This equals a masking ratio of 1.0. This extreme distortion increases Precision for all models except GeoChat. It simultaneously drives Recall down. The reductions in Recall outweigh the marginal gains in Precision. Qwen3.5 sees a notable −0.2787 drop in Recall. The overall F1-Score decreases for all models. Class-specific prediction behavior shifts significantly. Yet the ROC-AUC shows negligible change from the baseline.

Table 5.4: VCD performance using a black image (masking ratio 1.0) and Gaussian noise (max intensity) as the distorted input, alongside the difference to the baseline targeting the hallucination class. The dataset used is RS-POPE. Increased values over baseline in green and declines in red.

Model	Precision	Recall	F_1-Score	Accuracy	Yes Ratio	ROC-AUC
Masking Ratio 1.0 (Black Image)						
LLaVA 1.5	0.9278 (+0.0299)	0.4341 (−0.2196)	0.5915 (−0.1651)	0.6450 (−0.1060)	72.3000 (+15.4000)	0.8570 (−0.0224)
rs-LLaVA	0.8865 (+0.0973)	0.6199 (−0.1959)	0.7296 (−0.0727)	0.7280 (−0.0340)	58.6000 (+19.8000)	0.8417 (−0.0065)
Qwen3-VL	0.8787 (+0.0208)	0.8074 (−0.0186)	0.8415 (−0.0001)	0.8200 (+0.0040)	45.6000 (+2.6000)	0.8324 (−0.0693)
Qwen3.5	0.9577 (+0.0315)	0.4206 (−0.2787)	0.5845 (−0.2124)	0.6460 (−0.1430)	74.0000 (+18.7000)	0.9107 (−0.0070)
GeoChat	0.8424 (−0.0139)	0.4966 (−0.0068)	0.6249 (−0.0092)	0.6470 (−0.0090)	65.1000 (−0.1000)	0.7624 (−0.0374)
Gaussian Noise (Max Intensity)						
LLaVA 1.5	0.9353 (+0.0373)	0.4392 (−0.2145)	0.5977 (−0.1589)	0.6500 (−0.1010)	72.2000 (+15.3000)	0.8722 (−0.0071)
rs-LLaVA	0.8478 (+0.0586)	0.6774 (−0.1385)	0.7531 (−0.0493)	0.7370 (−0.0250)	52.7000 (+13.9000)	0.8481 (−0.0001)
Qwen3-VL	0.8787 (+0.0208)	0.8074 (−0.0186)	0.8415 (−0.0001)	0.8200 (+0.0040)	45.6000 (+2.6000)	0.8325 (−0.0692)
Qwen3.5	0.9577 (+0.0315)	0.4206 (−0.2787)	0.5845 (−0.2124)	0.6460 (−0.1430)	74.0000 (+18.7000)	0.9159 (−0.0018)
GeoChat	0.8078 (−0.0485)	0.5963 (+0.0929)	0.6861 (+0.0521)	0.6770 (+0.0210)	56.3000 (−8.9000)	0.7793 (−0.0205)

5.2.1.1. MCQ

Figure 5.9 shows the impact of combining VCD and MCQs across various distortion types and relative intensities on the RS-POPE dataset. Increasing distortion, and thus magnifying the effect of VCD, does not improve performance. Using a highly distorted contrastive input either caused no noticeable difference or except fro Qwen3.5 which saw a performance increase. A similar pattern is seen across all tested distortion types.

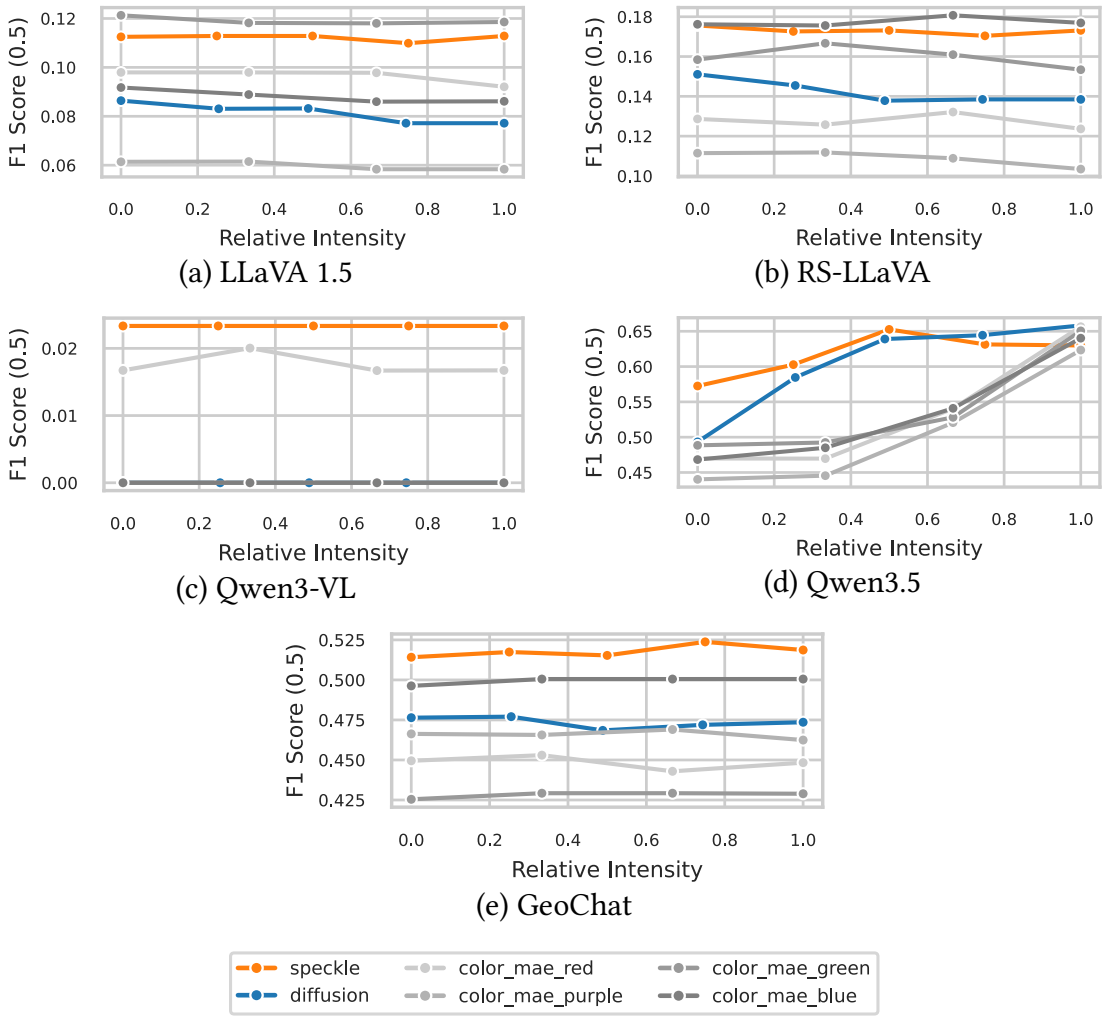
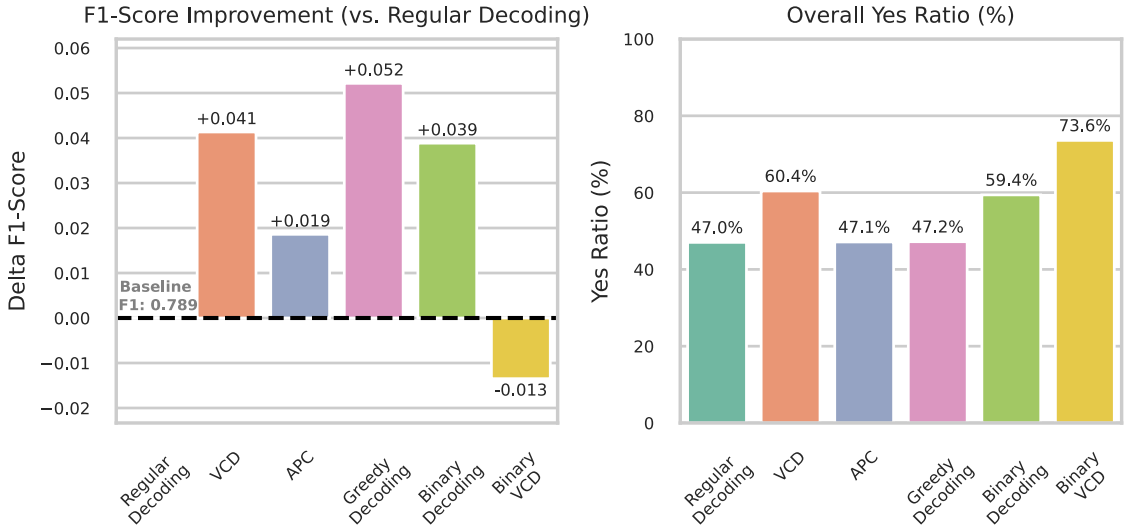


Figure 5.9: Effect of distortion type and intensity on hallucination detection performance using multiple choice questions and VCD on RSPOPE.

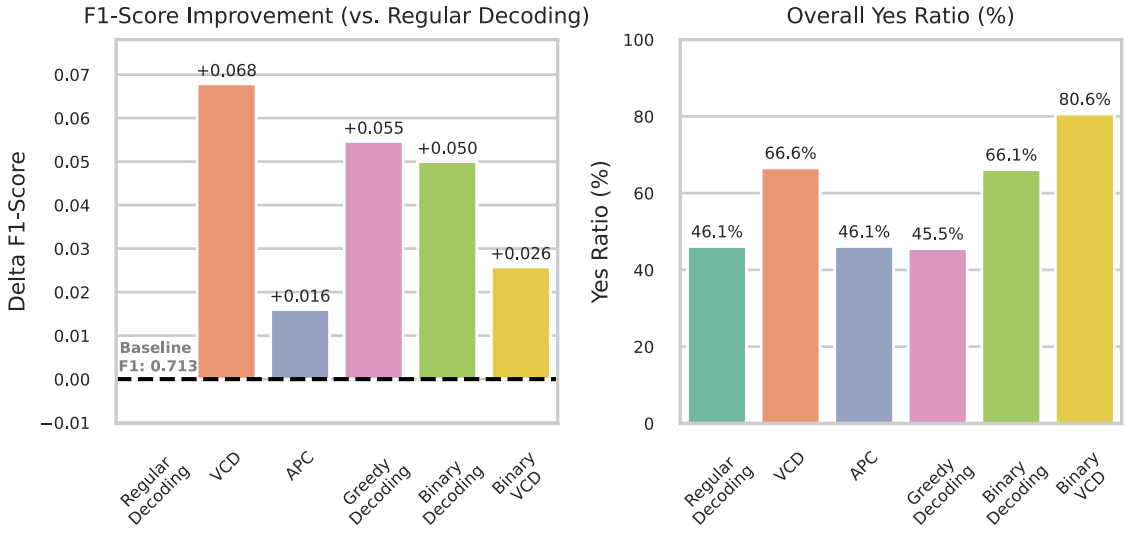
5.2.2. Isolating the influence of contrastive decoding

As seen in Figure 5.10, applying VCD significantly increases the F1-score and the Yes Ratio across the Polling-based Object Probing Evaluation (POPE) and RS-POPE datasets using LLaVA 1.5. The performance gain is larger for RS-POPE than POPE (+0.068 vs. +0.041), matching a steeper rise in the Yes Ratio (+20.5% vs. +0.041). Isolated from

contrastive decoding, the Adaptive Plausibility Constraint (APC) improves performance on both datasets without changing the baseline Yes Ratio. Greedy Decoding also increases performance and achieves the highest F1-score gain on POPE (+0.052) while keeping the Yes Ratio stable. Binary Decoding mirrors VCD, increasing both the F1-score and the Yes Ratio. Binary VCD skews the Yes Ratio further, reaching 73.6% on POPE and 80.6% on RS-POPE. This inflated ratio yields some gains on RS-POPE (+0.026) but drops the POPE F1-score below baseline (-0.013).



(a) POPE



(b) RS-POPE

Figure 5.10: Hallucination detection performance and Yes Ratios of different decoding strategies on POPE and RS-POPE.

5.2.3. Using uncertainty estimation to find hallucinations in VLM-generated captions in an existing Remote Sensing dataset

The 1211 object with the lowest confidence scores were manually evaluated. These objects had a confidence score ranging from 0.008 to 0.013. Out of these, 92.65% were object hallucinations. As some objects were extracted from the same image, this corresponds to 1006 images that were manually evaluated. 91.55% of all manually evaluated images contained hallucinations.

Table 5.5: Manual evaluation results for the lowest-confidence objects in Git-10M.

	Total Evaluated	Confidence Range	Hallucinations	Hallucination Rate
Objects	1,211	0.008 – 0.013	1,122	92.65%
Images	1,006	–	921	91.55%

5.3. Semantic Entropy for uncertainty estimation in RS

Semantic Entropy was tested on a Visual Question Answering (VQA) dataset which provided questions and answers. Additionally, Semantic Entropy was also tested on an image captioning dataset where extracted objects were probed using newly generated questions.

5.3.1. VQA

Figure 5.11 displays the distribution of Semantic Entropy scores, while Figure 5.12 presents the corresponding rejection accuracy curves when using Semantic Entropy for uncertainty estimation in VQA evaluated on the DDFAV [12] dataset. Across most models, correct predictions cluster densely at the lowest Semantic Entropy scores near 0.0, indicating high certainty. For Qwen3.5, incorrect predictions spread broadly across higher entropy values, creating the clearest separation between accurate answers and hallucinated ones. LLaVA 1.5 and RS-LLaVA show similar but less distinct spreads, with a notable portion of incorrect answers also receiving low entropy scores. Conversely, GeoChat fails to produce a meaningful uncertainty signal, collapsing nearly all predictions into the zero entropy bin regardless of correctness. The median Semantic Entropy scores vary by model, sitting at approximately 0.80 for LLaVA 1.5, 0.50 for RS-LLaVA, and 1.1 for Qwen3.5. As shown in the rejection accuracy curves, filtering out samples with the highest Semantic Entropy steadily increases the accuracy of the retained samples for all models except GeoChat, which exhibits a flat curve matching its baseline accuracy of roughly 0.30. Applying Qwen3.5′s optimal F1 rejection threshold filters out roughly 85% of the samples, which elevates the accuracy of the remaining samples to approximately 0.70. Similarly, LLaVA 1.5 and RS-LLaVA demonstrate consistent improvements, lifting their baseline accuracies of roughly 0.33 and 0.40 to retained accuracies of over 0.50 and 0.60, respectively, at higher rejection rates.

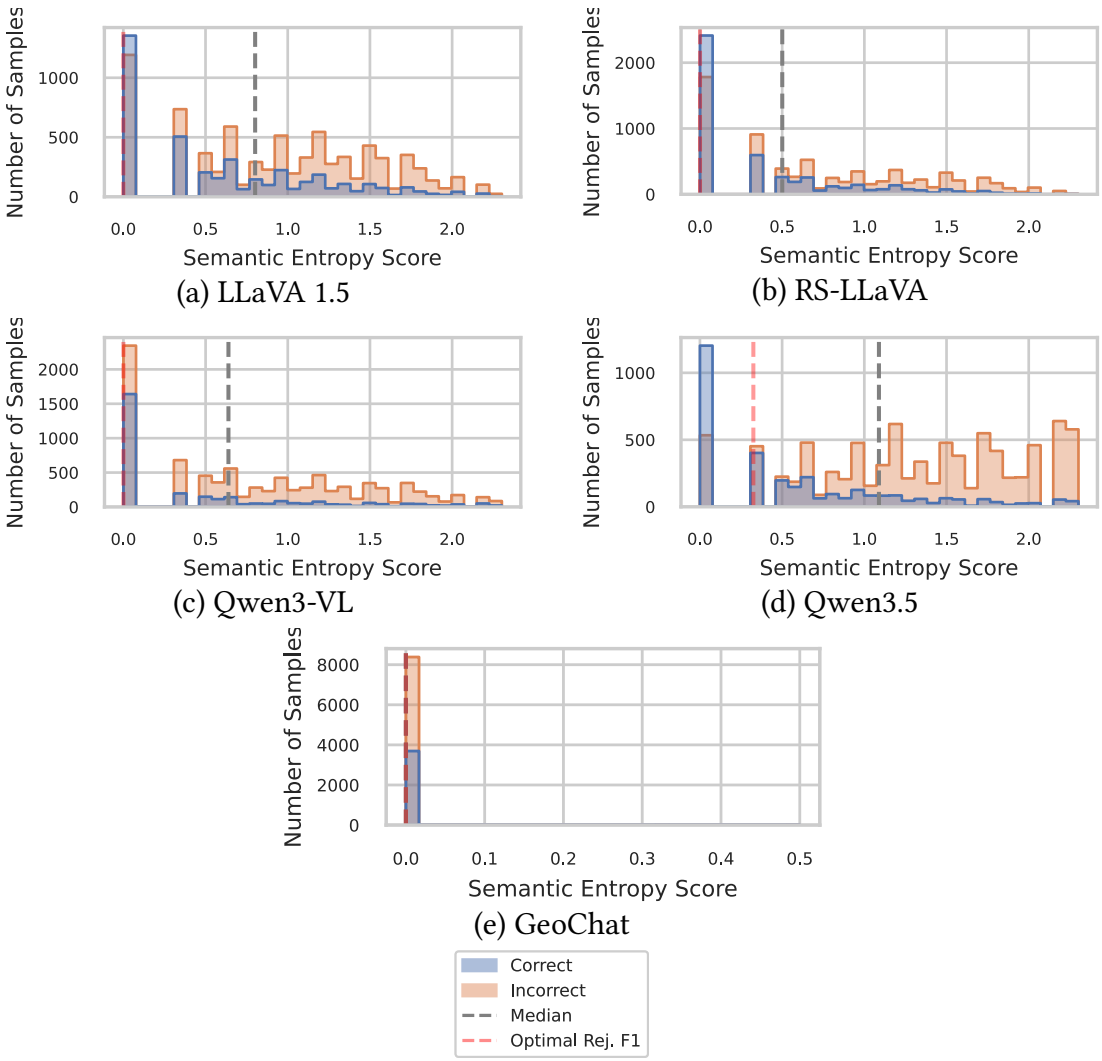


Figure 5.11: Confidence distributions for Hallucination Detection Performance using Semantic Entropy on VQA with the DDFAV dataset.

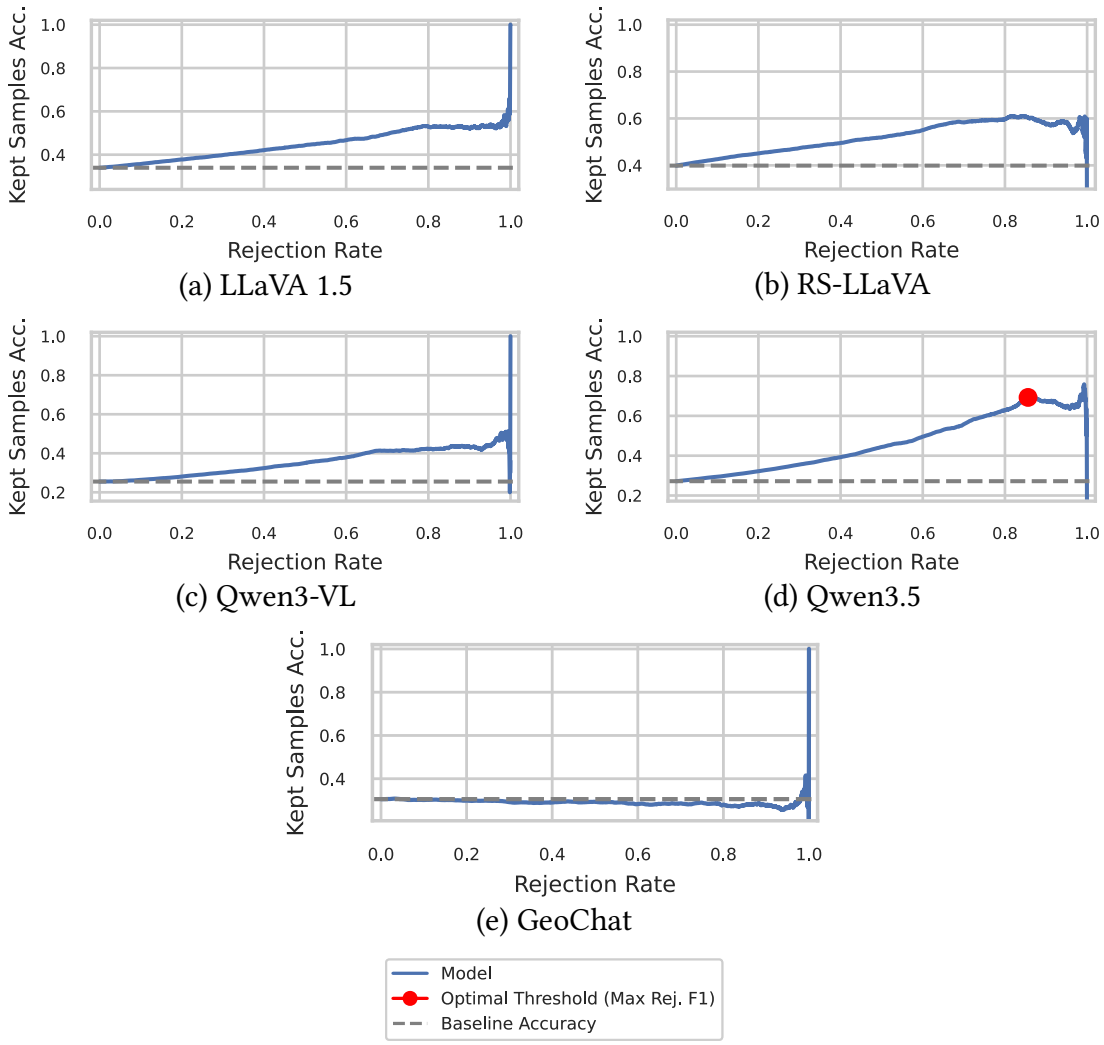


Figure 5.12: ROC-Curves for Hallucination Detection Performance using Semantic Entropy on VQA with the DDFAV dataset.

5.3.2. Captions

Figure 5.13 displays the distribution of Semantic Entropy scores, while Figure 5.14 presents the corresponding rejection accuracy curves when using Semantic Entropy for uncertainty estimation in image captioning with the Git-10M dataset. For most models, correct predictions are heavily concentrated at the lowest Semantic Entropy score of 0.0, indicating high certainty. Qwen3.5 exhibits the most dispersed distribution, with incorrect predictions spreading broadly across higher entropy values to create a clearer separation from accurate answers. Its median Semantic Entropy score sits at approximately 0.8, with an optimal F1 rejection threshold near 0.7. In contrast, LLaVA 1.5, RS-LLaVA, and Qwen3-VL show massive clustering at 0.0 for both correct and incorrect predictions, with optimal rejection thresholds much lower, near 0.3. GeoChat once again fails to produce a meaningful uncertainty signal, collapsing all predictions near zero. As shown in the rejection

accuracy curves, filtering out samples with the highest Semantic Entropy improves the accuracy of the retained samples for most models. Qwen3.5 shows the most substantial and steady gain, starting from a baseline accuracy of approximately 0.63 and reaching roughly 0.82 when applying its optimal F1 rejection threshold, which filters out roughly 55% of the samples. Qwen3-VL and LLaVA 1.5 also demonstrate noticeable improvements; Qwen3-VL climbs from a 0.75 baseline to approximately 0.88 at a 45% rejection rate, while LLaVA 1.5 rises from 0.71 to roughly 0.78 at a 30% rejection rate. RS-LLaVA starts with a very high baseline accuracy near 0.94 and shows only marginal improvement, reaching roughly 0.95 with a minimal rejection rate of 10%. Finally, GeoChat's curve remains flat baseline.

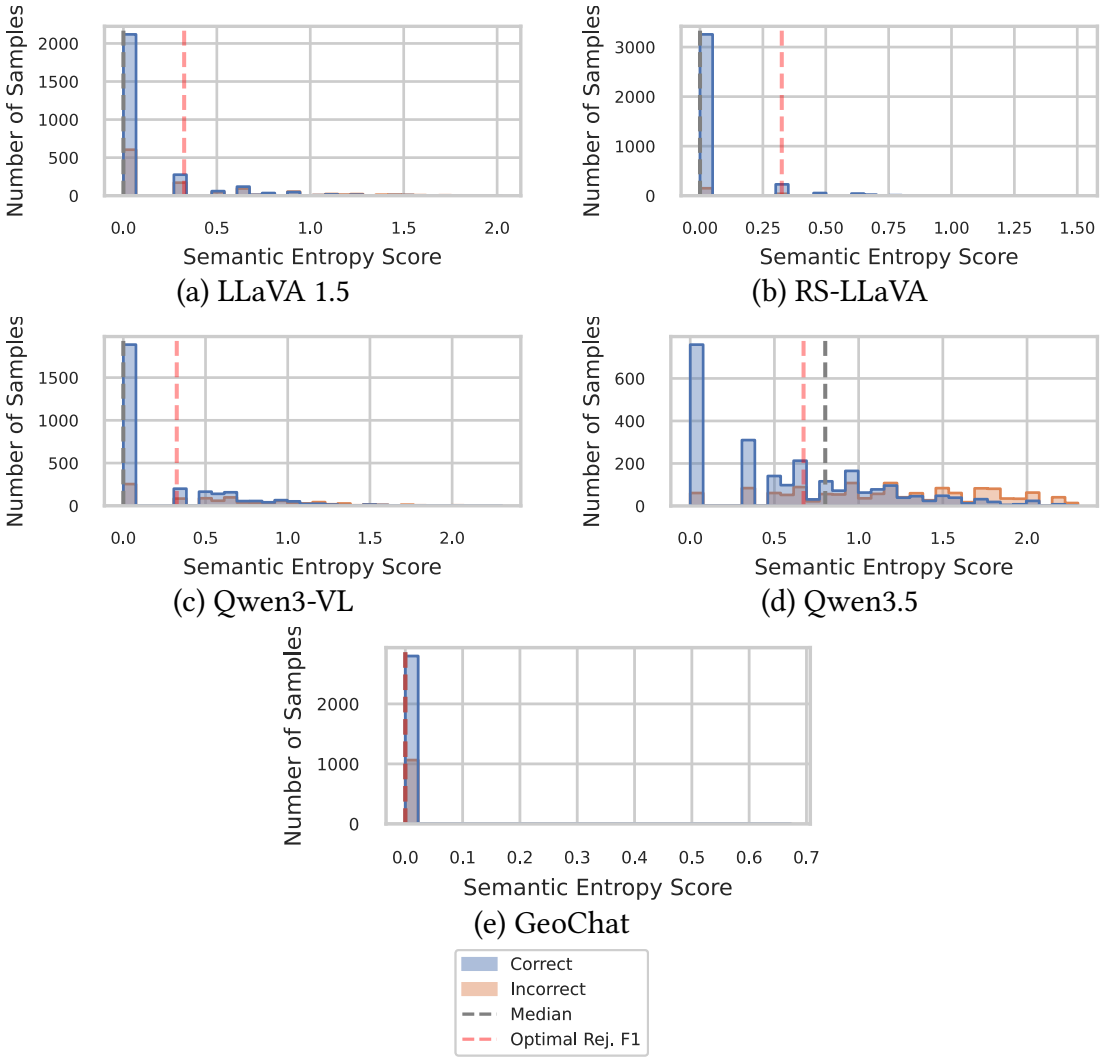


Figure 5.13: Confidence distributions for Hallucination Detection Performance using Semantic Entropy on image captions with the Git-10M dataset.

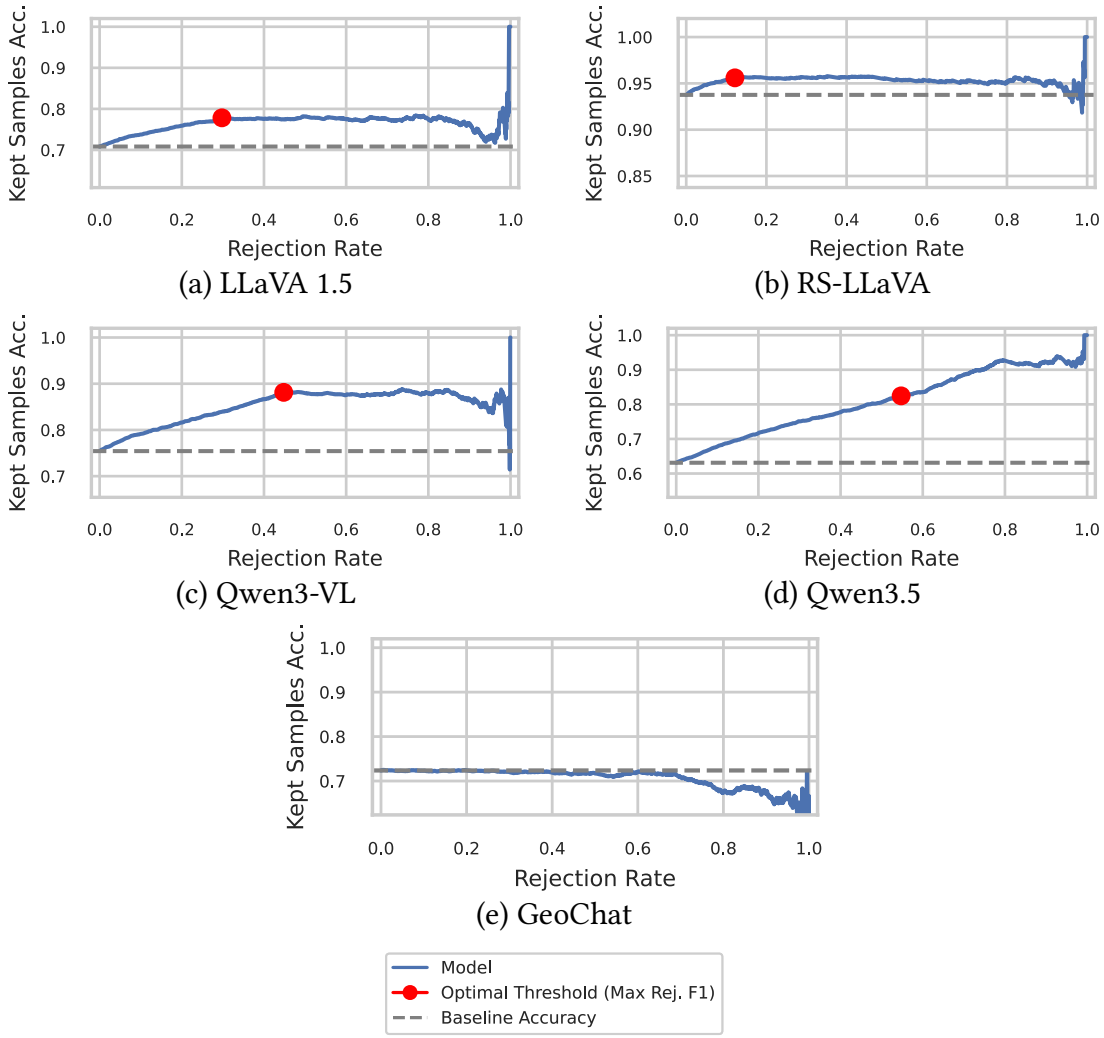


Figure 5.14: ROC-Curves for Hallucination Detection Performance using Semantic Entropy on images with the Git-10M dataset.

5.4. Using a reviser to remove hallucinations from existing Remote Sensing captions

Table 5.6 and Table 5.7 demonstrate the effect of thereviser fine-tuned on a subset of Git-10M. As shown in Table 5.6, the reviser generally improves hallucination metrics. On Git-10M, it reduces the Semantic-CHAIR$_i$ score by 0.0449 and the Semantic-CHAIR$_s$ score by 0.0420. On GAIA, the reviser decreases the Semantic-CHAIR$_i$ score by 0.0389 and the Semantic-CHAIR$_s$ score by 0.0587, and on RSTeller, it lowers the Semantic-CHAIR$_i$ score by 0.0458 and the Semantic-CHAIR$_s$ score by 0.0147. Performance is slightly mixed on DDFAV, where the reviser increases the Semantic-CHAIR$_i$ score by 0.0101 but decreases the Semantic-CHAIR$_s$ score by 0.0034. Table 5.7 shows that these reductions in hallucination do not heavily compromise general caption quality, which is largely preserved or slightly improved. On Git-10M and RSTeller, the reviser

yields minor gains across most metrics, including increases in BLEU-4 (+0.0012 and +0.0004, respectively) and CIDEr (+0.0138 and +0.0021). On DDFAV and GAIA, while the reviser causes marginal drops in BLEU-4 (-0.0052 and −0.0030), it continues to slightly improve CIDEr scores (+0.0017 and +0.0062).

Table 5.6: Effect of the reviser on Semantic-CHAIR scores on RS captions. Performance increases over baseline in green and declines in red.

Dataset	Model	Semantic-CHAIR$_i$	Semantic-CHAIR$_s$
Git-10M	Original	0.4970	0.9327
	Reviser	0.4521 (−0.0449)	0.8907 (−0.0420)
DDFAV	Original	0.8772	0.9926
	Reviser	0.8873 (+0.0101)	0.9892 (−0.0034)
GAIA	Original	0.3848	0.6520
	Reviser	0.3459 (−0.0389)	0.5933 (−0.0587)
RSTeller	Original	0.7416	0.8833
	Reviser	0.6958 (−0.0458)	0.8687 (−0.0147)

Table 5.7: Effect of the reviser on Remote Sensing captions on general caption quality metrics. Performance increases over baseline in green and declines in red.

Dataset	Model	BLEU-1	BLEU-2	BLEU-3	BLEU-4	CIDEr
Git-10M	Original	0.2932	0.1288	0.0615	0.0336	0.0821
	Reviser	0.2984	0.1338	0.0641	0.0348 (+0.0012)	0.0958 (+0.0138)
DDFAV	Original	0.2324	0.1014	0.0480	0.0257	0.0301
	Reviser	0.2060	0.0853	0.0388	0.0205 (−0.0052)	0.0318 (+0.0017)
GAIA	Original	0.3649	0.1465	0.0576	0.0285	0.0161
	Reviser	0.3603	0.1382	0.0517	0.0256 (−0.0030)	0.0223 (+0.0062)
RSTeller	Original	0.1920	0.0469	0.0167	0.0095	0.0123
	Reviser	0.1961	0.0474	0.0172	0.0099 (+0.0004)	0.0144 (+0.0021)

6. Discussion

6.1. Viability of VLMs as object-probing hallucination detectors

The baseline evaluation results outlined in Section 5.1 indicate that VLMs can detect hallucinations based on RS images using binary probing questions. All tested Vision-Language Models (VLMs) separated visible and non-visible object confidences. However, all models performed significantly worse on the real-world, model-generated hallucinations (LID) than on the synthetic hallucinations (RS-POPE). The overlap in confidence distributions on the Labelled Illusion Dataset (LID) dataset suggests that real hallucinations are harder to detect. Hallucinations have multi-factorial and context-dependent causes. Thus, the simple negative samples as present in RS-POPE might not serve as perfect analogs for Remote Sensing (RS) hallucinations.

Additionally, the experiment did not find clear evidence that fine-tuning on RS data significantly improved performance in this task. While RS-LLaVA showed stronger performance at the neutral decision threshold of 0.5, LLaVA 1.5 resulted in better Area Under the ROC Curve (ROC-AUC) and Average Precision scores than both RS-LLaVA and GeoChat on RS-POPE (both models are based on LLaVA 1.5). More important than RS fine-tuning might be a more modern architecture or model size, as both Qwen3-VL and Qwen3.5 performed better than the remaining models. One limitation of this experiment is that matching sizes of all models do not exist. Thus, it is not clear exactly why the slightly larger models were the best performers.

6.1.1. Performance effect of different object-probing formats

The comparison of different object probing question shown in Section 5.1.1, show binary questions are better suited for visual object confidence than multiple choice questions. Overall hallucination detection performance degraded significantly after introducing Multiple-Choice Questions (MCQs) across all VLMs. This shows a significant difference from the results presented by Ren et al. [41] but is not surprising as the experiment performed in this work poses a big domain shift. Ren et al. [41] explored both types of prompts as a method for unimodal language model self-evaluation. Specifically, they used this method to select the best answer out of n samples. This goal changed when bringing this method over to the visual RS domain. The goal here was not to select a single least (or most) likely object, instead it is to select all objects below a certain confidence threshold. While the comparison of different output inherent in MCQs might be helpful

when selecting a single output, this comparison does not seem as useful for determining object visibility.

6.1.2. Effect of Visual Contrastive Decoding on hallucination detection

The results presented in Section 5.2 show that Visual Contrastive Decoding (VCD) with Gaussian noise distortion negatively impacts hallucination detection across the board. While ROC-AUC remains relatively stable, VCD causes significant drops in F1-score across both datasets and prompting strategies. Interestingly, precision rises as recall falls, and these changes are consistent with an increasing Yes-Ratio. This points to VCD simply biasing the model to answering that a model is visible in the image. These results are not consistent with the results by Leng et al. [13]. Consequently, this raises the question of whether VCD is specifically ineffective for RS imagery, or if the method is inherently unsuitable for this detection task.

To explore this idea further, different distortion techniques were evaluated. The results in Section 5.2.1 do not show that any distortion technique is universally better than Gaussian noise for VCD. Surprisingly, for instances where the F1-score did rise above baseline, like Qwen3-VL, performance peaked at the maximum distortion level. Masking 100% of the image (resulting in a completely black image) resulted in peak performance for Qwen3-VL. This finding contradicts the premise that VCD improves performance by triggering uncertainty-driven hallucinations via noisy inputs [13]. Probing for an object in a completely black image is a trivial task, therefore the performance increase cannot stem from triggered model uncertainty.

This edge case is further explored and the results are in Table 6.4. These results follow the same pattern as with same pattern observed with Gaussian noise: recall falls as precision and the Yes-Ratio rise. These results suggest that VCD is not triggering hallucinations. Instead, when probed about an extremely distorted image, the model correctly predicts the absence of the object with high confidence. As VCD contrasts the original logits with the distorted logits, the “not visible” response is punished. This results in artificially inflated confidence that the object is visible, increasing the Yes-Ratio.

To compare results directly, the POPE benchmark performed by Leng et al. [13] was replicated using different decoding strategies. Additionally, the same set-up was used with RS-POPE, as seen in Section 5.2.2. These results align with the paper by Yin et al. [11], which suggests that the apparent performance gains of contrastive methods is primarily due to the shifting yes-ratio and effectively mimicking greedy decoding. Applying the Adaptive Plausibility Constraint (APC) alone improves performance without increasing yes ratio. While APC was originally designed to prevent VCD from penalizing high-confidence tokens essential for text fluency, it functionally punishes all tokens below a predefined threshold, rendering the decoding process greedier. Like APC, Greedy Decoding improved performance across both datasets without increasing Yes Ratio.

APC does not account for all apparent performance gains of VCD. Both VCD and the Binary Decoding method performed in this thesis yield similar performance gains

across both datasets, this is accompanied by a matching rise in Yes Ratio. This indicates that both methods artificially shift the yes-ratio when compared to baseline. The performance increase is simply coincidental and just happens to cause a performance increase in certain benchmarks with certain models. However, the mechanism driving this shift differs: VCD directly penalizes negative answers, whereas Binary Decoding likely introduces bias through fixed answer tokens or positional preferences (e.g., favoring option "A" over "B"). When combining both methods, the Yes-Ratio climbs even further. In any case, this provides further evidence that the increased performance is incidental as combining both methods drives the Yes-Ratio even higher, degrading performance on Polling-based Object Probing Evaluation (POPE) while increasing it on RS-POPE. In both instances, the optimal Yes-Ratio threshold is exceeded, leading to degraded overall detection reliability compared to utilizing either decoding strategy independently. Overall, these results suggest that VCD is not specifically unsuited for RS imagery, instead it seems to be a deeper mechanistic problem.

6.2. Viability of Semantic Entropy for Remote Sensing

The results presented in Section 5.3 indicate that Semantic Entropy is capable of detecting hallucinations caused by RS imagery. Semantic Entropy is a viable method for detecting hallucinations in RS Visual Question Answering (VQA). Hallucination detection performance was notably worse when applying Semantic Entropy on RS image captions. This might be because the pipeline needed for image captioning is significantly more complex, adding noise. Even so, all models achieved performance above a random classifier, except for GeoChat. While Semantic Entropy is a reasonable approach, the binary object probing method achieved better performance. Additionally, Semantic Entropy requires full generation of n answers and the use of language models for semantic clustering. In comparison, binary object probing only requires a single forward pass. For object hallucination detection, binary object probing seems to be overall superior. Semantic Entropy might be better suited for more complex visual questions that do not have a binary yes or no answer.

6.3. Revising Remote Sensing image captions

The trained hallucination reviser appears to marginally improve hallucinations rates in RS image captions without reducing overall caption quality as seen in Section 5.4. The improvement is less pronounced than in the paper by Zhou et al. [8]. This might be because none of datasets used in this work provide exhaustive reference object labels. Thus, it was not possible to calculate the CHAIR scores as defined by Rohrbach et al. [39]. Instead, the evaluation in this work used the Semantic-CHAIR as an approximation as described in Section 3.3. This also explains the inflated baseline hallucinations scores. This problem also accentuates the need for an RS image captioning datasets with exhaustive reference object labels.

6.4. Finding Hallucinations in Git-10M

The method proposed in Section 3.2 successfully found a subset of hallucinated captions in Git-10M. As only the lowest confidence objects were evaluated, the hallucinations ended up being more drastic than expected. Based on my qualitative impression, most of the manually evaluated captions were completely wrong instead of simply containing isolated object hallucinations. Thus, the manually labeled subset would not be a useful benchmark for hallucination detection, as most of the cases would be trivial. This method seems to be particularly well suited to prune off the most obvious cases of object hallucinations in large RS image captioning datasets. For smaller datasets containing less hallucinations, this method might not be precise enough.

7. Conclusion

The Remote Sensing (RS) field has an abundance of image data but only scarce captioning in comparison. Vision-Language Models (VLMs) might help to automate this expensive process. To increase trust in VLM-generated captions the problem of hallucinations needs to be better understood and addressed.

The first object of this thesis was to explore confidence estimation methods for object hallucination detection in Remote Sensing images. To this end, the efficacy of object-probing techniques, Visual Contrastive Decoding, and Semantic Entropy on RS imagery was evaluated. This work found that VLMs are indeed capable to detecting object hallucinations in RS images. This work also provides further evidence, in accordance with the work by Yin et al.[11], that Visual Contrastive Decoding (VCD) does not aid hallucination detection due to fundamental mechanistic flaws. The results further indicate that Semantic Entropy may be a suitable method for detecting hallucinations in RS visual question answering, while binary object probing proved better suited for object-level hallucination detection. Finally, the proposed confidence estimation pipeline can locate hallucinations in existing RS datasets: a manually evaluated subset of Git-10M exhibited a hallucination rate above 90%.

The second objective was to train and deploy a hallucination reviser. This was achieved, and the trained reviser slightly reduced hallucinations in RS captions while preserving overall text quality.

7.1. Limitations

This thesis is subject to several limitations. First, the Multiple-Choice Question (MCQ) method becomes computationally expensive for captions containing many objects, since fully eliminating positional bias requires evaluating a number of permutations that grows factorially with the number of options. Second, the Labelled Illusion Dataset (LID) subset used in this work contains only a single manually labeled object hallucination per image, in contrast to the POPE framework, which samples several negative questions per image. In addition, LID consists exclusively of drone imagery, which makes it difficult to draw conclusions about hallucinations in the broader RS field. Third, matching model sizes do not exist for all evaluated VLMs, so performance differences cannot be attributed to architecture or model size alone. Finally, all experiments were conducted as single runs due to limited resources, and the reviser was evaluated with Semantic-CHAIR, because none of the used datasets provide exhaustive reference object labels.

7.2. Future Work

The object probing pipeline presented in this work would benefit greatly by having extensive, categorized lists of relevant RS objects. Such lists would allow fine-grained evaluation of future hallucination detection methods and, ultimately, a deeper understanding of RS confabulations. Future work should also focus on more fine-grained RS hallucination detection. This work only explores object hallucinations, but confabulations in RS images are more complex as whole, and categorizing RS hallucinations by type and magnitude would be essential for understanding their causes. Additionally, the work presented here relies on VLMs for object extraction. Incorporating methods to improve this step could reduce noise and possibly allow for targeting specific types of object hallucinations depending on the use-case. Finally, multi-spectral images were not evaluated in this thesis even though they are an important part of many RS applications. It would be valuable to understand how current hallucination detection and revision techniques would need to be adapted for multi-spectral imagery.

Bibliography

[1] M. Anderson, M. Cha, W. T. Freeman, J. T. Perron, N. Maidel, and K. Cahoy, "Measuring and Mitigating Hallucinations in Vision-Language Dataset Generation for Remote Sensing." Accessed: Sept. 16, 2025. [Online]. Available: http://arxiv.org/abs/2501.14905

[2] A. Zavras, D. Michail, X. X. Zhu, B. Demir, and I. Papoutsis, "GAIA: A Global, Multi-modal, Multi-scale Vision-Language Dataset for Remote Sensing Image Analysis." Accessed: Mar. 02, 2026. [Online]. Available: http://arxiv.org/abs/2502.09598

[3] D. Basak, S. Bhatt, S. Kanduri, and M. S. Desarkar, "Aerial Mirage: Unmasking Hallucinations in Large Vision Language Models," in *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Feb. 2025, pp. 5500–5508. doi: 10.1109/WACV61041.2025.00537.

[4] C. Liu, K. Chen, R. Zhao, Z. Zou, and Z. Shi, "Text2Earth: Unlocking Text-Driven Remote Sensing Image Generation with a Global-Scale Dataset and a Foundation Model," *IEEE Geoscience and Remote Sensing Magazine*, vol. 13, no. 3, pp. 238–259, Sept. 2025, doi: 10.1109/MGRS.2025.3560455.

[5] J. Ge, X. Zhang, Y. Zheng, K. Guo, and J. Liang, "RSTeller: Scaling Up Visual Language Modeling in Remote Sensing with Rich Linguistic Semantics from Openly Available Data and Large Language Models," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 226, pp. 146–163, Aug. 2025, doi: 10.1016/j.isprsjprs.2025.05.002.

[6] X. Li, W. Diao, X. Li, and X. Sun, "Insight Any Instance: Promptable Instance Segmentation for Remote Sensing Images," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 63, pp. 1–15, 2025, doi: 10.1109/TGRS.2025.3543636.

[7] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting Hallucinations in Large Language Models Using Semantic Entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, June 2024, doi: 10.1038/s41586-024-07421-0.

[8] Y. Zhou *et al.*, "Analyzing and Mitigating Object Hallucination in Large Vision-Language Models," in *International Conference on Representation Learning*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., 2024, pp. 56969–56998. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/fc625e831361cfcc82cb74224fdc66cb-Paper-Conference.pdf

[9] X. Chen *et al.*, "Multi-Object Hallucination in Vision Language Models," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., Curran Associates, Inc., 2024, pp. 44393–44418. doi: 10.52202/079017-1409.

[10] Y. Li, Y. Du, K. Zhou, J. Wang, X. Zhao, and J.-R. Wen, "Evaluating Object Hallucination in Large Vision-Language Models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore: Association for Computational Linguistics, 2023, pp. 292–305. doi: 10.18653/v1/2023.emnlp-main.20.

[11] H. Yin, G. Si, and Z. Wang, "The Mirage of Performance Gains: Why Contrastive Decoding Fails to Mitigate Object Hallucinations in MLLMs?," presented at the The Thirty-ninth Annual Conference on Neural Information Processing Systems, Oct. 2025. Accessed: June 12, 2026. [Online]. Available: https://openreview.net/forum?id=wGeQr5cARm&referrer=%5Bthe%20profile%20of%20Guangzong%20Si%5D(%2Fprofile%3Fid%3D~Guangzong_Si1)

[12] H. Li, X. Zhang, and H. Qu, "DDFAV: Remote Sensing Large Vision Language Models Dataset and Evaluation Benchmark," *Remote Sensing*, vol. 17, no. 4, p. 719, Jan. 2025, doi: 10.3390/rs17040719.

[13] Sicong Leng *et al.*, "Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA: IEEE, June 2024, pp. 13872–13882. doi: 10.1109/CVPR52733.2024.01316.

[14] A. Dosovitskiy *et al.*, "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale," presented at the International Conference on Learning Representations, Oct. 2020. Accessed: June 09, 2026. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[15] A. F. Biten, L. Gómez, and D. Karatzas, "Let There Be a Clock on the Beach: Reducing Object Hallucination in Image Captioning," presented at the Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 2022, pp. 1381–1390. Accessed: Feb. 15, 2026. [Online]. Available: https://openaccess.thecvf.com/content/WACV2022/html/Biten_Let_There_Be_a_Clock_on_the_Beach_Reducing_Object_WACV_2022_paper.html

[16] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning." Accessed: Dec. 11, 2025. [Online]. Available: http://arxiv.org/abs/2304.08485

[17] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: 10.1038/nature14539.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778. Accessed: Sept. 03, 2026. [Online]. Available: https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

[19] H. Liu *et al.*, "A Survey on Hallucination in Large Vision-Language Models." Accessed: Oct. 20, 2025. [Online]. Available: http://arxiv.org/abs/2402.00253

[20] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the 38th International Conference on Machine Learning*, PMLR, July 2021, pp. 8748–8763. Accessed: June 09, 2026. [Online]. Available: https://proceedings.mlr.press/v139/radford21a.html

[21] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, Paris, France: IEEE, Oct. 2023, pp. 11941–11952. doi: 10.1109/ICCV51070.2023.01100.

[22] M. Tschannen *et al.*, "SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features." Accessed: June 09, 2026. [Online]. Available: http://arxiv.org/abs/2502.14786

[23] J.-B. Alayrac *et al.*, "Flamingo: A Visual Language Model for Few-Shot Learning," presented at the Advances in Neural Information Processing Systems, Oct. 2022. Accessed: June 09, 2026. [Online]. Available: https://openreview.net/forum?id=EbMuimAbPbs

[24] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," in *Proceedings of the 40th International Conference on Machine Learning*, PMLR, July 2023, pp. 19730–19742. Accessed: June 09, 2026. [Online]. Available: https://proceedings.mlr.press/v202/li23q.html

[25] H. Liu, C. Li, Y. Li, and Y. J. Lee, "Improved Baselines with Visual Instruction Tuning," presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 26296–26306. Accessed: Aug. 27, 2026. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2024/html/Liu_Improved_Baselines_with_Visual_Instruction_Tuning_CVPR_2024_paper.html

[26] S. Bai *et al.*, "Qwen3-VL Technical Report." Accessed: Jan. 01, 2026. [Online]. Available: http://arxiv.org/abs/2511.21631

[27] F. Liu *et al.*, "RemoteCLIP: A Vision Language Foundation Model for Remote Sensing," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–16, 2024, doi: 10.1109/TGRS.2024.3390838.

[28] Y. Bazi, L. Bashmal, M. M. Al Rahhal, R. Ricci, and F. Melgani, "RS-LLaVA: A Large Vision-Language Model for Joint Captioning and Question Answering in Remote Sensing Imagery," *Remote Sensing*, vol. 16, no. 9, p. 1477, Jan. 2024, doi: 10.3390/rs16091477.

[29] H. Lin *et al.*, "RS-MoE: A Vision–Language Model With Mixture of Experts for Remote Sensing Image Captioning and Visual Question Answering," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 63, pp. 1–18, 2025, doi: 10.1109/TGRS.2025.3547988.

[30] K. Filippova, "Controlled Hallucinations: Learning to Generate Faithfully from Noisy Data," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds., Online: Association for Computational Linguistics, Nov. 2020, pp. 864–870. doi: 10.18653/v1/2020.findings-emnlp.76.

[31] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Comput. Surv.*, vol. 55, no. 12, p. 248:1–248:38, Mar. 2023, doi: 10.1145/3571730.

[32] R. Lebret, D. Grangier, and M. Auli, "Neural Text Generation from Structured Data with Application to the Biography Domain," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, J. Su, K. Duh, and X. Carreras, Eds., Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 1203–1213. doi: 10.18653/v1/D16-1128.

[33] F. Tang *et al.*, "Seeing Far and Clearly: Mitigating Hallucinations in MLLMs with Attention Causal Decoding," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 26147–26159. doi: 10.1109/CVPR52734.2025.02435.

[34] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, P. Isabelle, E. Charniak, and D. Lin, Eds., Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, July 2002, pp. 311–318. doi: 10.3115/1073083.1073135.

[35] S. Banerjee and A. Lavie, "METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments," in *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, J. Goldstein, A. Lavie, C.-Y. Lin, and C. Voss, Eds., Ann Arbor, Michigan: Association for Computational Linguistics, June 2005, pp. 65–72. Accessed: May 05, 2026. [Online]. Available: https://aclanthology.org/W05-0909/

[36] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out*, Barcelona, Spain: Association for Computational Linguistics, July 2004, pp. 74–81. Accessed: May 08, 2026. [Online]. Available: https://aclanthology.org/W04-1013/

[37] R. Vedantam, C. Lawrence Zitnick, and D. Parikh, "CIDEr: Consensus-Based Image Description Evaluation," presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 4566–4575. Accessed: May 05, 2026. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2015/html/Vedantam_CIDEr_Consensus-Based_Image_2015_CVPR_paper.html

[38] P. Anderson, B. Fernando, M. Johnson, and S. Gould, "SPICE: Semantic Propositional Image Caption Evaluation," in *Computer Vision – ECCV 2016*, vol. 9909, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds., Cham: Springer International Publishing, 2016, pp. 382–398. doi: 10.1007/978-3-319-46454-1_24.

[39] A. Rohrbach, L. A. Hendricks, K. Burns, T. Darrell, and K. Saenko, "Object Hallucination in Image Captioning," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds., Brussels, Belgium: Association for Computational Linguistics, Oct. 2018, pp. 4035–4045. doi: 10.18653/v1/D18-1437.

[40] Z. Zhang, T. Zhao, Y. Guo, and J. Yin, "RS5M and GeoRSCLIP: A Large Scale Vision-Language Dataset and A Large Vision-Language Model for Remote Sensing," *IEEE*

Transactions on Geoscience and Remote Sensing, vol. 62, pp. 1–23, 2024, doi: 10.1109/TGRS.2024.3449154.

[41] J. Ren, Y. Zhao, T. Vu, P. J. Liu, and B. Lakshminarayanan, "Self-Evaluation Improves Selective Generation in Large Language Models," in *Proceedings On*, PMLR, Apr. 2023, pp. 49–64. Accessed: Mar. 27, 2026. [Online]. Available: https://proceedings.mlr.press/v239/ren23a.html

[42] Y. Wu *et al.*, "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation." Accessed: Aug. 27, 2026. [Online]. Available: http://arxiv.org/abs/1609.08144

[43] D. Schwenk, A. Khandelwal, C. Clark, K. Marino, and R. Mottaghi, "A-OKVQA: A Benchmark for Visual Question Answering Using World Knowledge," in *Computer Vision – ECCV 2022*, vol. 13668, S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds., Cham: Springer Nature Switzerland, 2022, pp. 146–162. doi: 10.1007/978-3-031-20074-8_9.

[44] T.-Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context," in *Computer Vision – ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds., Cham: Springer International Publishing, 2014, pp. 740–755. doi: 10.1007/978-3-319-10602-1_48.

[45] D. A. Hudson and C. D. Manning, "GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019, pp. 6693–6702. doi: 10.1109/CVPR.2019.00686.

[46] K. Li, G. Wan, G. Cheng, L. Meng, and J. Han, "Object Detection in Optical Remote Sensing Images: A Survey and a New Benchmark," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 159, pp. 296–307, Jan. 2020, doi: 10.1016/j.isprsjprs.2019.11.023.

[47] G.-S. Xia *et al.*, "DOTA: A Large-Scale Dataset for Object Detection in Aerial Images," presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 3974–3983. Accessed: Aug. 27, 2026. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Xia_DOTA_A_Large-Scale_CVPR_2018_paper.html

[48] X. Sun *et al.*, "FAIR1M: A Benchmark Dataset for Fine-Grained Object Recognition in High-Resolution Remote Sensing Imagery," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 184, pp. 116–130, Feb. 2022, doi: 10.1016/j.isprsjprs.2021.12.004.

[49] J. Wang, W. Yang, H. Guo, R. Zhang, and G.-S. Xia, "Tiny Object Detection in Aerial Images," in *2020 25th International Conference on Pattern Recognition (ICPR)*, Jan. 2021, pp. 3791–3798. doi: 10.1109/ICPR48806.2021.9413340.

[50] D. Du *et al.*, "VisDrone-DET2019: The Vision Meets Drone Object Detection in Image Challenge Results," in *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, Oct. 2019, pp. 213–226. doi: 10.1109/ICCVW.2019.00030.

[51] P. Zhu *et al.*, "Detection and Tracking Meet Drones Challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, Nov. 2022, doi: 10.1109/TPAMI.2021.3119563.

[52] K. Kuckreja, M. S. Danish, M. Naseer, A. Das, S. Khan, and F. S. Khan, "GeoChat: Grounded Large Vision-Language Model for Remote Sensing," presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 27831–27840. Accessed: Jan. 14, 2026. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2024/html/Kuckreja_GeoChat_Grounded_Large_Vision-Language_Model_for_Remote_Sensing_CVPR_2024_paper.html

[53] Q. Team, "Qwen3.5-Omni Technical Report." Accessed: June 09, 2026. [Online]. Available: http://arxiv.org/abs/2604.15804

Generative Tools Declaration

Besides the generative models discussed throughout this work, generative models were used for language editing, text refinement, systematic research and translation. The services used were:

- Gemini 3.1 Pro by Google. Last Accessed: September 06, 2026. [Online]. Available: https://gemini.google.com
- DeepSeek V4 Pro by DeepSeek. Last Accessed: September 06, 2026. [Online]. Available: https://www.deepseek.com/en/

Appendix

A.1. Multiple Choice Questions for hallucination detection: All Models

The plots in this section show the hallucination detection performance using Multiple-Choice Questions (MCQs). Figure A.1 and Figure A.2 show the confidence distributions and ROC curves for all tested models on RS-POPE. Figure A.3 and Figure A.4 show the confidence distributions and ROC curves for all tested models on LID.

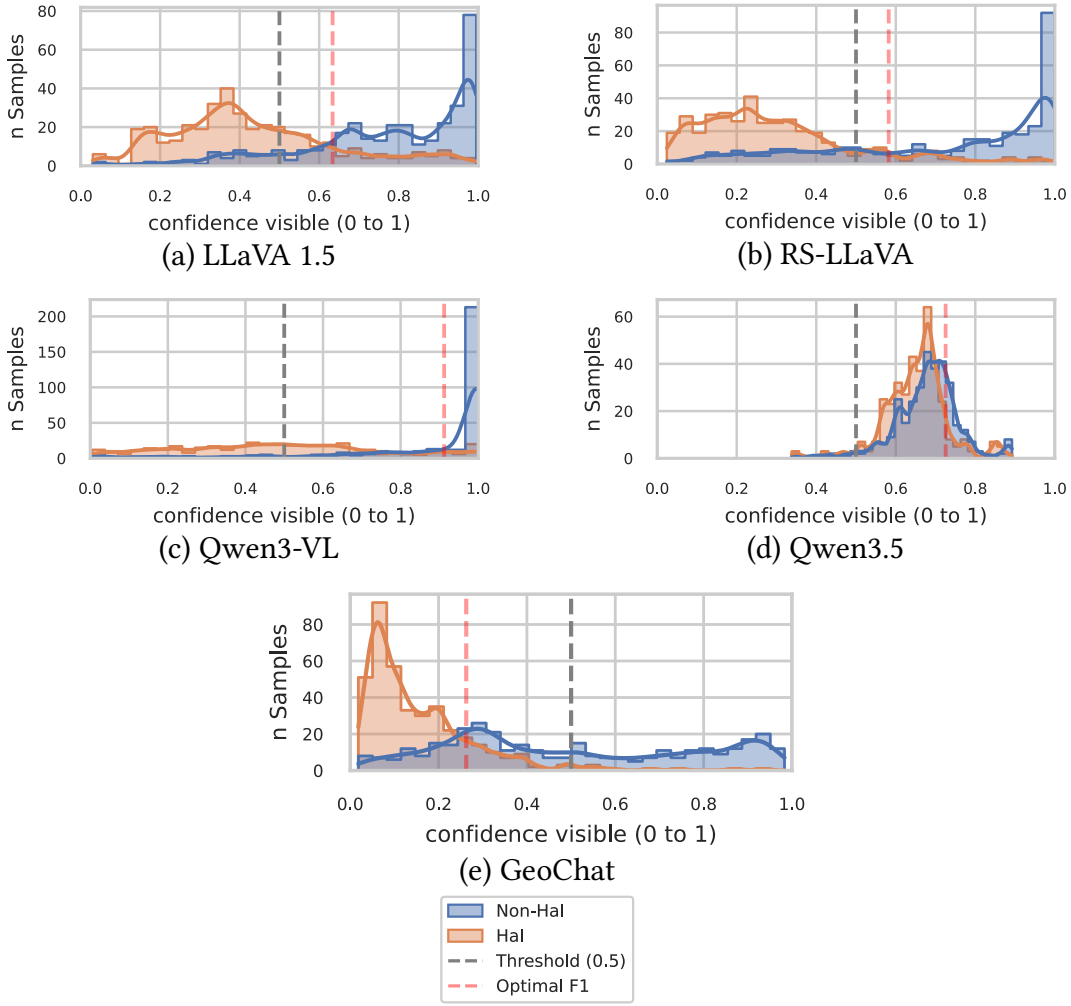


Figure A.1: Confidence Distributions for Hallucination Detection Performance using Mulitple Choice Questions on RS-POPE.

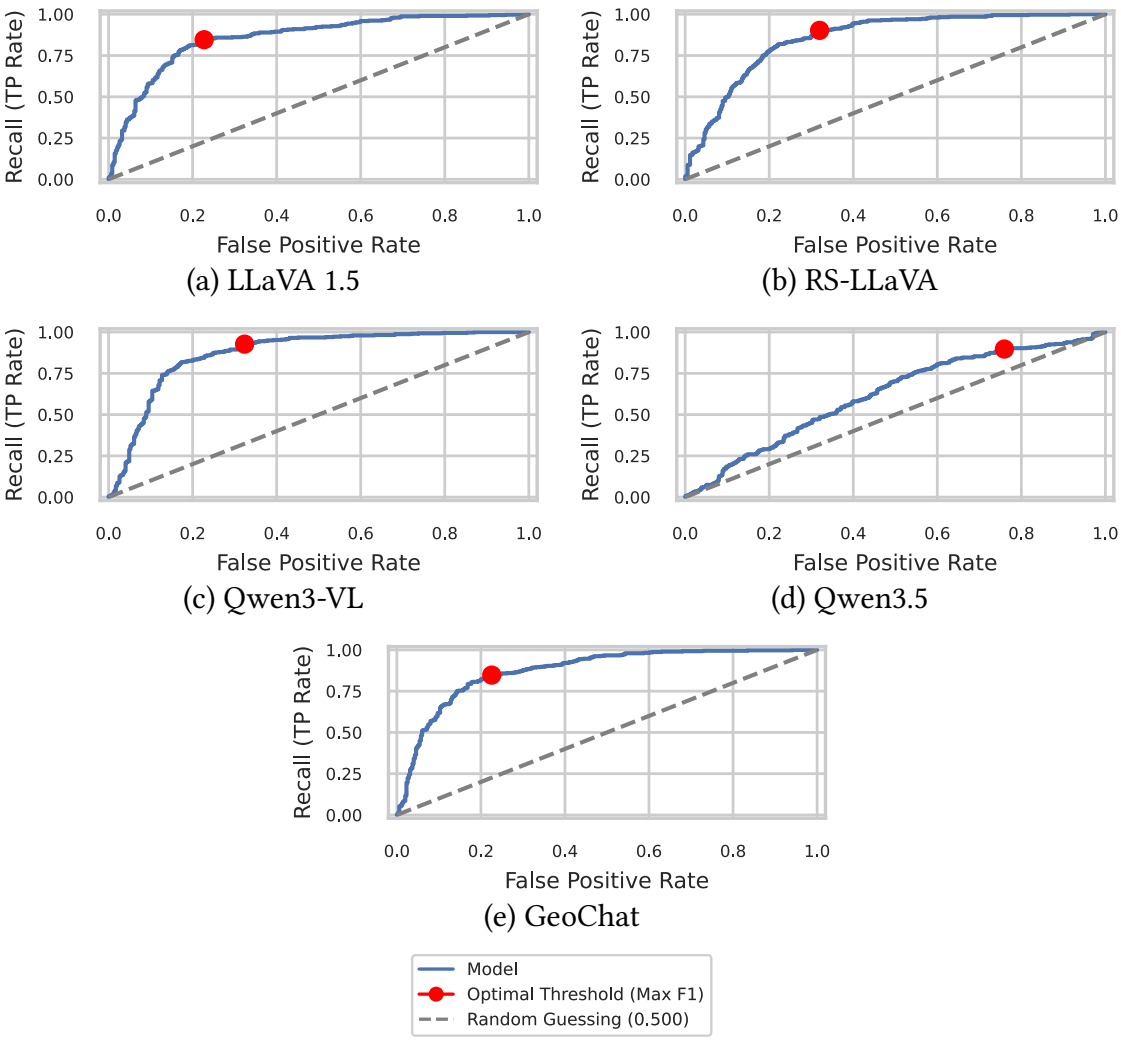


Figure A.2: ROC curves showing hallucination detection performance using multiple choice questions on RS-POPE.

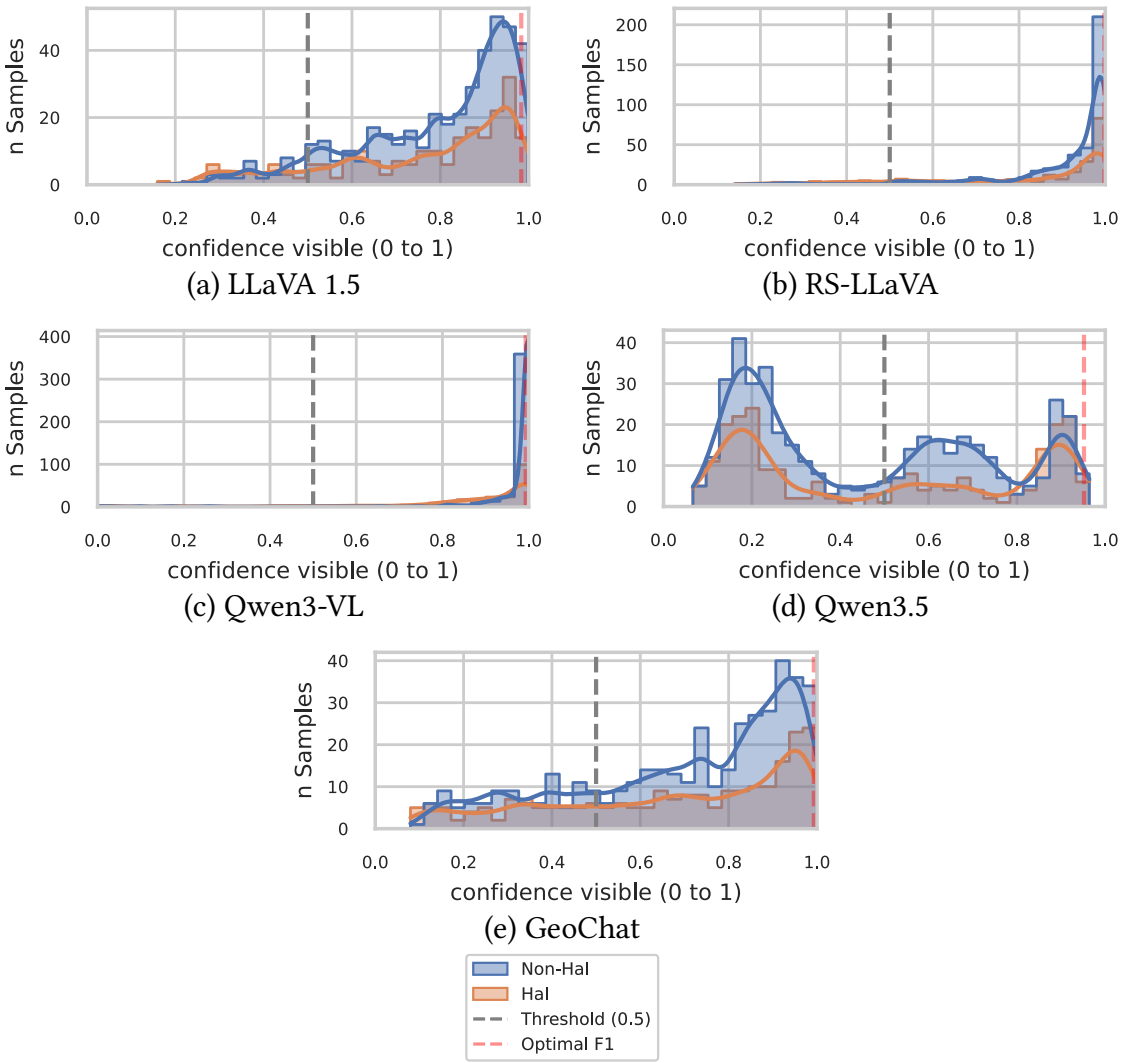


Figure A.3: Confidence Distributions for Hallucination Detection Performance using Mulitple Choice Questions on LID.

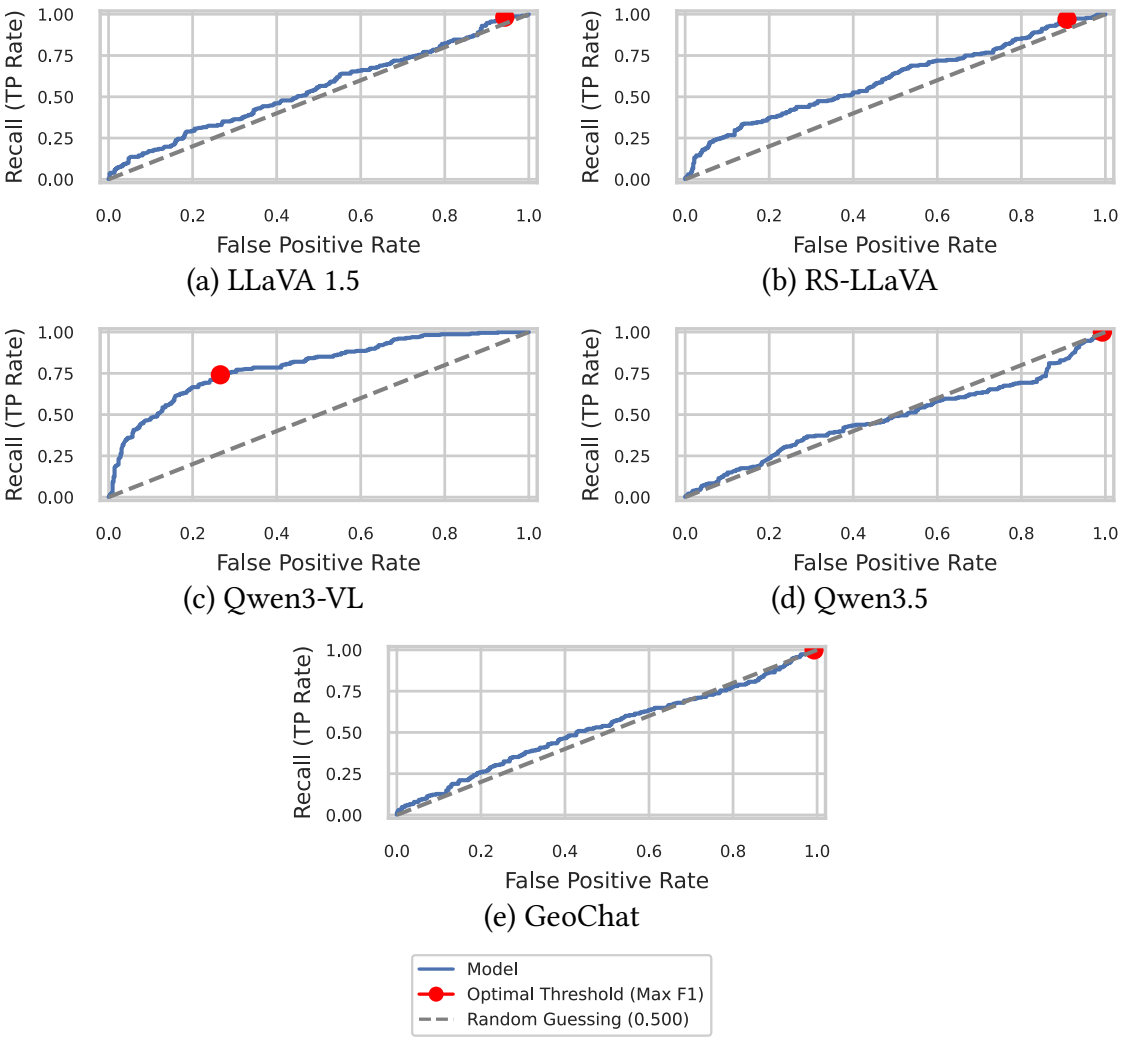
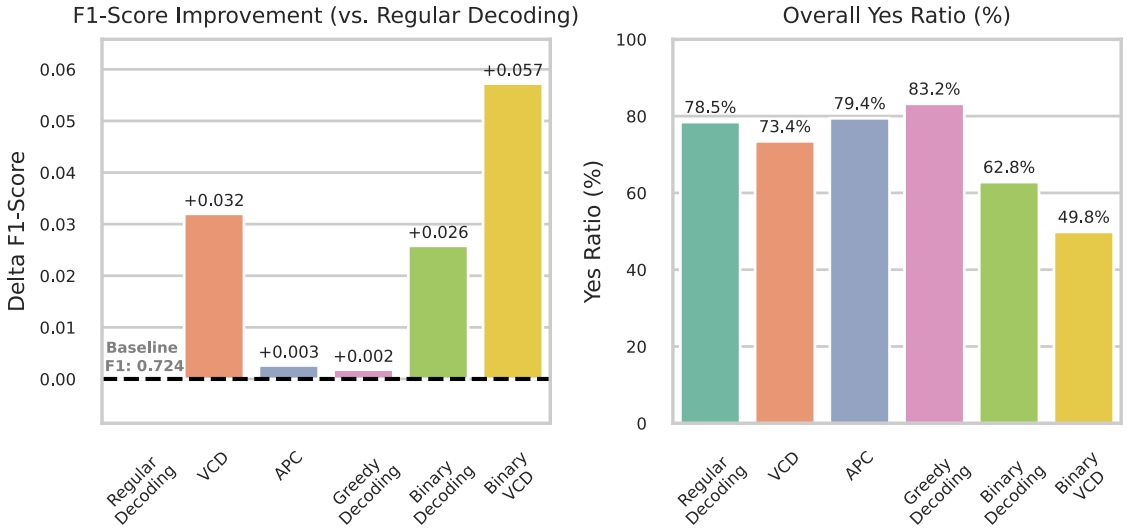


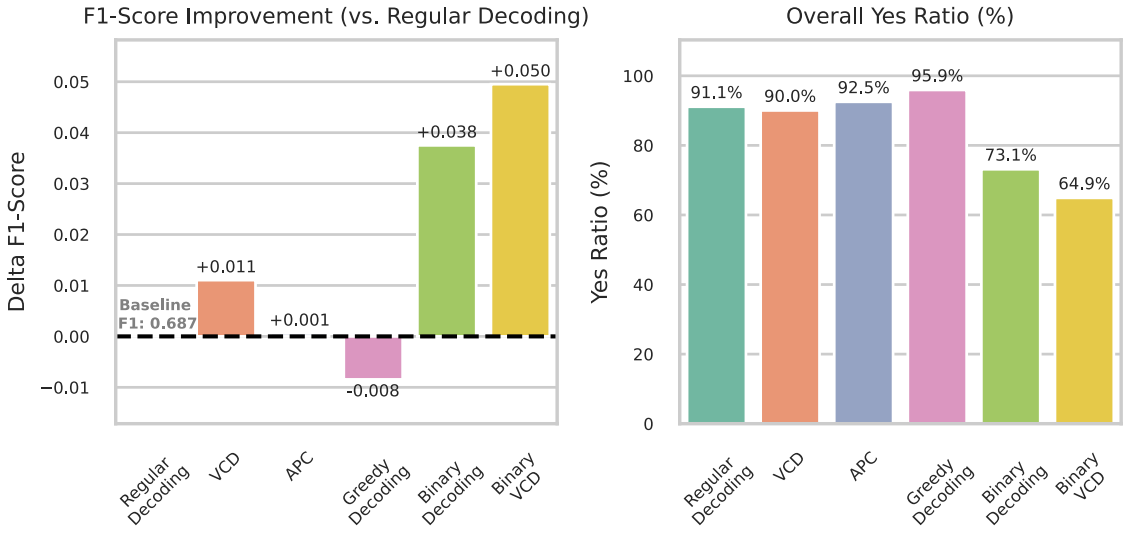
Figure A.4: ROC curves showing hallucination detection performance using multiple choice questions on LID.

A.2. Isolating the influence of contrastive decoding: All Models

The plots in this section show the effect of different decoding strategies in order to isolate the effect of contrastive decoding on the remaining models not shown in Section 5.2.2: GeoChat, Qwen3-VL, Qwen3.5 and RS-LLaVA.

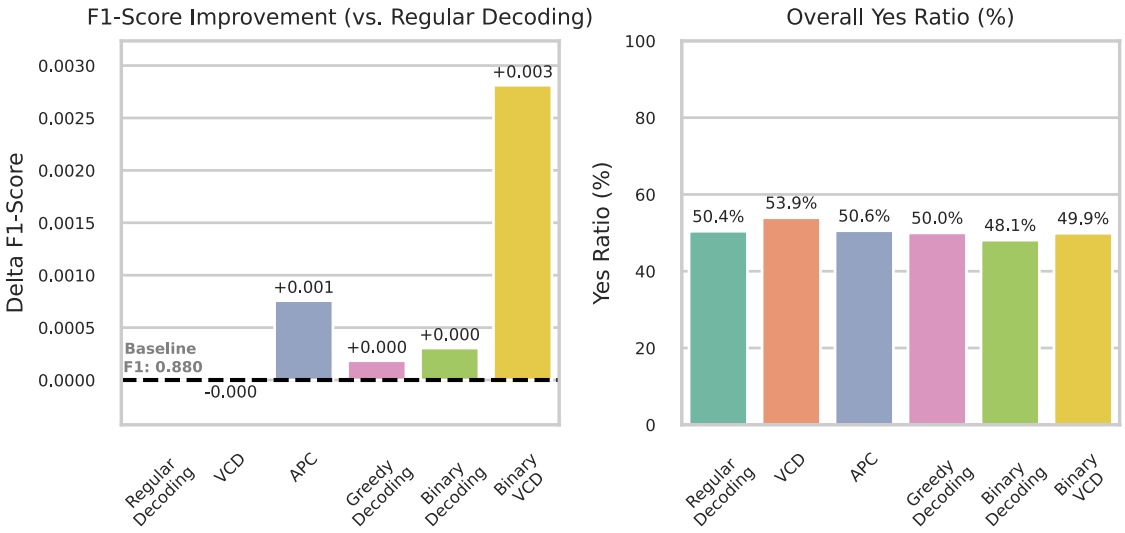


(a) POPE

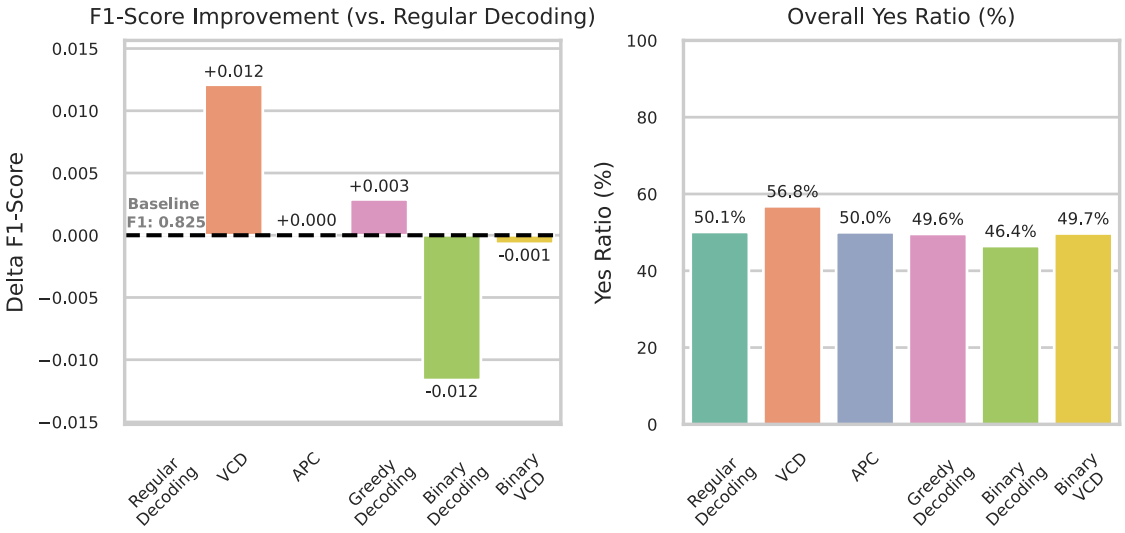


(b) RS-POPE

Figure A.5: Hallucination detection performance and Yes Ratios of different decoding strategies on POPE and RS-POPE with GeoChat.

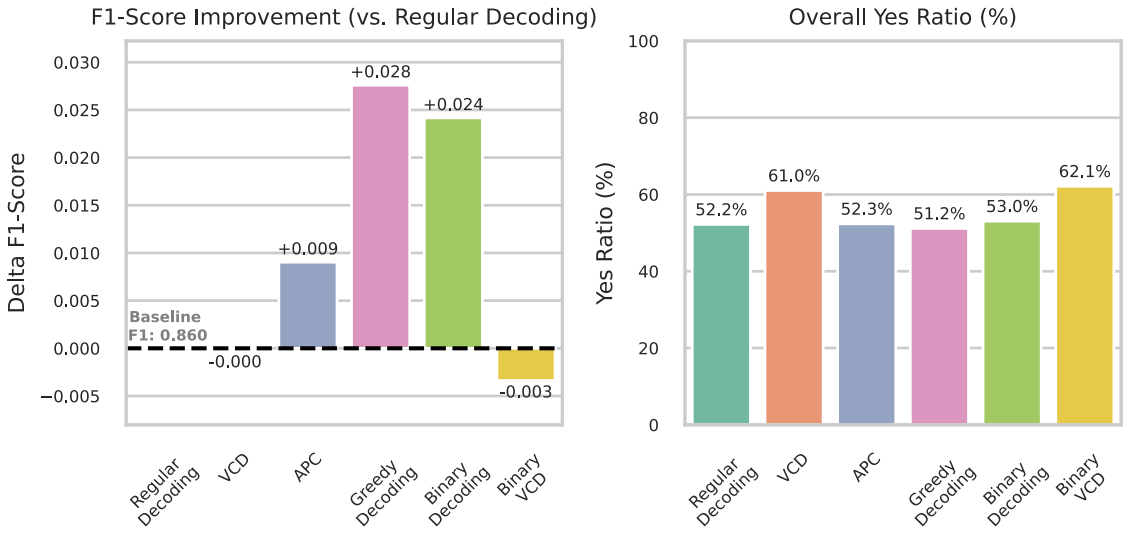


(a) POPE

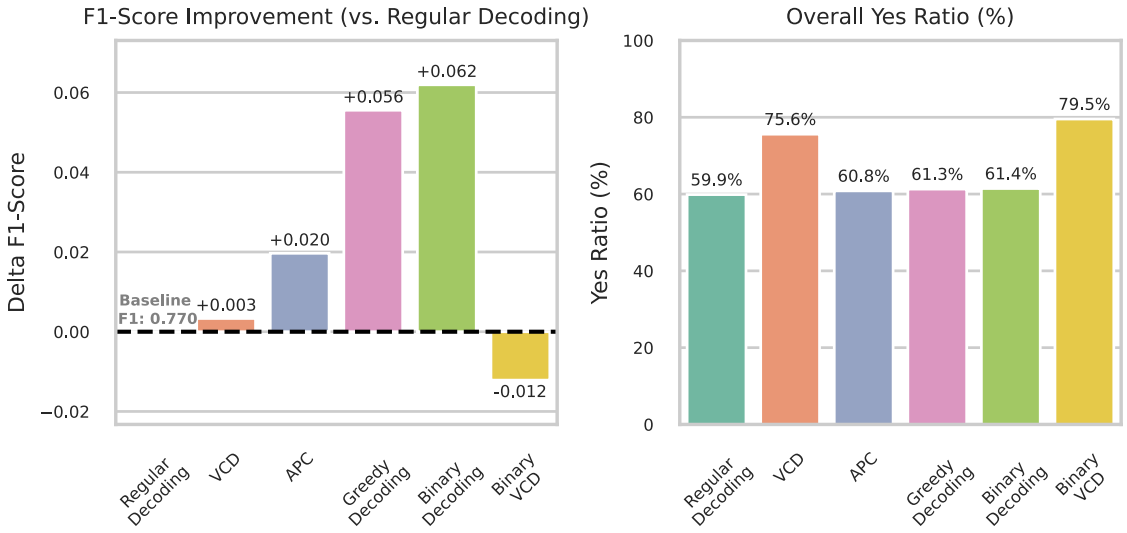


(b) RS-POPE

Figure A.6: Hallucination detection performance and Yes Ratios of different decoding strategies on POPE and RS-POPE with Qwen3-VL.

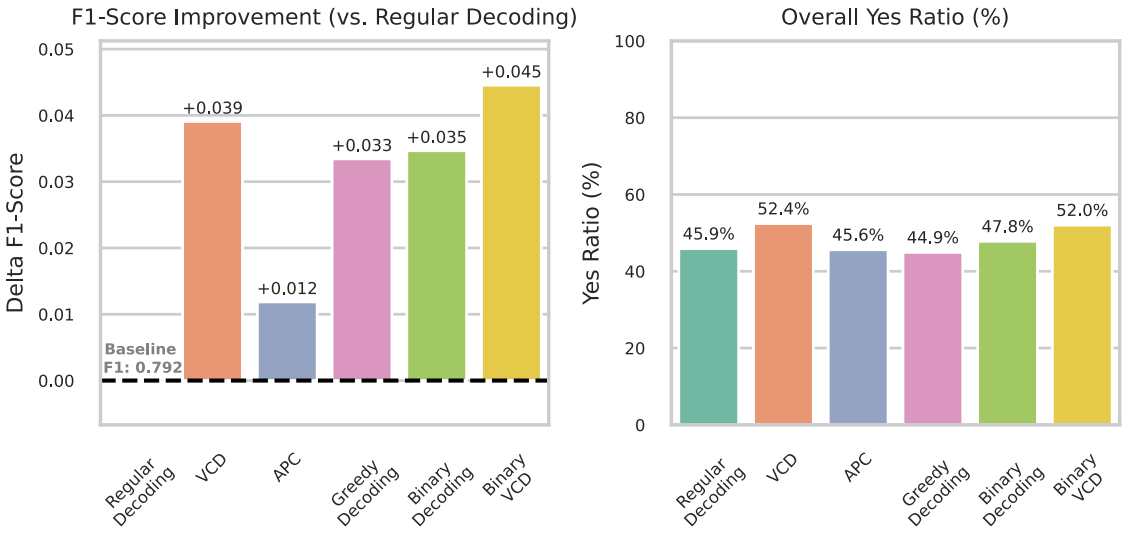


(a) POPE

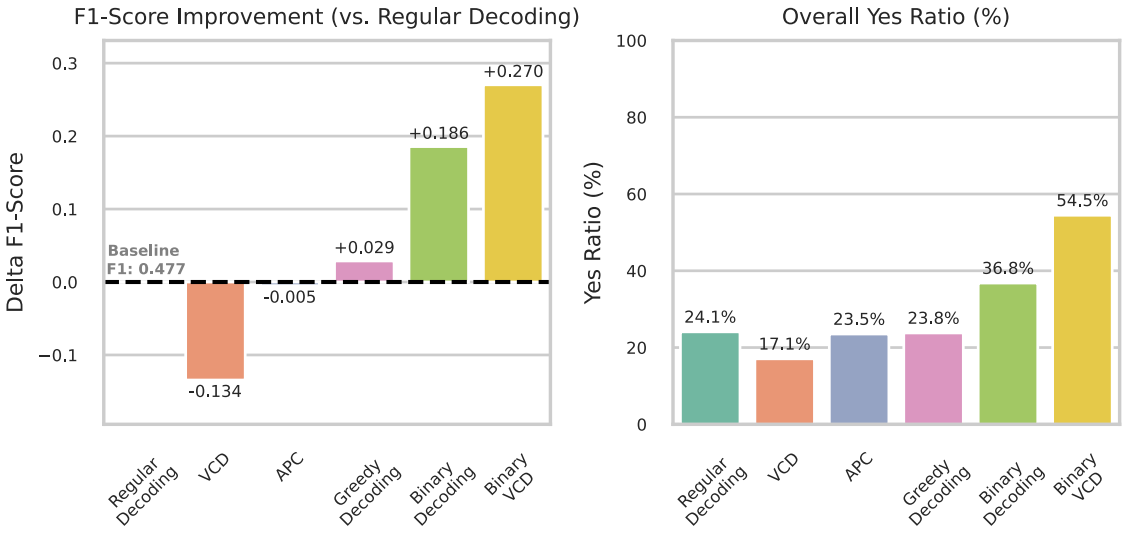


(b) RS-POPE

Figure A.7: Hallucination detection performance and Yes Ratios of different decoding strategies on POPE and RS-POPE with Qwen3.5.



(a) POPE



(b) RS-POPE

Figure A.8: Hallucination detection performance and Yes Ratios of different decoding strategies on POPE and RS-POPE with RS-LLaVA.